Athlete Housing Search and Residence Approval System

Semester Group Project — Final Submission

Group 1

David Eccles School of Business, University of Utah

IS 4430 : System Analysis and Design

Ellie Choi, Patric Hopes, Nicholas Pardon, Michael Vargas

July 30, 2026

Table of Contents

1. **Milestone 1 — System Requirements**
2. **Milestone 2 — User Interface Design**

    Interactive Prototype and Screen Inventory
3. **Milestone 3 — Data Flow Diagrams and Process Descriptions**

    Context, Level 0, and Level 1 Diagrams (Figures 3.1 – 3.12)

    Level 0 and Level 1 Process Descriptions
4. **Milestone 4 — Entity Relationship Diagram**

    Final ER Diagram, Trigger Summary, and Attribute Domain Catalog
5. **Data Tables & Activity Diagrams**

    Data Tables with Primary and Foreign Keys

    Activity Diagrams for the Level 0 Processes

A. **Appendix A — Diagram Source Code (Mermaid)**

1

Milestone 1

System Requirements

Assumptions

1. Athletes will log in using their University UID.
2. Housing budgets can be entered manually or imported from an approved university data source.
3. Athletic Department Reviewer is a broad role that can include coaches, managers, or approved athletic staff.
4. The admin model is tiered, but kept simple for this class project.
5. The system will search approved rental and residential websites only.
6. Listing data may come from APIs, feeds, or other approved collection methods.
7. Approval comments and rejection reasons are optional.
8. Security requirements are included at a basic systems analysis level.

Requirements List

9. REQ-AUTH-001: Athlete UID Login and Session Security
10. REQ-PROFILE-002: Athlete Profile Management11. REQ-BUDGET-003: Housing Budget Management
12. REQ-ROLE-004: Tiered Role and Permission Management
13. REQ-SEC-005: Student Data Storage and Security
14. REQ-SITE-006: Approved Rental Website Management
15. REQ-LISTING-007: Automated Residence Listing Collection
16. REQ-DATA-008: Listing Data Quality and Duplicate Handling
17. REQ-SEARCH-009: Residence Search and Filtering
18. REQ-DETAIL-010: Residence Details and Budget Comparison
19. REQ-SUBMIT-011: Residence Selection Submission and Review Workflow
20. REQ-NOTIFY-012: Email Notification and Status Tracking

Full Requirements

REQ-AUTH-001: Athlete UID Login, Logout, and Session Security

Requirement ID: REQ-AUTH-001

Title: Athlete UID Login, Logout, and Session Security

Version: 1.0 Date: May 30, 2026 Author: Nick Pardon Status: Draft Priority: High Must Have

1. Description

The system shall require athletes and athletic department users to log in with a valid University

UID before using protected housing system features. The system shall also allow users to log out and shall protect active sessions from basic unauthorized access.

2. Rationale / Business Value

- Limits access to approved university users.
- Connects each athlete to the correct profile, budget, residence submissions, and status history.
- Supports different dashboards for athletes, coaches, managers, and system admins.
- Helps protect student housing information from users who should not see it.

3. Functional Requirements

- The system shall display a login page before protected housing features are shown.
- The system shall require a valid University UID for athlete and staff access.
- The system shall reject login attempts when the UID is not found, inactive, or not allowed to use the housing system.
- The system shall route athletes to the athlete dashboard after login.
- The system shall route staff users to the correct dashboard based on their assigned role.
- The system shall provide a logout option on protected pages.
- The system shall end the active session when a user logs out.
- The system shall block protected pages after logout unless the user logs in again.

4. Non-Functional Requirements

- The login page shall load in less than 3 seconds under normal use.
- The system shall time out inactive sessions after a set period of inactivity.
- The system shall not display protected student or housing data before login.
- The system shall record failed login attempts for basic security review.
- The system shall show a simple error message for failed login without revealing private user details.

5. Dependencies

- University UID authentication source or a class project mock UID list.
- Tiered role and permission management requirement REQ-ROLE-004.
- Student data storage and security requirement REQ-SEC-005.

6. Acceptance Criteria

- Given an athlete has a valid UID, when the athlete logs in, then the system opens the athlete dashboard.
- Given a user enters an invalid UID, when the user tries to log in, then the system rejects the login and shows an error message.•
- Given a staff user logs in, when the UID is verified, then the system opens the correct staff dashboard for that user's role.

Given a logged in user clicks logout, when logout is completed, then the system ends the session and returns the user to the login page.

Given a logged out user tries to open a protected page, when the page loads, then the system requires login again.

REQ-PROFILE-002: Athlete Profile Management

Requirement ID: REQ-PROFILE-002

Title: Athlete Profile Management

Version: 1.0 Date: May 30, 2026 Author: Nick Pardon Status: Draft Priority: High Must Have

1. Description

The system shall store athlete profile information needed for housing search, budget comparison, residence submission, and athletic department review.

2. Rationale / Business Value

- Keeps athlete housing records organized in one place.
- Gives reviewers enough information to review a residence request.
- Connects each athlete to their sport, contact email, assigned budget, and current housing status.
- Reduces confusion by showing athletes and staff the same basic profile data.

3. Functional Requirements

- The system shall store athlete name, UID, sport, university email, class year if available, and housing status.
- The system shall allow athletes to view their own profile information.
- The system shall allow approved staff to view athlete profiles needed for housing review.
- The system shall allow a System Admin to correct profile data when needed.
- The system shall connect each athlete profile to that athlete's housing budget, saved listings, submitted requests, and status history.
- The system shall show the athlete's current residence request status on the athlete profile or dashboard.

4. Non-Functional Requirements

- Athletes shall only be able to view their own profile.
- Profile changes shall be saved with the user and date of the change.
- Profile pages shall use clear labels for student information.
- The system shall not show athlete profile data to users who do not have permission.24. The profile page shall be simple enough for an athlete or staff member to understand without training.

5. Dependencies

- Athlete UID login, logout, and session security requirement REQ-AUTH-001.
- Housing budget management requirement REQ-BUDGET-003.
- Student data storage and security requirement REQ-SEC-005.

6. Acceptance Criteria

- Given an athlete is logged in, when the athlete opens the profile page, then the system shows only that athlete's profile.
- Given an approved staff user opens an athlete record, when the staff user has permission, then the system shows the profile information needed for review.

- Given a System Admin updates profile data, when the update is saved, then the system records the change in history.
- Given a user without permission tries to open another athlete's profile, when the page loads, then the system blocks access.

REQ-BUDGET-003: Housing Budget Management

Requirement ID: REQ-BUDGET-003

Title: Housing Budget Management

Version: 1.0 Date: May 30, 2026 Author: Nick Pardon Status: Draft Priority: High Must Have

1. Description

The system shall store each athlete's assigned housing budget and use it to calculate whether a residence is within budget or requires out of pocket payment from the athlete.

2. Rationale / Business Value

- Helps athletes understand what the school will cover.
- Helps the athletic department control housing spending.
- Makes residence approval decisions easier to review.
- Shows athletes when a housing choice may cost more than the school budget allows.

3. Functional Requirements

- The system shall allow a System Admin to manually enter or edit an athlete housing budget.
- The system shall support importing housing budgets from an approved university data source.
- The system shall show the athlete their assigned housing budget.
- The system shall compare listing rent to the athlete's assigned budget.29. The system shall calculate estimated out of pocket cost when rent is above the assigned budget.
- The system shall show budget information to approved reviewers during request review.
- The system shall keep a history of budget changes, including old value, new value, user, and date.

4. Non-Functional Requirements

- Only approved staff users shall be able to edit budgets.
- Budget values shall be stored in a consistent dollar format.
- Budget changes shall be logged for review.
- The budget display shall be easy for athletes to read.
- The system shall not allow an athlete to change their own assigned budget.

5. Dependencies

- Athlete profile management requirement REQ-PROFILE-002.
- Residence details and budget comparison requirement REQ-DETAIL-010.
- Student data storage and security requirement REQ-SEC-005.
- Optional university budget import source.

6. Acceptance Criteria

- Given an athlete has an assigned budget, when the athlete opens the dashboard, then the budget is shown.
- Given a listing is above the athlete budget, when the athlete views the listing, then the system shows the estimated out of pocket amount.
- Given a System Admin changes a budget, when the change is saved, then the system records the change in history.
- Given an athlete tries to edit their own budget, when the athlete opens budget information, then the system does not allow editing.
- Given budget data is imported from an approved source, when the import is completed, then the athlete's current budget is updated or confirmed.

REQ-ROLE-004: Tiered Role and Permission Management

Requirement ID: REQ-ROLE-004

Title: Tiered Role and Permission Management

Version: 1.0 Date: May 30, 2026 Author: Nick Pardon Status: Draft Priority: High Must Have

1. Description

The system shall use simple tiered roles so athletes, coaches, managers, and system admins only access the features they are allowed to use.

2. Rationale / Business Value

- Keeps athlete data private.

Prevents users from changing records they should not control.

Supports a realistic athletic department review workflow.

3. Functional Requirements

- The system shall support the Athlete role for search, residence selection, request submission, and status viewing.
- The system shall support the Athletic Coach role for viewing assigned athlete submissions and adding comments or recommendations.
- The system shall support the Athletic Manager role for approving or rejecting residence requests.
- The system shall support the System Admin role for managing users, budgets, approved websites, and system settings.
- The system shall allow a System Admin to assign or change user roles.

4. Non-Functional Requirements

- Permission checks shall happen before protected pages and actions are shown.
- Users shall not be able to access another role's features by changing a URL.
- Role changes shall be logged in system history.

5. Dependencies

- • Athlete UID login and session security requirement REQ-AUTH-001.

Student data storage and security requirement REQ-SEC-005.

6. Acceptance Criteria

- Given an athlete is logged in, when the athlete opens the system, then the athlete can search and submit housing choices but cannot approve requests.
- Given an Athletic Manager is logged in, when the manager opens a pending request, then the manager can approve or reject it.
- Given a System Admin is logged in, when the admin opens role settings, then the admin can change a user's role.

REQ-SEC-005: Student Data Storage and Security

Requirement ID: REQ-SEC-005

Title: Student Data Storage and Security

Version: 1.0 Date: May 30, 2026 Author: Nick Pardon Status: Draft Priority: High Must Have

1. Description

The system shall store student housing data securely and limit access to users with a valid reason to view or change that data.

2. Rationale / Business Value

- Housing choices, budgets, and student records are sensitive information.

The athletic department needs a reliable record of submissions and decisions.

Basic security supports trust in the system without making this project overly technical.

3. Functional Requirements

- The system shall store athlete profiles, budgets, listings, residence requests, review decisions, comments, and status history.
- The system shall separate athlete data from staff management data using clear tables or data stores.
- The system shall allow athletes to view only their own profile, budget, submissions, and status history.
- The system shall allow approved staff to view only records needed for their role.
- The system shall keep an audit history for budget changes, role changes, submissions, approvals, and rejections.

4. Non-Functional Requirements

- The system shall protect stored student data from unauthorized access.
- The system shall not display sensitive student data on public pages.
- The system shall keep audit records readable enough for a staff review.

5. Dependencies

- Tiered role and permission management requirement REQ-ROLE-004.

Housing budget management requirement REQ-BUDGET-003.

Residence selection submission and review workflow requirement REQ-SUBMIT-011.

6. Acceptance Criteria

- Given an athlete logs in, when the athlete opens status history, then the system shows only that athlete's records.
- Given a budget is changed, when the change is saved, then the system stores an audit record.
- Given an unauthenticated visitor opens a protected page, when the page loads, then the system blocks access.

REQ-SITE-006: Approved Rental Website Management

Requirement ID: REQ-SITE-006

Title: Approved Rental Website Management

Version: 1.0 Date: May 30, 2026 Author: Nick Pardon Status: Draft Priority: High Must Have

1. Description

The system shall allow authorized admins to manage the rental and residential websites that the system is allowed to search.**2. Rationale / Business Value**

- Lets the athletic department control where listings come from.
- Keeps the listing search focused on approved sources.
- Makes it possible to add or remove sources over time.

3. Functional Requirements

- The system shall allow a System Admin to add an approved website with a name, URL, source type, and notes.
- The system shall allow a System Admin to edit approved website details.
- The system shall allow a System Admin to disable an approved website without deleting its history.
- The system shall allow a System Admin to remove a website if it was entered by mistake.
- The system shall show whether each approved website is active or inactive.

4. Non-Functional Requirements

- Website management pages shall only be available to System Admin users.
- The system shall validate that a website URL is in a usable format before saving it.
- Changes to approved websites shall be logged with user and date.

5. Dependencies

- Tiered role and permission management requirement REQ-ROLE-004.

Automated residence listing collection requirement REQ-LISTING-007.

6. Acceptance Criteria

- Given a System Admin enters Zillow as an approved website, when the admin saves it, then the website appears as an active source.

- Given an approved website is disabled, when listing collection runs, then the system does not collect new listings from that source.
- Given a non-admin user opens website management, when the page loads, then access is denied.

REQ-LISTING-007: Automated Residence Listing Collection

Requirement ID: REQ-LISTING-007

Title: Automated Residence Listing Collection

Version: 1.0 Date: May 30, 2026 Author: Nick Pardon Status: Draft Priority: High Must Have

1. Description

The system shall collect residence listings from approved rental and residential websites so athletes can search current housing options near campus.

2. Rationale / Business Value

- Saves athletes time because listings are gathered in one place.

Supports the project goal of automatically searching rental and residential websites.

Gives the athletic department a consistent view of where residence choices came from.

3. Functional Requirements

- The system shall collect listings from active approved websites.
- The system shall store listing source, source URL, property type, rent, address or area, availability date, and basic details when available.
- The system shall mark old listings as inactive when they are no longer found or are manually removed.
- The system shall allow staff to view when a listing was last updated.

4. Non-Functional Requirements

- The collection process shall continue running for other approved sources if one source fails.
- The system shall store enough source information so a reviewer can verify where a listing came from.
- The system shall not collect listings from websites that are not approved by an admin.

5. Dependencies

- Approved rental website management requirement REQ-SITE-006.

Listing data quality and duplicate handling requirement REQ-DATA-008.

6. Acceptance Criteria

- Given an approved website is active, when the collection process runs, then matching listings from that source are saved or updated.
- Given a website source fails, when collection runs, then the system records the issue and continues with other active sources.

- Given a listing is collected, when a reviewer opens it, then the system shows the source website and source URL.

REQ-DATA-008: Listing Data Quality and Duplicate Handling

Requirement ID: REQ-DATA-008

Title: Listing Data Quality and Duplicate Handling

Version: 1.0 Date: May 30, 2026 Author: Nick Pardon Status: Draft Priority: Medium Should Have

1. Description

The system shall clean, label, and manage collected listing data so athletes and reviewers can understand which listings are current and useful.

2. Rationale / Business Value

- Rental websites may show incomplete or duplicate listings.

Athletes need search results that are clear and not filled with repeated records.

Reviewers need to know whether a listing is current enough to approve.

3. Functional Requirements

- The system shall avoid showing duplicate listings when the same residence appears from more than one source.
- The system shall show when listing data is missing or unavailable.
- The system shall store the date a listing was collected or last updated.
- The system shall allow staff to mark a listing inactive if it is no longer useful.

4. Non-Functional Requirements

- The system shall keep original source data when possible for review.
- The system shall label missing values clearly instead of showing confusing blanks.
- The duplicate check shall use simple matching fields such as address, rent, and source link.

5. Dependencies

- Automated residence listing collection requirement REQ-LISTING-007.

Residence search and filtering requirement REQ-SEARCH-009.

6. Acceptance Criteria

- Given the same listing appears from two sources, when the system detects the duplicate, then search results show one main listing record.
- Given a listing is missing pet information, when the athlete views it, then the system shows that pet information is unavailable.
- Given a staff user marks a listing inactive, when athletes search, then the inactive listing is not shown in normal results.

REQ-SEARCH-009: Residence Search and Filtering

Requirement ID: REQ-SEARCH-009

Title: Residence Search and Filtering

Version: 1.0 Date: May 30, 2026 Author: Nick Pardon Status: Draft Priority: High Must Have

1. Description

The system shall allow athletes to search and filter available residences using as many logical housing filters as the system can support.

2. Rationale / Business Value

- Helps athletes find housing that fits their needs.
- Makes the system more useful than a basic list of links.• Supports better residence choices before the athlete submits for approval.

3. Functional Requirements

- The system shall allow athletes to search listings by keyword or location.
- The system shall allow filtering by rent amount and whether the rent fits within the athlete budget.
- The system shall allow filtering by distance from campus.
- The system shall allow filtering by apartment, house, or other supported residence type.
- The system shall allow filtering by bedrooms, bathrooms, availability date, lease length, furnished status, parking, pets allowed, utilities included, transit access, and source website when data is available.
- The system shall allow athletes to sort results by rent, distance, newest listing, and budget fit.

4. Non-Functional Requirements

- Search results shall load in a reasonable time for a class project prototype.
- Filters shall not hide all results without giving the athlete a clear reset option.
- The system shall show when some filter data is unavailable for a listing.

5. Dependencies

- Automated residence listing collection requirement REQ-LISTING-007.

Housing budget management requirement REQ-BUDGET-003.

Listing data quality and duplicate handling requirement REQ-DATA-008.

6. Acceptance Criteria

- Given an athlete filters by rent under their budget, when results load, then the system shows residences that fit the selected rent filter.
- Given an athlete filters by house, when results load, then the system shows house listings when available.
- Given no listings match the filters, when results load, then the system displays a clear no results message and a reset option.

REQ-DETAIL-010: Residence Details and Budget Comparison

Requirement ID: REQ-DETAIL-010

Title: Residence Details and Budget Comparison

Version: 1.0 Date: May 30, 2026 Author: Nick Pardon Status: Draft Priority: Medium Should Have

1. Description

The system shall show detailed residence information and compare the residence cost against the athlete's housing budget.**2. Rationale / Business Value**

- Helps athletes understand what they are selecting before they submit.
- Helps athletes see whether they may owe money out of pocket.
- Helps reviewers make a decision using the same listing details.

3. Functional Requirements

- The system shall display listing name or address, rent, property type, source website, and source link.
- The system shall display bedrooms, bathrooms, lease length, availability date, parking, utilities, furnished status, and pet rules when available.
- The system shall display listing photos when the source provides photos.
- The system shall show estimated distance from campus when location data is available.
- The system shall compare rent to the athlete's budget and show estimated out of pocket cost.

4. Non-Functional Requirements

- The details page shall clearly show when information is missing from a listing source.
- The system shall not let athletes edit listing details collected from source websites.
- The budget comparison shall use the most recent budget stored for the athlete.

5. Dependencies

- Housing budget management requirement REQ-BUDGET-003.
- Automated residence listing collection requirement REQ-LISTING-007.
- Residence search and filtering requirement REQ-SEARCH-009.

6. Acceptance Criteria

- Given an athlete opens a listing, when the details page loads, then the system shows rent, source, property type, and available housing details.
- Given the rent is higher than the athlete budget, when the listing loads, then the system shows the estimated out of pocket amount.
- Given a listing does not have photos, when the details page loads, then the system still shows the available text details.

REQ-SUBMIT-011: Residence Selection Submission and Review Workflow

Requirement ID: REQ-SUBMIT-011

Title: Residence Selection Submission and Review Workflow

Version: 1.0 Date: May 30, 2026 Author: Nick Pardon Status: Draft Priority: High Must Have

1. Description

The system shall allow an athlete to submit a selected residence and allow approved athletic department users to approve or reject that choice.

2. Rationale / Business Value

- Creates a formal request for the athletic department to review.
- Keeps the selected residence, athlete, budget, notes, and decision connected in one record.
- Supports the main workflow described in the project scenario.

3. Functional Requirements

- The system shall allow an athlete to select a residence from the listing details page.
- The submission form shall include athlete name, UID, selected residence, rent, source website, budget, estimated out of pocket cost, and optional athlete notes.
- The system shall require the athlete to confirm that any amount above the housing budget must be paid out of pocket.
- The system shall save submitted requests with a Pending Review status.
- The system shall allow approved reviewers to approve or reject a pending request.
- The system shall allow optional approval comments and optional rejection reasons.
- The system shall allow a rejection reason to be left blank.

4. Non-Functional Requirements

- Only users with review permissions shall approve or reject requests.
- Review decisions shall be recorded in audit history.
- The review page shall show enough information for a decision without forcing the reviewer to search multiple screens.

5. Dependencies

- Residence details and budget comparison requirement REQ-DETAIL-010.
- Tiered role and permission management requirement REQ-ROLE-004.
- Email notification and status tracking requirement REQ-NOTIFY-012.

6. Acceptance Criteria

- Given an athlete selects a residence, when the athlete submits the form, then the system creates a Pending Review request.
- Given a request is pending, when an Athletic Manager approves it, then the request status changes to Approved.
- Given a request is pending, when an Athletic Manager rejects it without a reason, then the request status changes to Rejected and the blank reason is allowed.

REQ-NOTIFY-012: Email Notification and Status Tracking

Requirement ID: REQ-NOTIFY-012

Title: Email Notification and Status Tracking

*Version: 1.0 Date: May 30, 2026 Author: Nick Pardon Status: Draft Priority: High Must Have**1. Description** The system shall notify athletes by email when a residence request is approved or rejected and shall allow athletes to view request status inside the system.*

2. Rationale / Business Value

- Keeps athletes informed after they submit a residence choice.
- Reduces confusion about whether a request is still waiting for review.
- Creates a record of status changes and decision communication.

3. Functional Requirements

- The system shall send an email to the athlete when a request is approved.
- The system shall send an email to the athlete when a request is rejected.
- The email shall include request status, selected residence, decision date, and reviewer comments or rejection reason if provided.
- The system shall show current request status on the athlete dashboard.
- The system shall store status history for Submitted, Pending Review, Approved, Rejected, and Cancelled when used.

4. Non-Functional Requirements

- Email notification shall be sent shortly after the decision is saved.
- If email delivery fails, the request status shall still be saved and visible in the athlete dashboard.
- Status history shall be viewable by approved staff users.

5. Dependencies

- Residence selection submission and review workflow requirement REQ-SUBMIT-011.
- Student data storage and security requirement REQ-SEC-005.
- Email service or mock email service for the class project.

6. Acceptance Criteria

- Given a request is approved, when the reviewer saves the decision, then the athlete receives an approval email and the dashboard shows Approved.
- Given a request is rejected with a reason, when the reviewer saves the decision, then the athlete receives a rejection email with the reason included.
- Given an email fails to send, when the decision is saved, then the system still saves the status and lets the athlete view it in the system

2

Milestone 2

User Interface Design

User Interface Design

Interactive prototype: https://is4430gp.netlify.app/

The full user interface was built as a clickable web prototype covering all 20 screens. The inventory below maps every screen to the requirement that motivates it, followed by a capture of each screen taken from the live prototype.

Screen Inventory by Role

ID	Screen	Role	Source Req	Key elements
S-01	Login (UID + password)	All	REQ-AUTH-001	UID field, password, error state
S-02	Access Denied	All	REQ-ROLE-004 §3.6, REQ-SEC-005	Clean "no details" message, return-to-login
A-01	Athlete Dashboard	Athlete	REQ-NOTIFY-012 FR-4	Active request status, recent activity, quick search
A-02	Residence Search + Filter	Athlete	REQ-SEARCH-009	Filter rail, sort, result cards, budget-fit badge
A-03	Residence Details + Budget Comparison	Athlete	REQ-DETAIL-010	Photos, attributes, source link, out-of-pocket calc
A-04	Residence Submission Form	Athlete	REQ-SUBMIT-011	Prefilled summary, out-of-pocket confirm checkbox
A-05	Request Status / History	Athlete	REQ-NOTIFY-012 FR-5	Lifecycle timeline, cancel-pending action
A-06	Athlete Profile *(new)	Athlete	REQ-ROLE-004 §3.1	Profile fields, assigned budget, status history
C-01	Coach Dashboard	Coach	REQ-ROLE-004 §3.2	Assigned-athlete submissions queue
C-02	Submission Review + Comment	Coach	REQ-ROLE-004 §3.2	Read-only request, add recommendation (no decision)
C-03	Assigned Athlete Profile/Budget	Coach	REQ-ROLE-004 §3.2	Scoped athlete view
M-01	Manager Dashboard	Manager	REQ-ROLE-004 §3.3	All-pending queue, filters
M-02	Request Review + Decision	Manager	REQ-SUBMIT-011 FR-6/7	Single decision view, approve/reject, comment/reason
M-03	Request Status History (all)	Manager	REQ-ROLE-004 §3.3	Full managed-athlete history
D-01	Admin Dashboard / Settings	Admin	REQ-ROLE-004 §3.4	System settings, collection health summary
D-02	User + Role Management	Admin	REQ-ROLE-004 §3.5	User table, role dropdown, change-history note

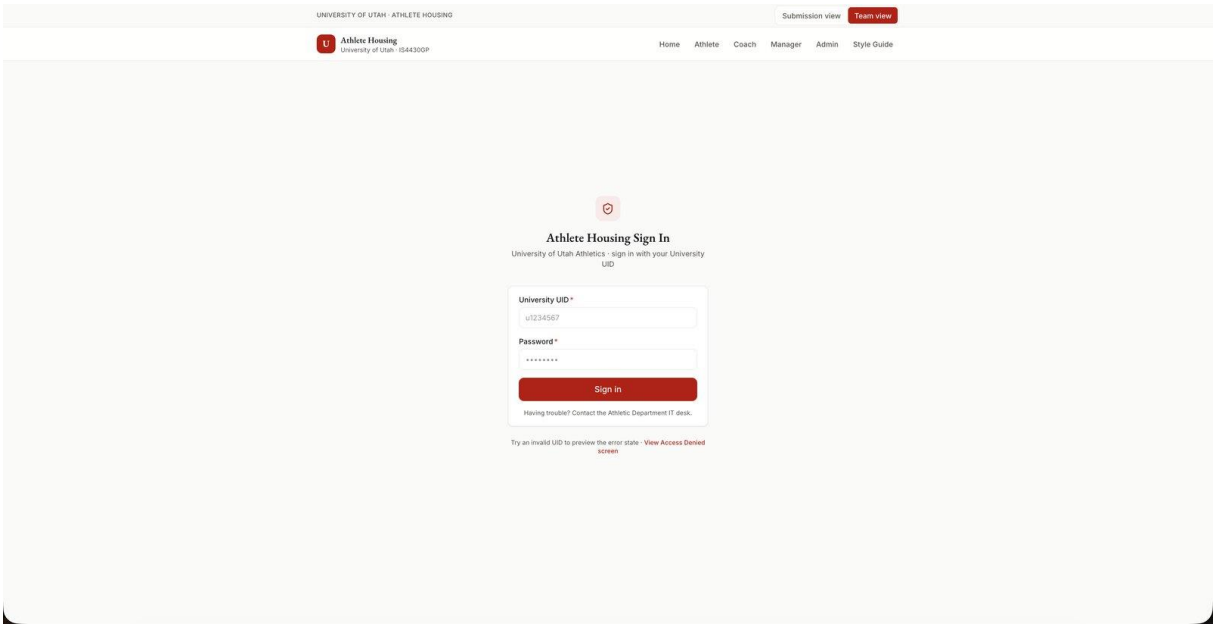
D-03	Housing Budget Management	Admin	REQ-BUDGET-003, REQ-SEC-005	Budget edit, old/new audit capture
D-04	Approved Website Management	Admin	REQ-SITE-006	Add/edit/disable/remove, active/inactive status
D-05	Listing Collection History	Admin	REQ-LISTING-007 §3.6	Per-source status, last run, failure log
D-06	Audit / System History	Admin	REQ-SEC-005 §3.5	Read-only event log

20 screens total. S-01 and S-02 are shared shells reused by every role.

Screen Captures from the Live Prototype

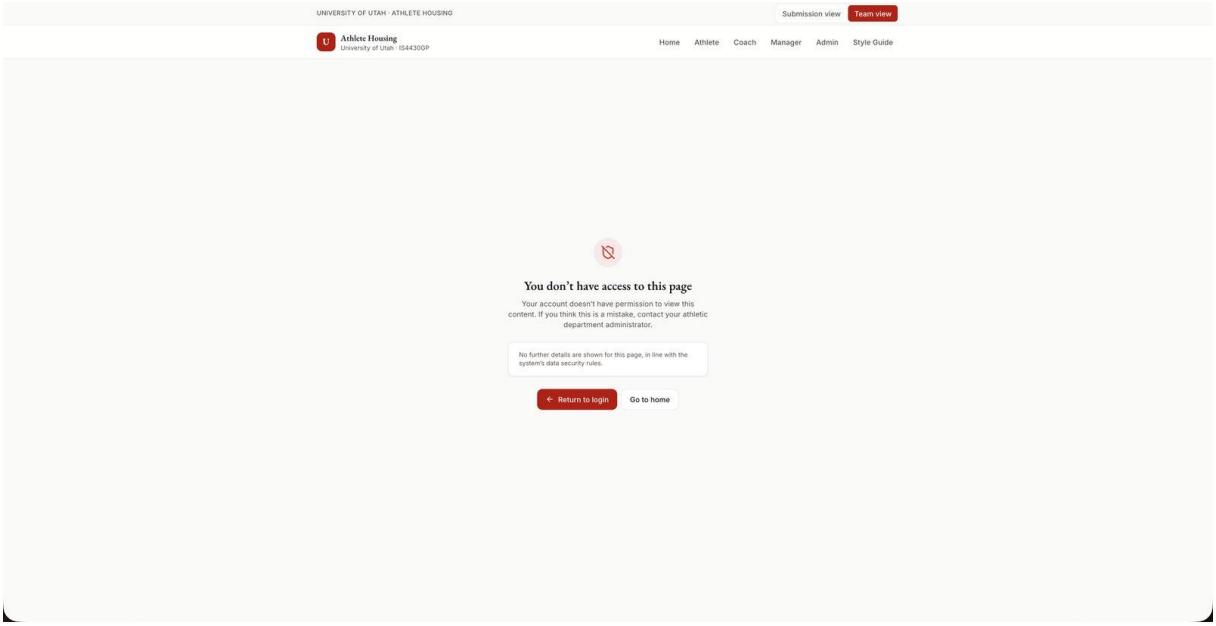
S-01 — Login (UID + Password)

Role: All roles | Traces to REQ-AUTH-001



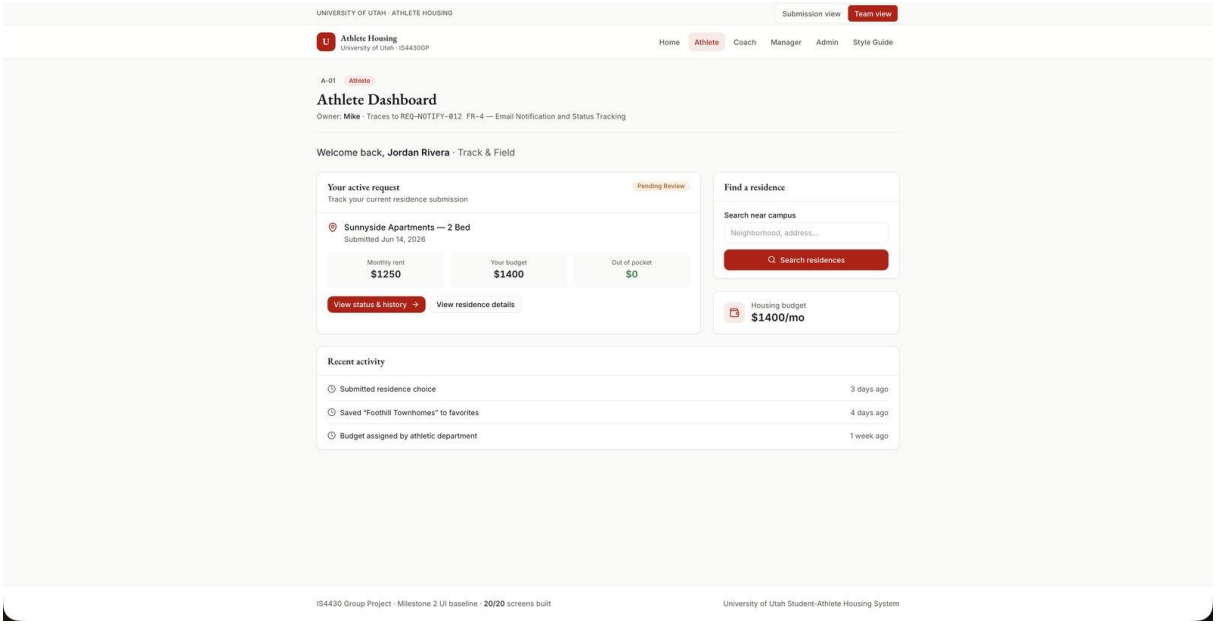
S-02 — Access Denied

Role: All roles | Traces to REQ-ROLE-004 §3.6, REQ-SEC-005



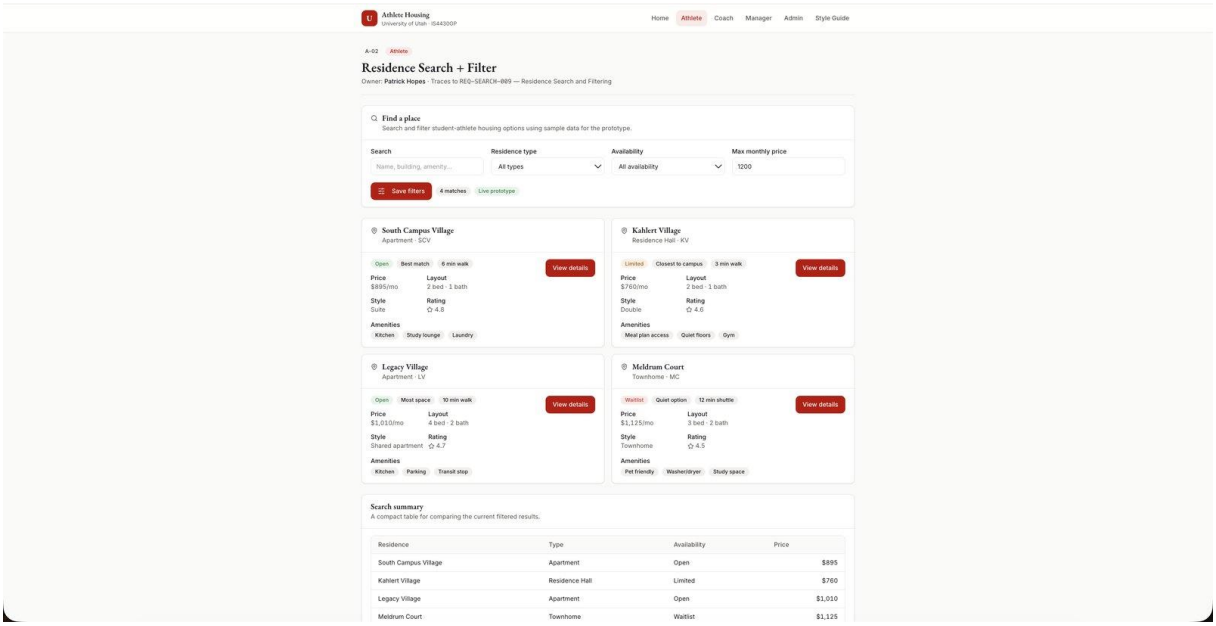
A-01 — Athlete Dashboard

Role: Athlete | Traces to REQ-NOTIFY-012 FR-4



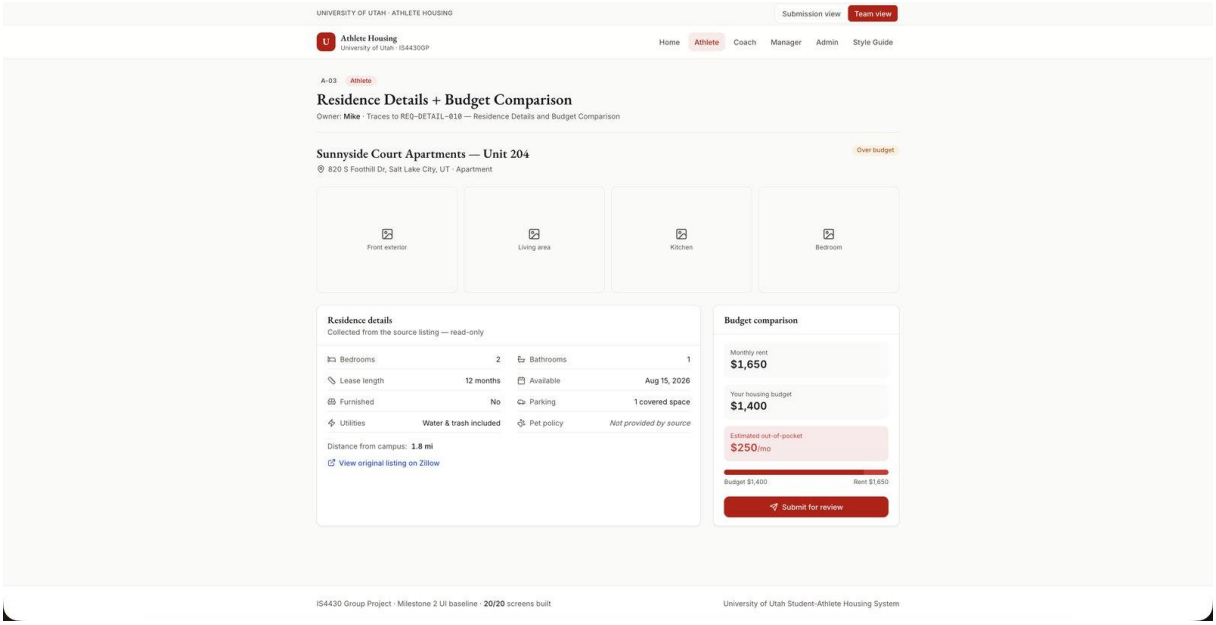
A-02 — Residence Search + Filter

Role: Athlete | Traces to REQ-SEARCH-009



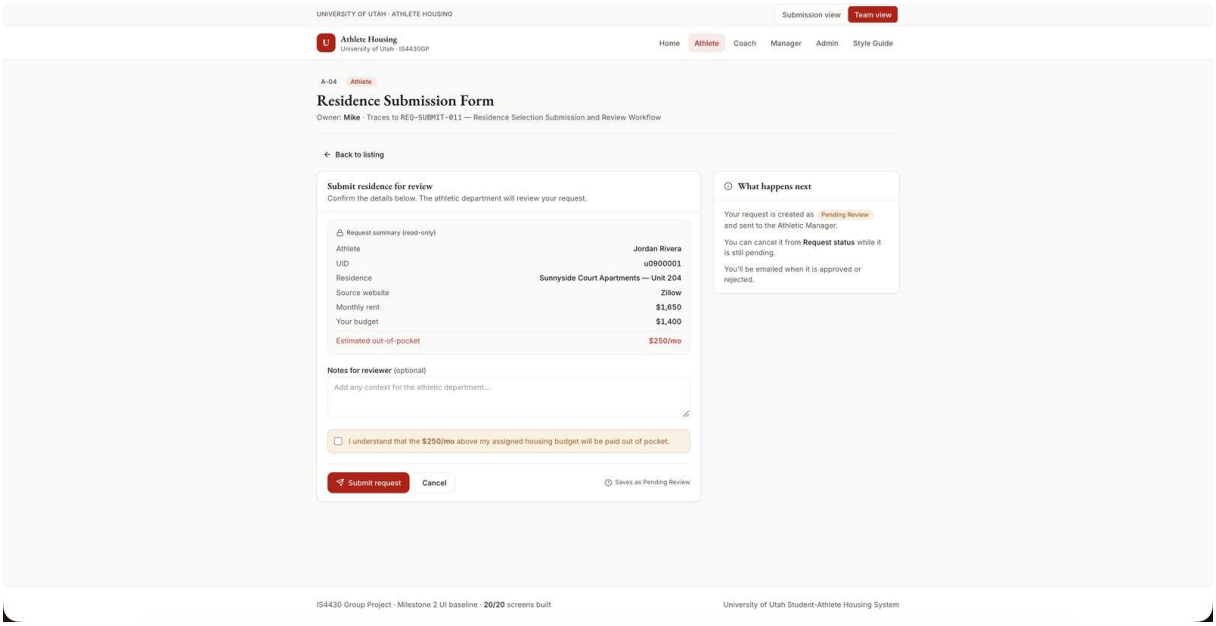
A-03 — Residence Details + Budget Comparison

Role: Athlete | Traces to REQ-DETAIL-010



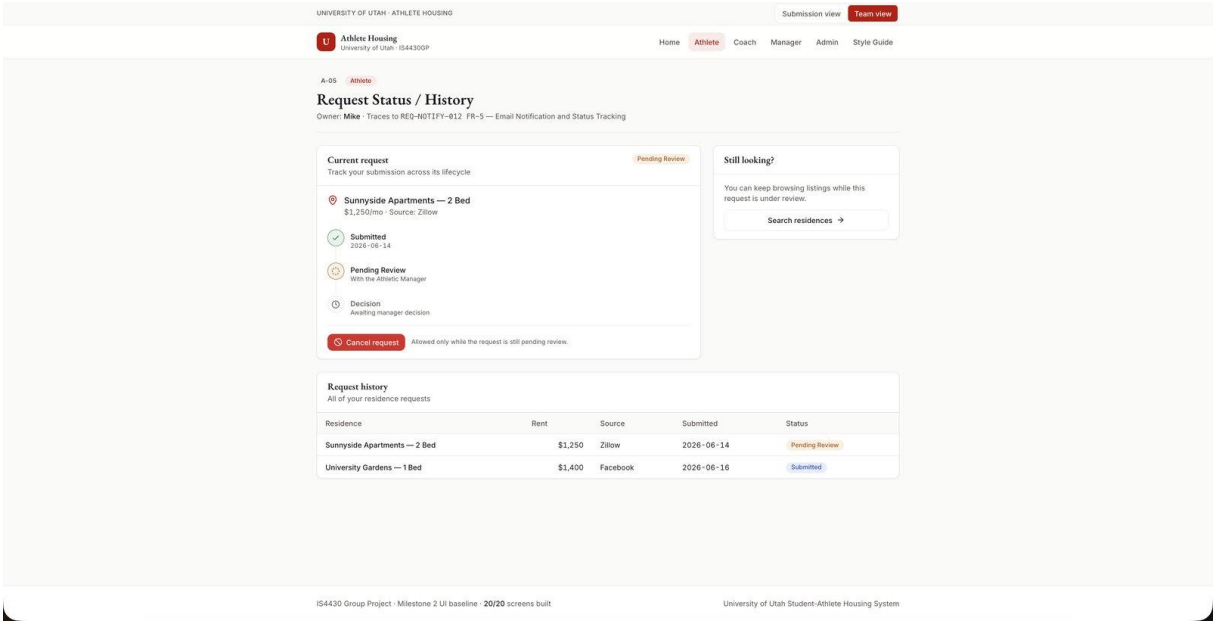
A-04 — Residence Submission Form

Role: Athlete | Traces to REQ-SUBMIT-011



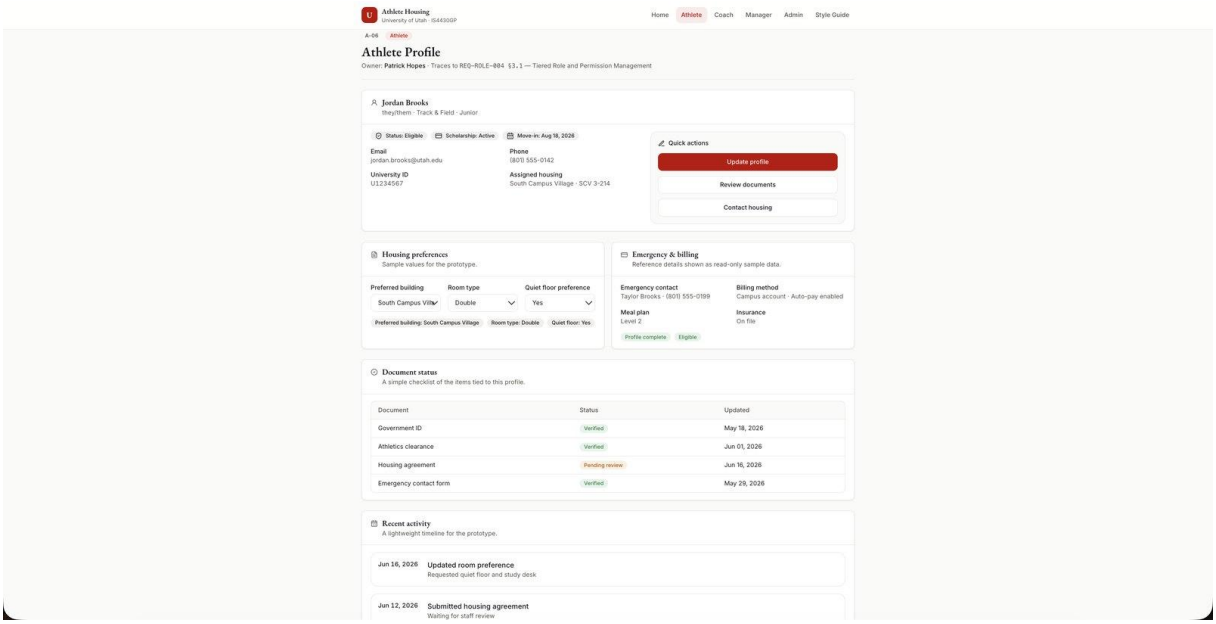
A-05 — Request Status / History

Role: Athlete | Traces to REQ-NOTIFY-012 FR-5



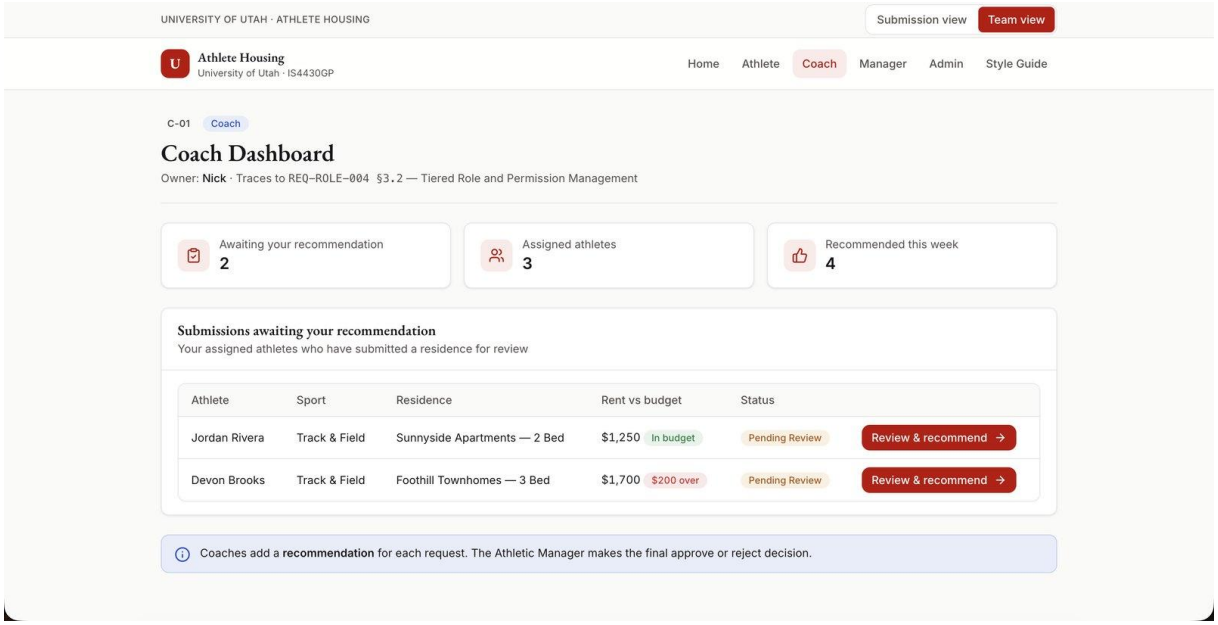
A-06 — Athlete Profile

Role: Athlete | Traces to REQ-ROLE-004 §3.1



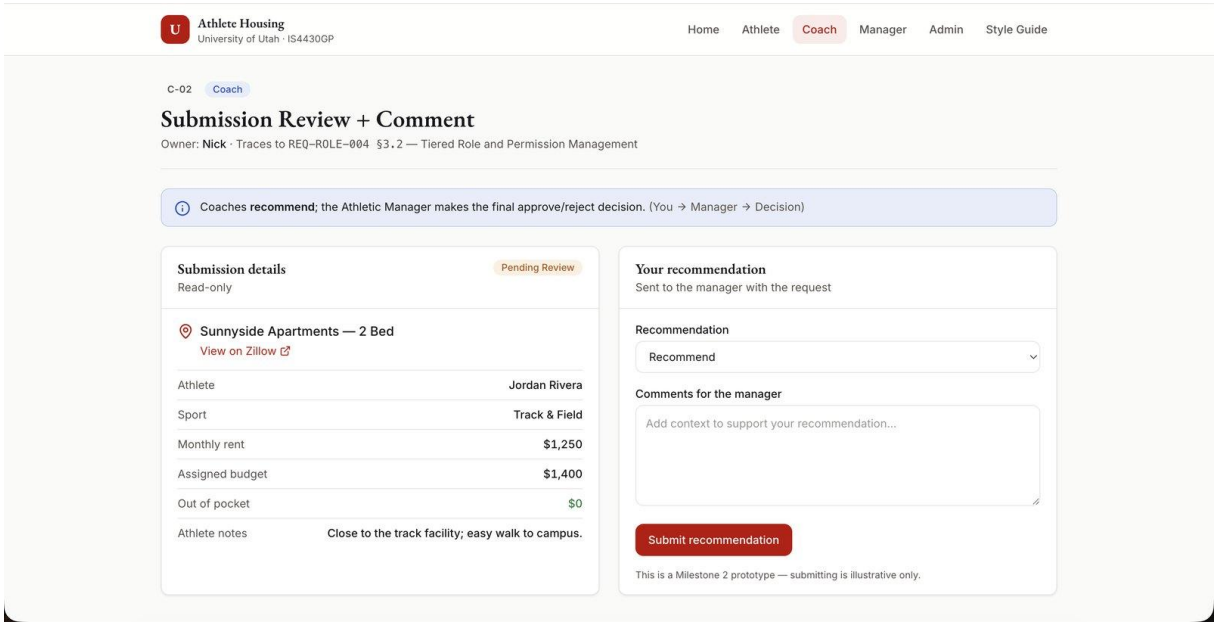
C-01 — Coach Dashboard

Role: Coach | Traces to REQ-ROLE-004 §3.2



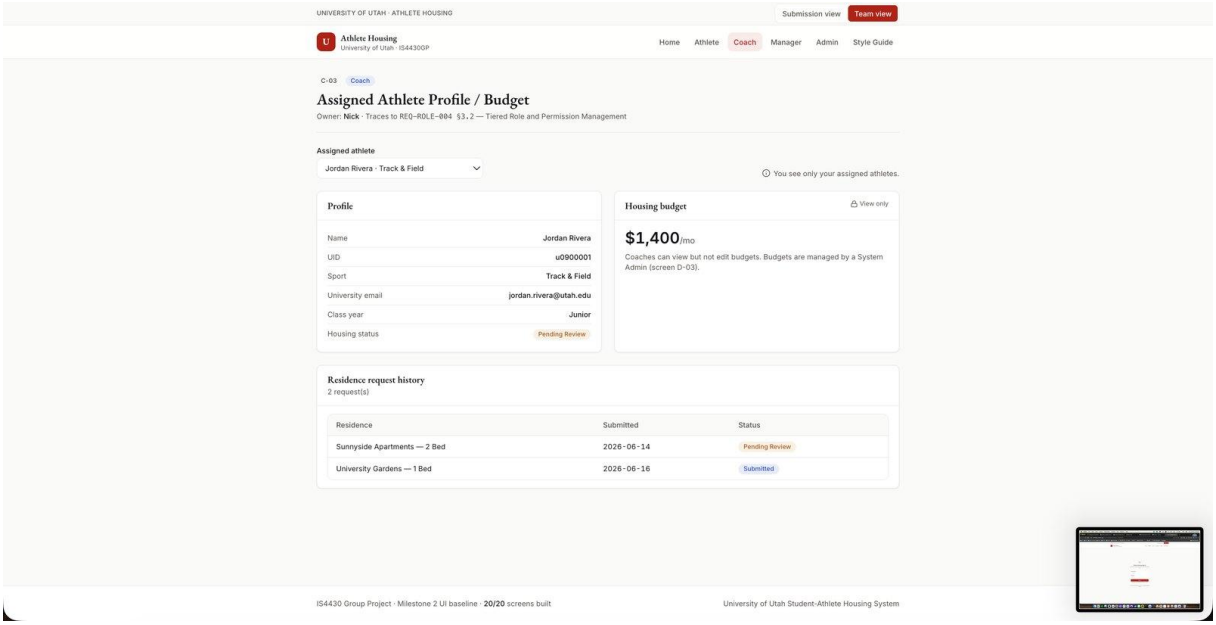
C-02 — Submission Review + Comment

Role: Coach | Traces to REQ-ROLE-004 §3.2



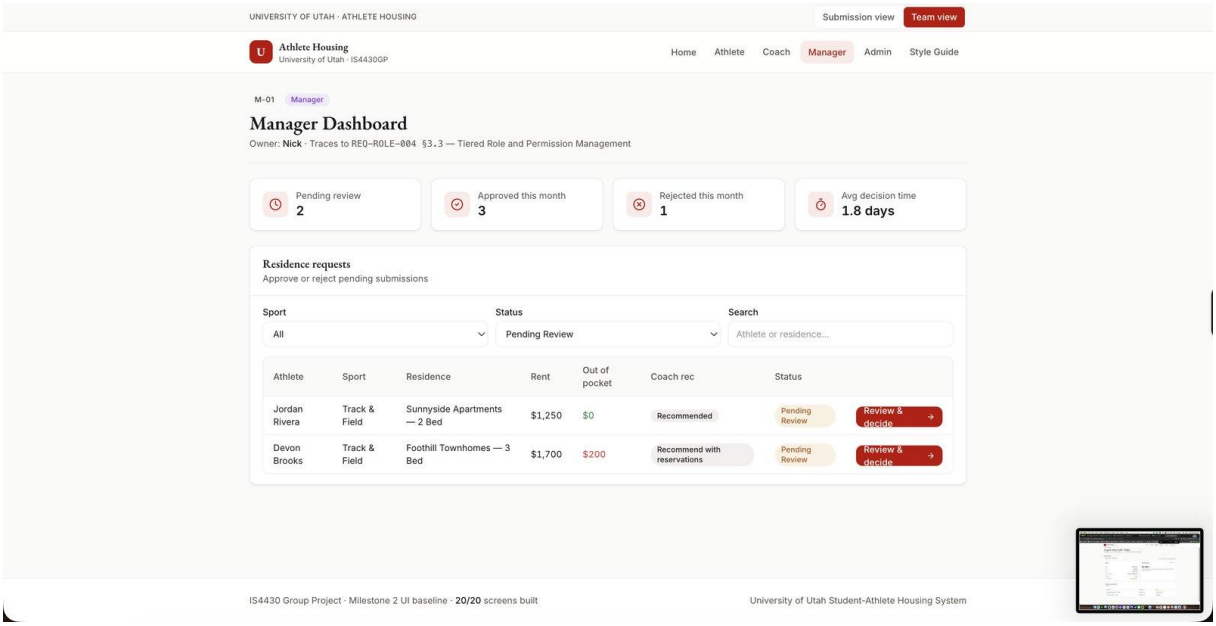
C-03 — Assigned Athlete Profile / Budget

Role: Coach | Traces to REQ-ROLE-004 §3.2



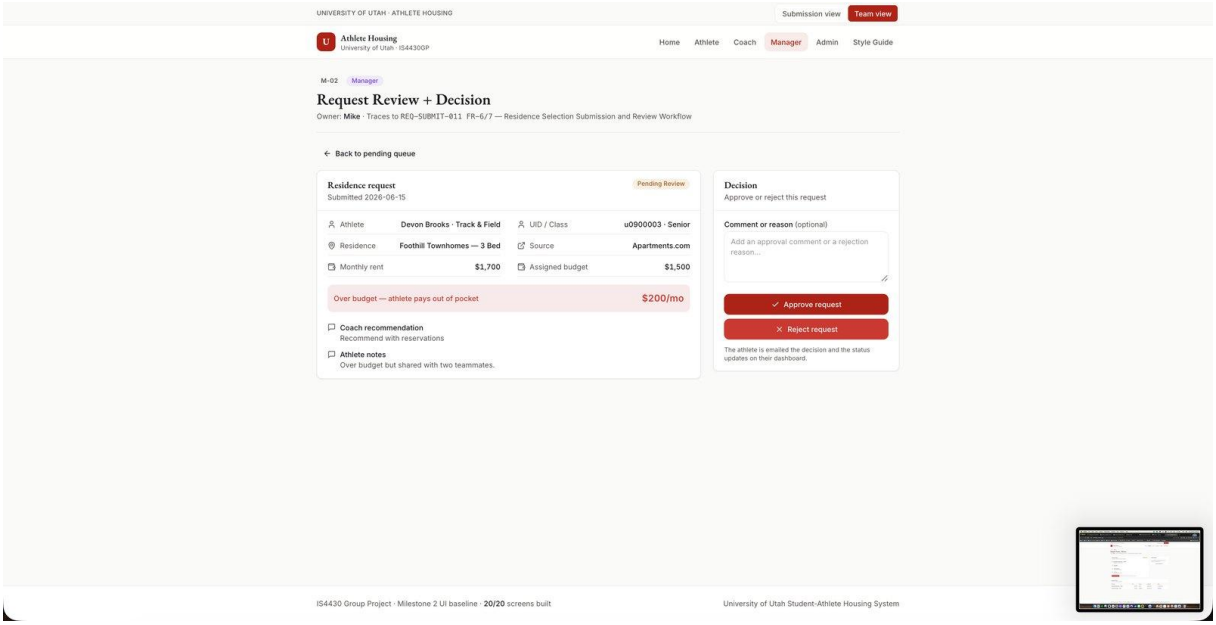
M-01 — Manager Dashboard

Role: Manager | Traces to REQ-ROLE-004 §3.3



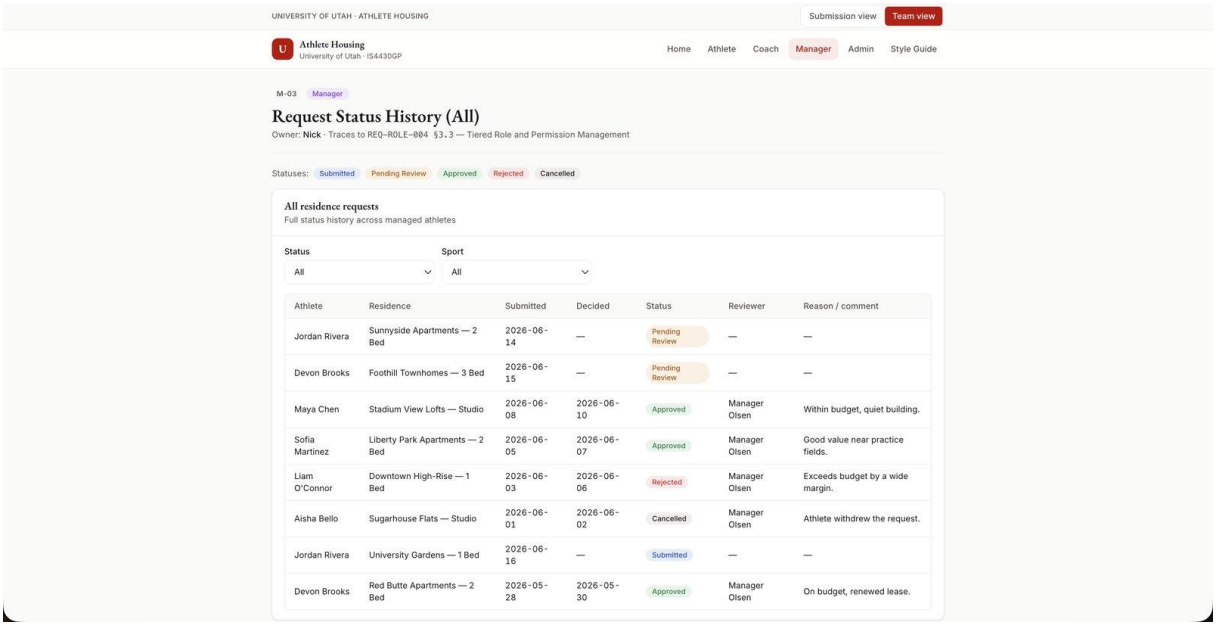
M-02 — Request Review + Decision

Role: Manager | Traces to REQ-SUBMIT-011 FR-6/7



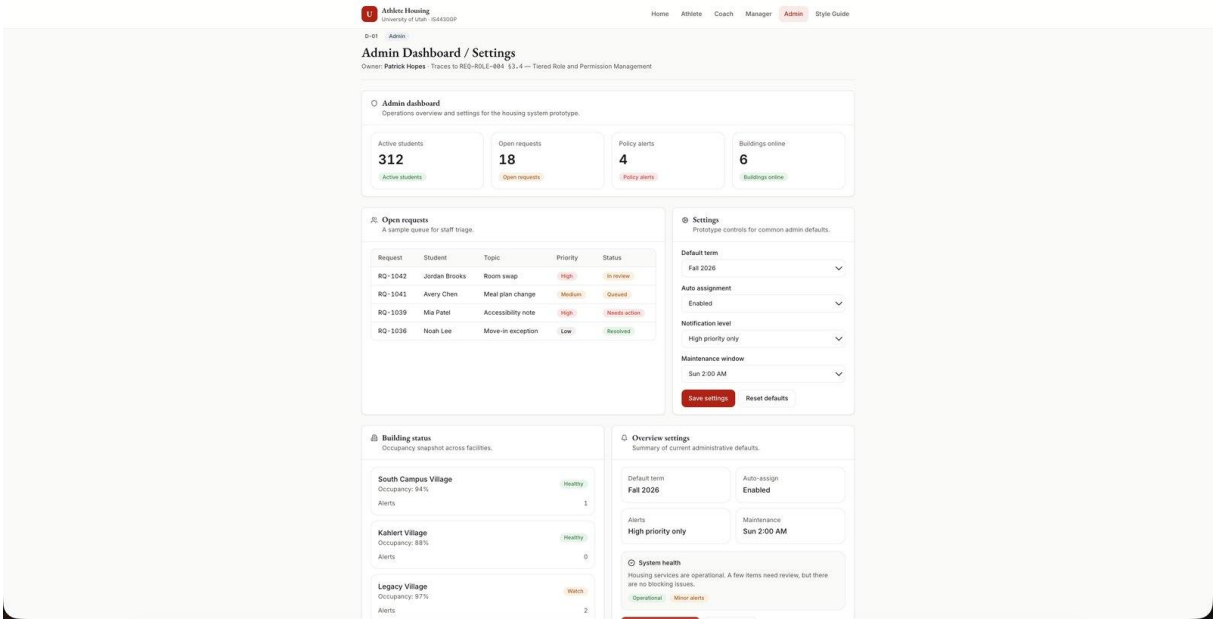
M-03 — Request Status History (All)

Role: Manager | Traces to REQ-ROLE-004 §3.3



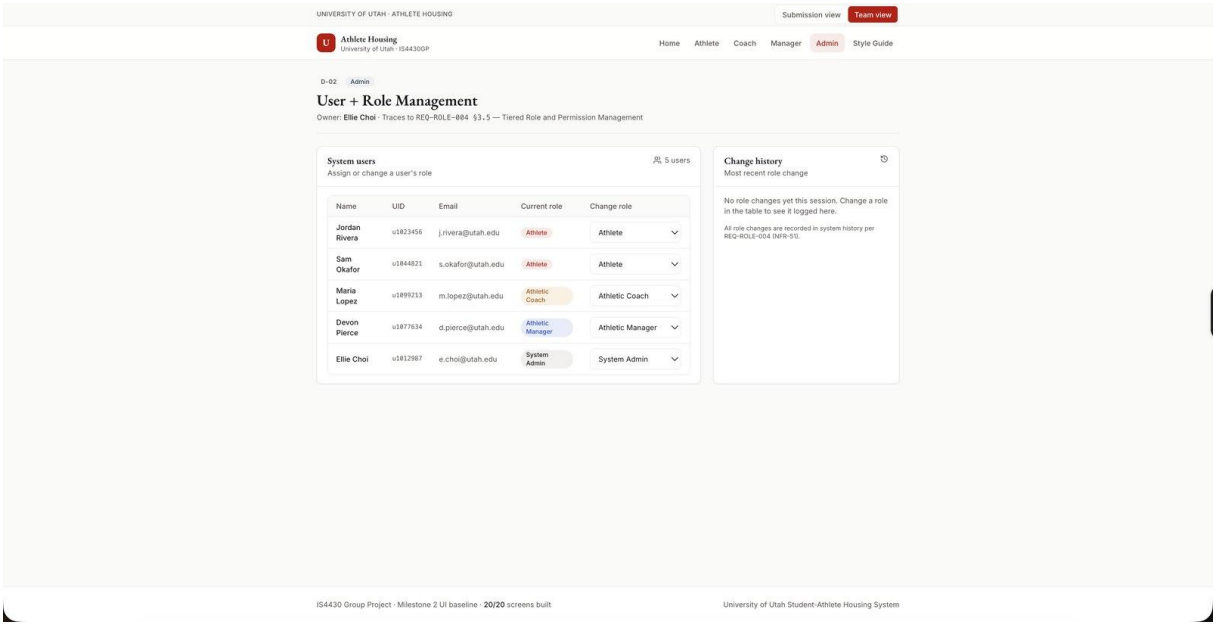
D-01 — Admin Dashboard / Settings

Role: Admin | Traces to REQ-ROLE-004 §3.4



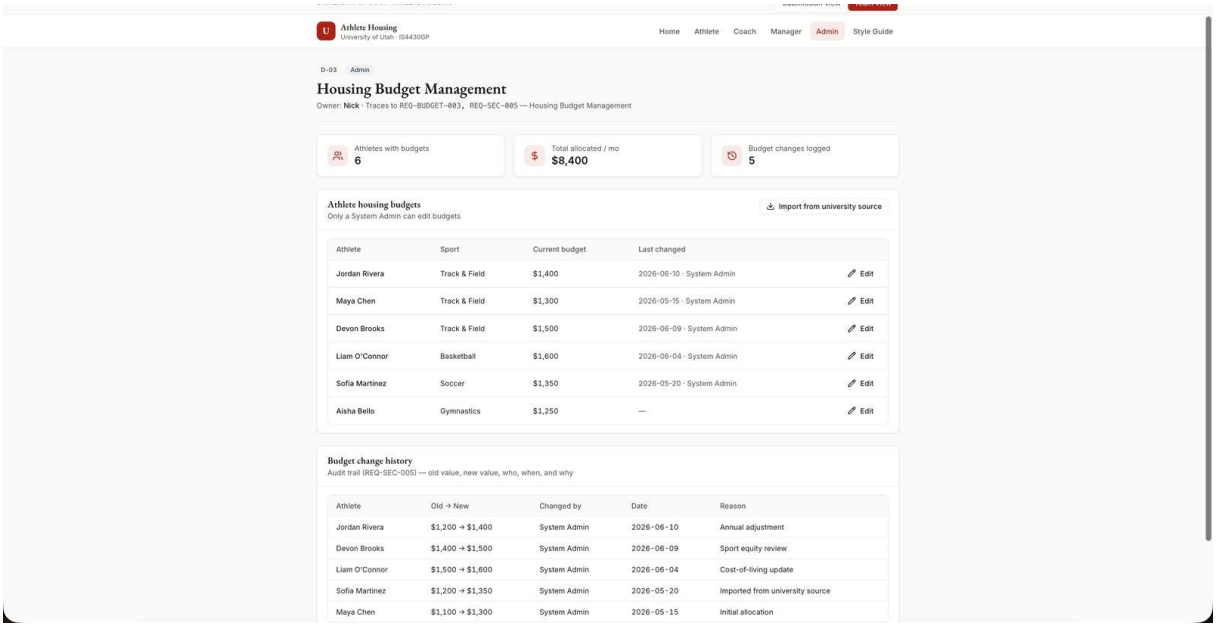
D-02 — User + Role Management

Role: Admin | Traces to REQ-ROLE-004 §3.5



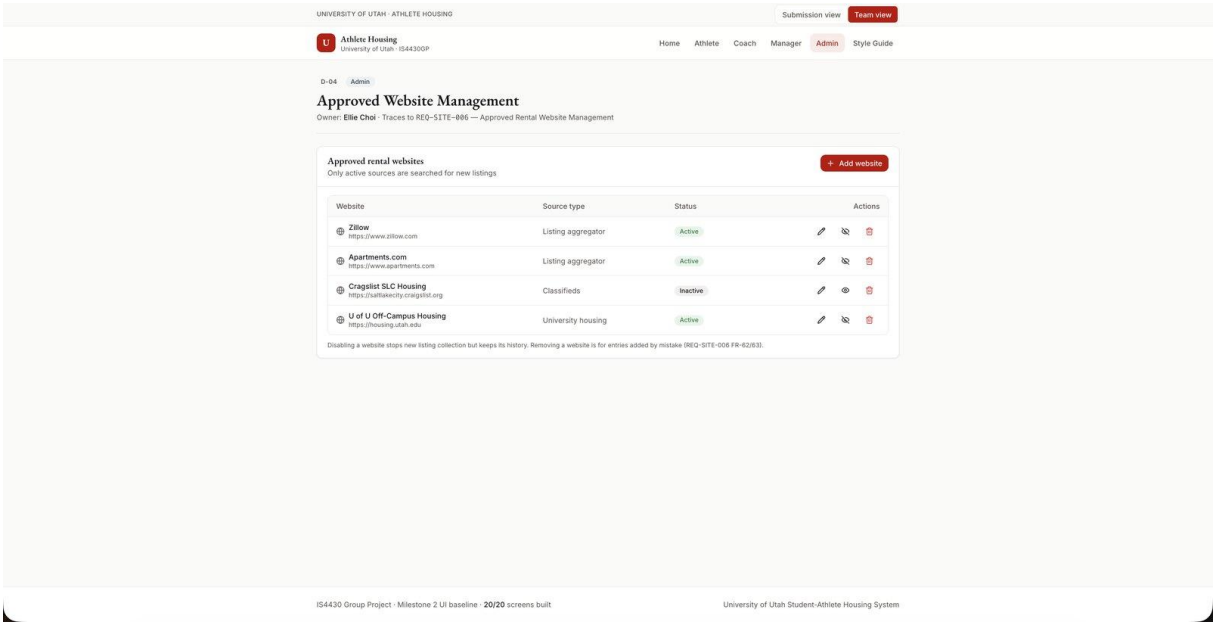
D-03 — Housing Budget Management

Role: Admin | Traces to REQ-BUDGET-003, REQ-SEC-005



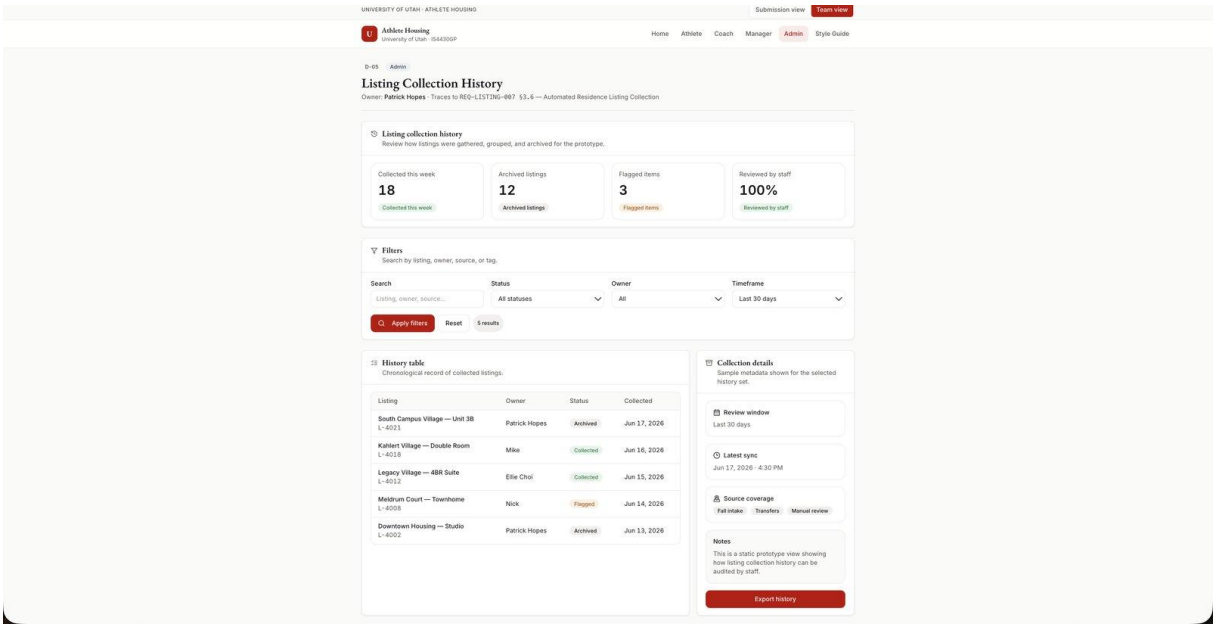
D-04 — Approved Website Management

Role: Admin | Traces to REQ-SITE-006



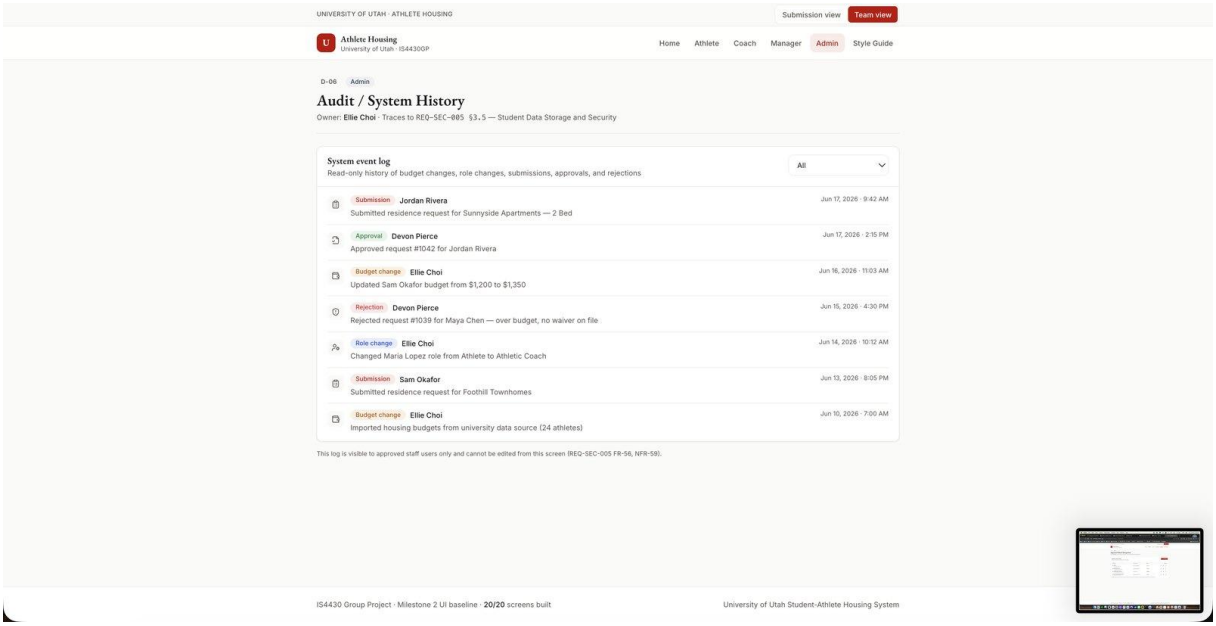
D-05 — Listing Collection History

Role: Admin | Traces to REQ-LISTING-007 §3.6



D-06 — Audit / System History

Role: Admin | Traces to REQ-SEC-005 §3.5



3

Milestone 3

Data Flow Diagrams and Process Descriptions

Data Flow Diagram Scope

This section presents the system boundary, the Level 0 internal model, and one Level 1 decomposition for every Level 0 process. The final model uses ten Level 0 processes, which preserves the one-to-one link to the ten activity diagrams in Milestone 5.

The diagrams use the data flow diagram symbols demonstrated in class. External entities use rectangles. Processes use the rounded Gane-Sarson process symbol with a numbered header. Data stores use the open-ended Gane-Sarson symbol. Directional arrows carry noun-phrase data labels.

Every Context boundary flow repeats on the Level 0 diagram. Every Level 1 boundary flow repeats the exact label and direction of its parent Level 0 process. Level 1 diagrams show only the decomposed subprocesses and boundary arrows, so external entities and data-store boxes are not repeated.

The Athletic Department Reviewer represents the assignment's Athletic Coach role and other authorized athletic staff. The System Admin represents the assignment's Athletic Administrator role.

Data-Store Alignment

The seven logical DFD data stores map to the updated logical design and the final relational tables as follows.

ID	Logical data store	Current relational implementation
D1	User and Role Records	USER, ATHLETE, STAFF, ATHLETIC_DEPARTMENT_REVIEWER, SYSTEM_ADMIN, ROLE, PERMISSION, ROLE_PERMISSION, USER_ROLE, and USER_SESSION
D2	Athlete Profiles	Logical profile view implemented through ATHLETE and its related USER record
D3	Budgets and Budget Change History	HOUSING_BUDGET and BUDGET_CHANGE
D4	Approved Website Sources	APPROVED_WEBSITE_SOURCE
D5	Residence Listings and Source Data	RESIDENCE_LISTING and COLLECTION_RUN
D6	Residence Requests	RESIDENCE_REQUEST and REQUEST_REVIEW
D7	Status and Audit History	REQUEST_STATUS_HISTORY, EMAIL_NOTIFICATION, and AUDIT_EVENT

Context Data Flow Diagram

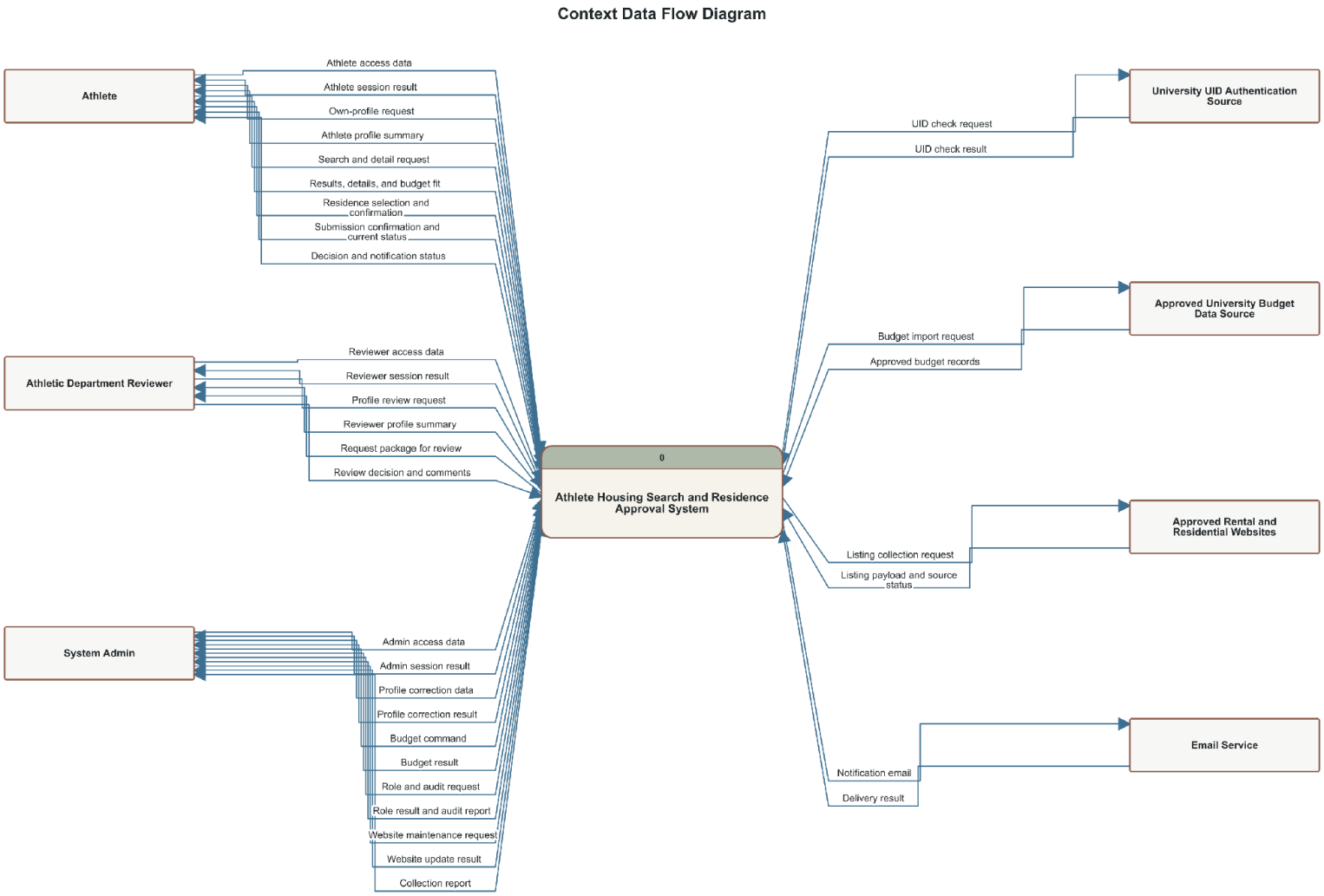


Figure 3.1. Context data flow diagram.

Level 0 Data Flow Diagram

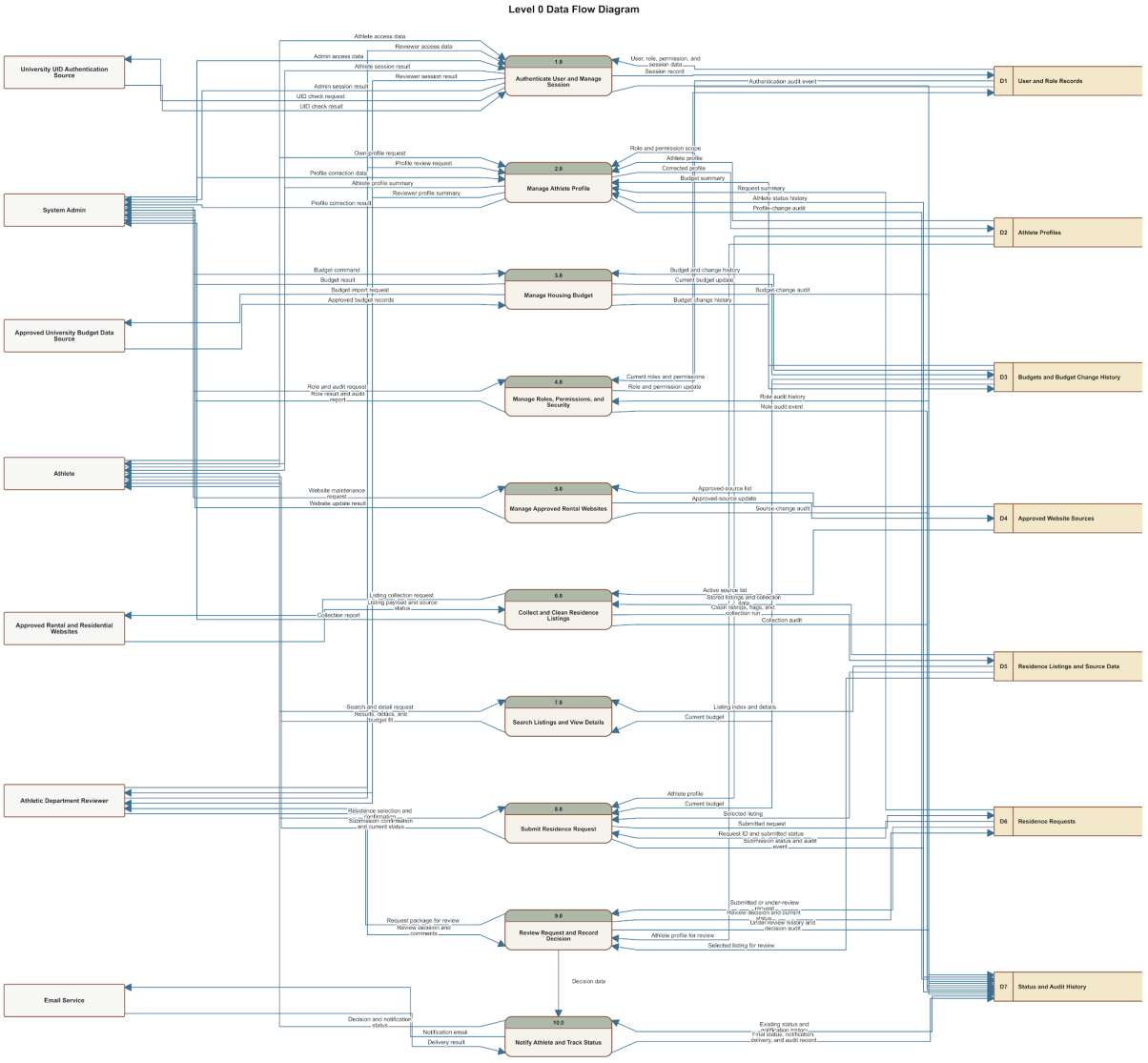


Figure 3.2. Level 0 data flow diagram.

Level 1 Data Flow Diagram for Process 1.0

Boundary labels and directions repeat the parent Level 0 process exactly.

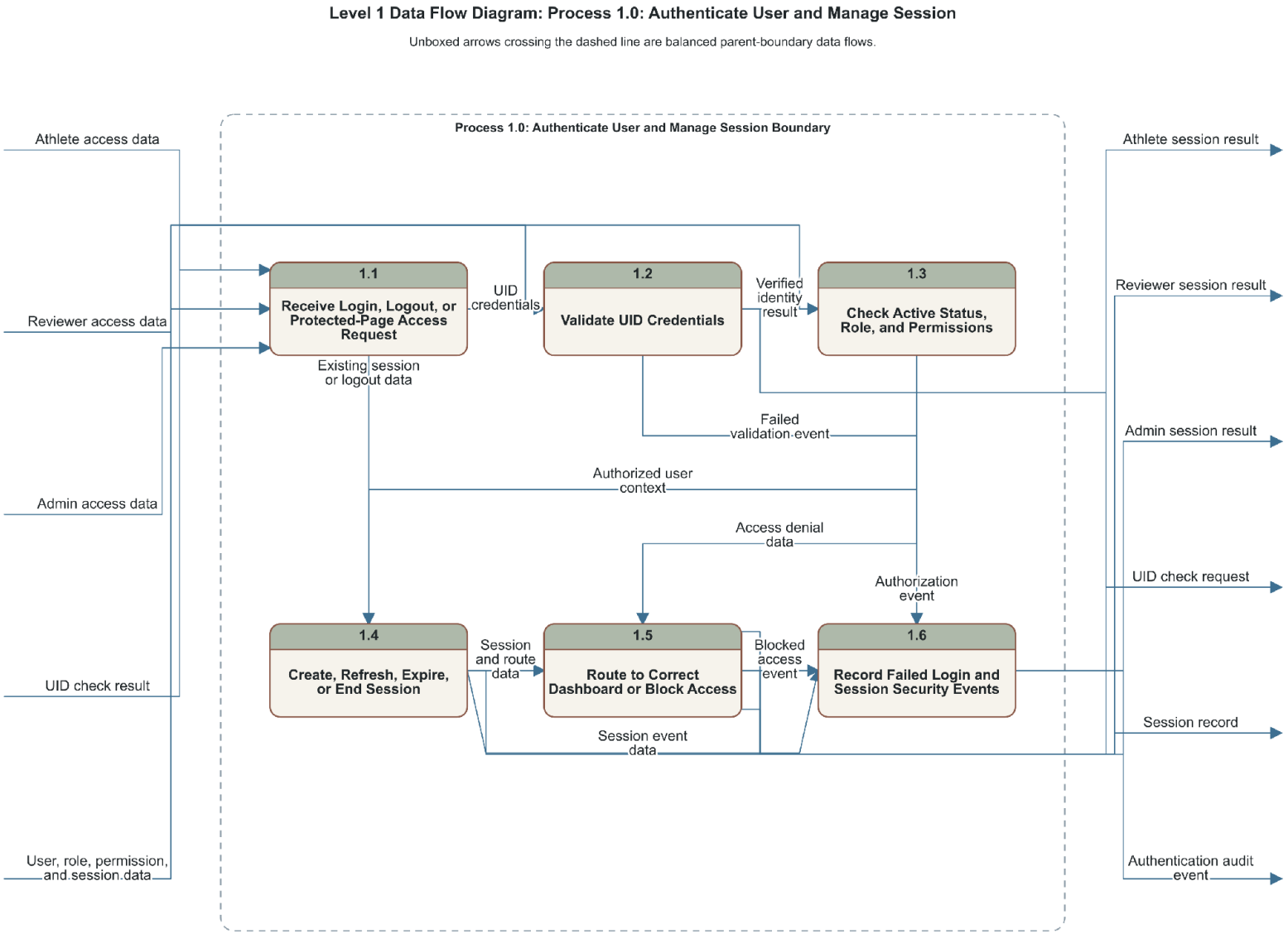


Figure 3.3. Level 1 decomposition of Process 1.0, Authenticate User and Manage Session.

Level 1 Data Flow Diagram for Process 2.0

Boundary labels and directions repeat the parent Level 0 process exactly.

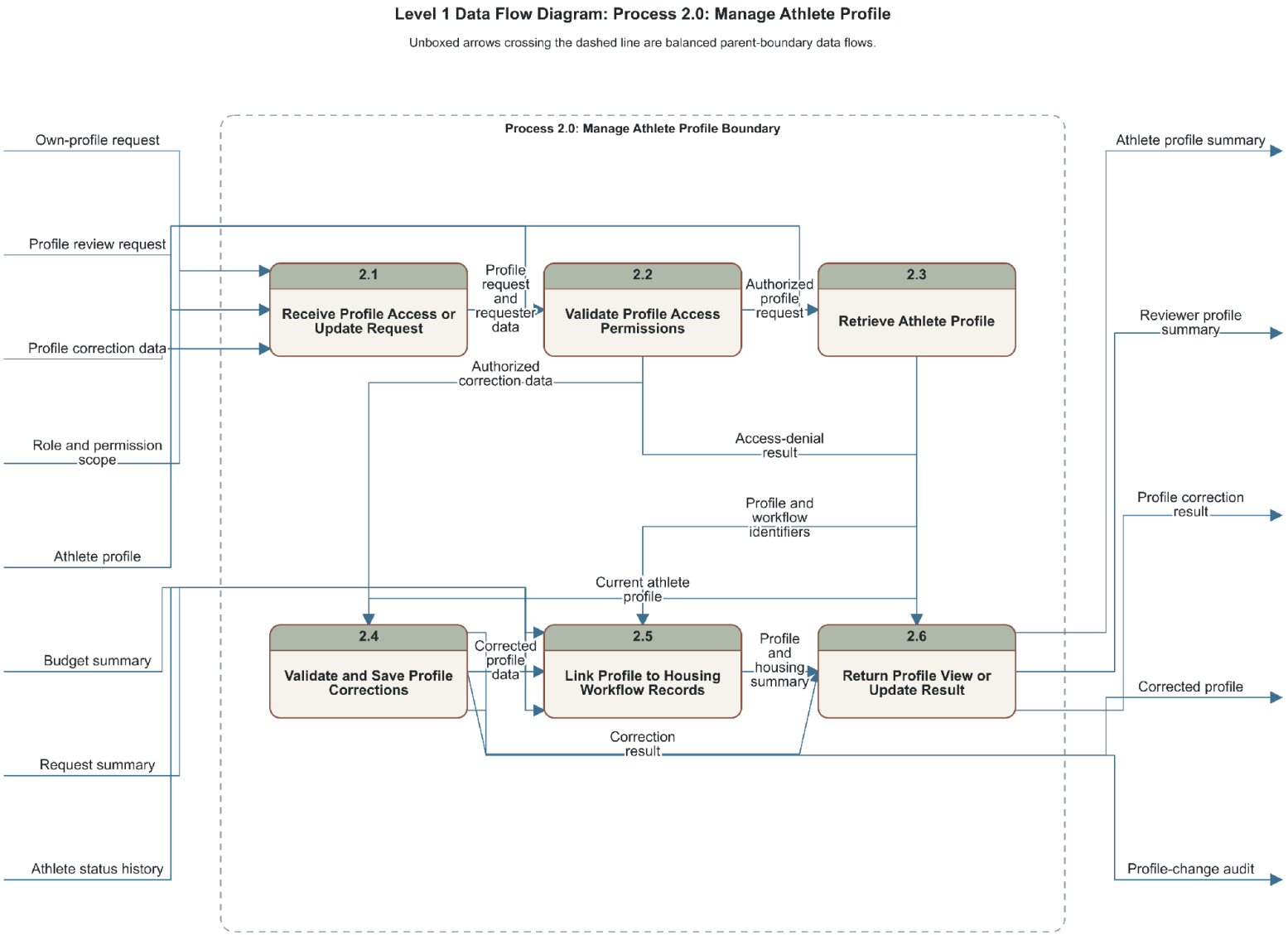


Figure 3.4. Level 1 decomposition of Process 2.0, Manage Athlete Profile.

Level 1 Data Flow Diagram for Process 3.0

Boundary labels and directions repeat the parent Level 0 process exactly.

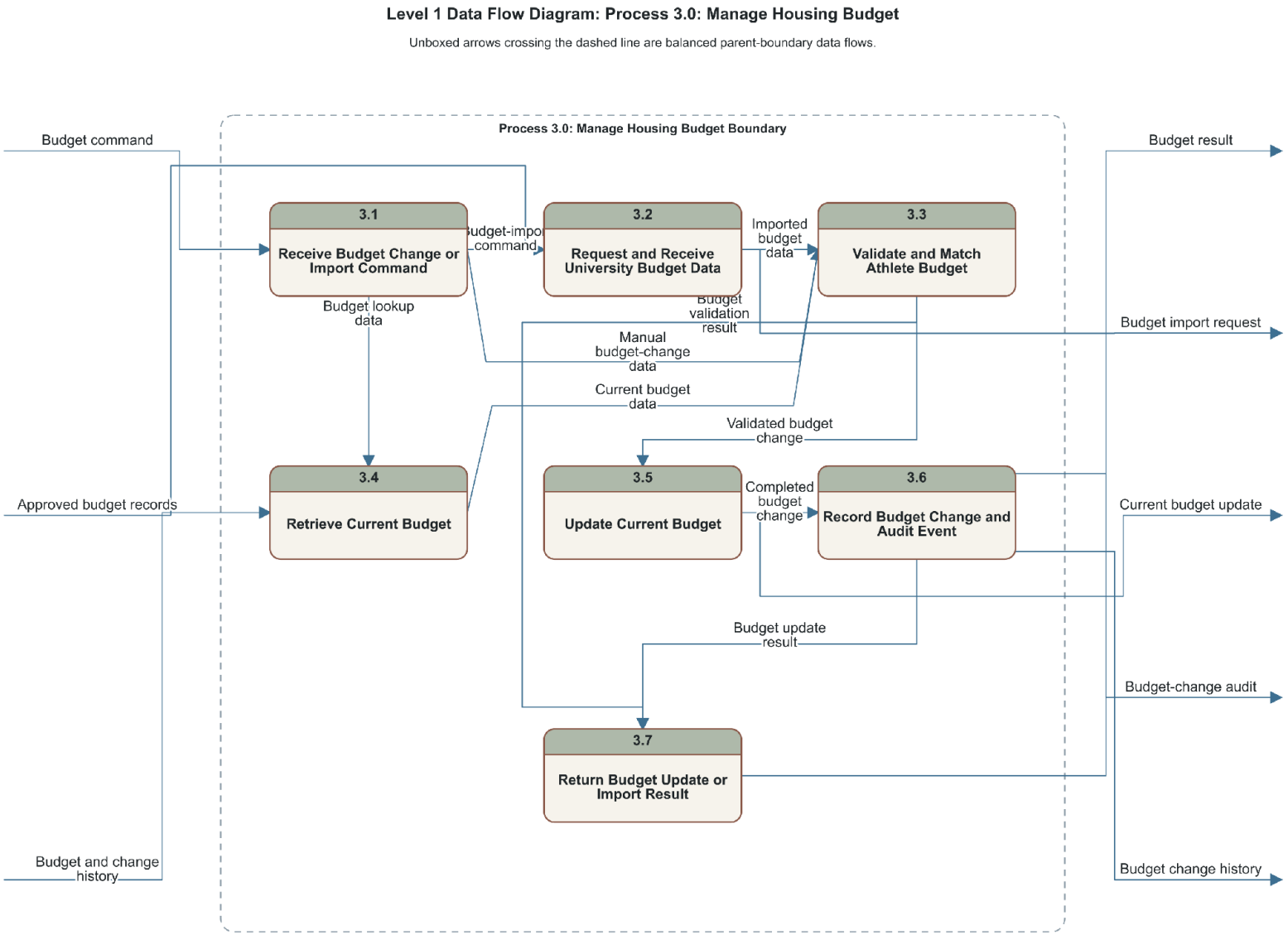


Figure 3.5. Level 1 decomposition of Process 3.0, Manage Housing Budget.

Level 1 Data Flow Diagram for Process 4.0

Boundary labels and directions repeat the parent Level 0 process exactly.

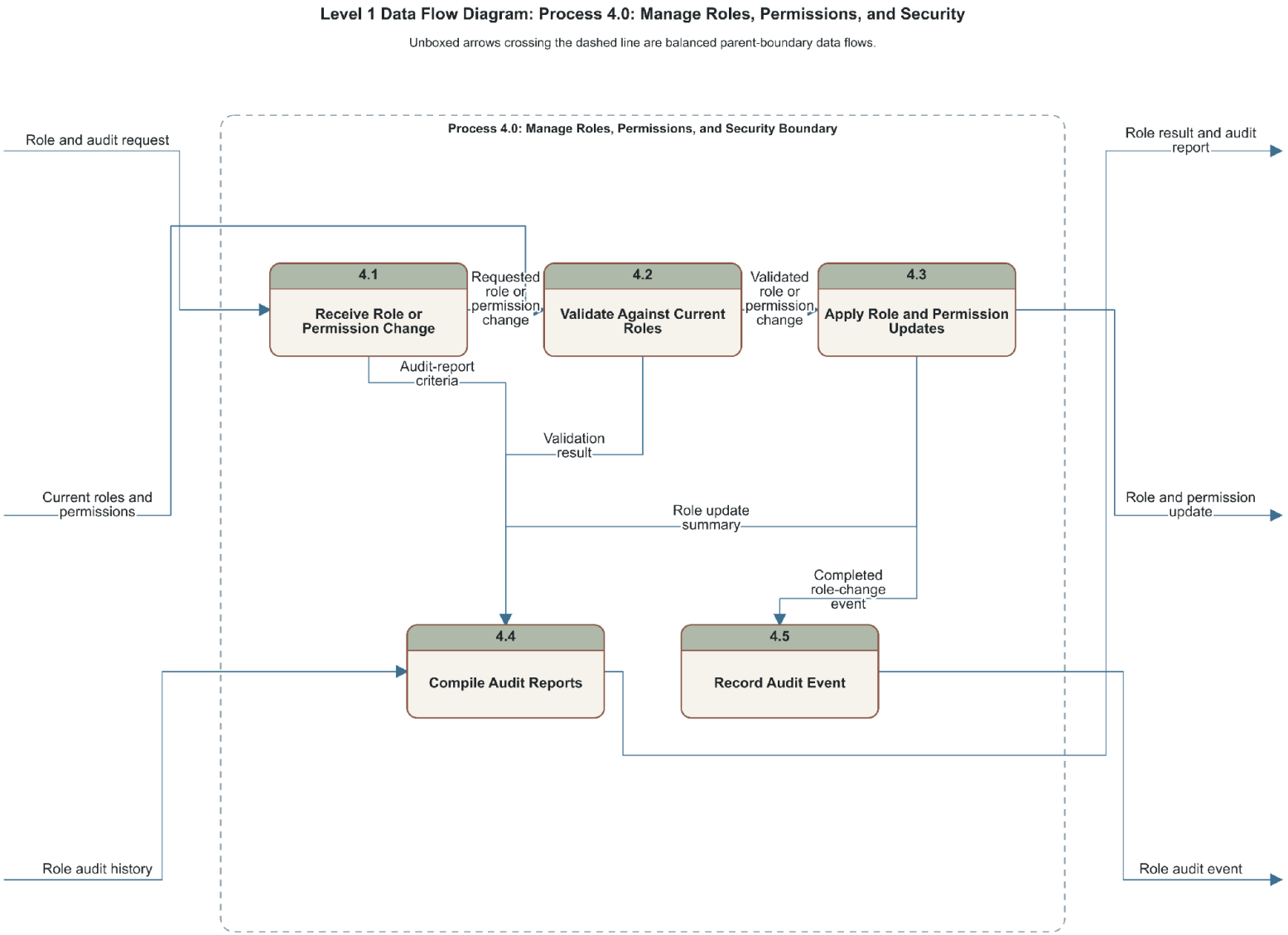


Figure 3.6. Level 1 decomposition of Process 4.0, Manage Roles, Permissions, and Security.

Level 1 Data Flow Diagram for Process 5.0

Boundary labels and directions repeat the parent Level 0 process exactly.

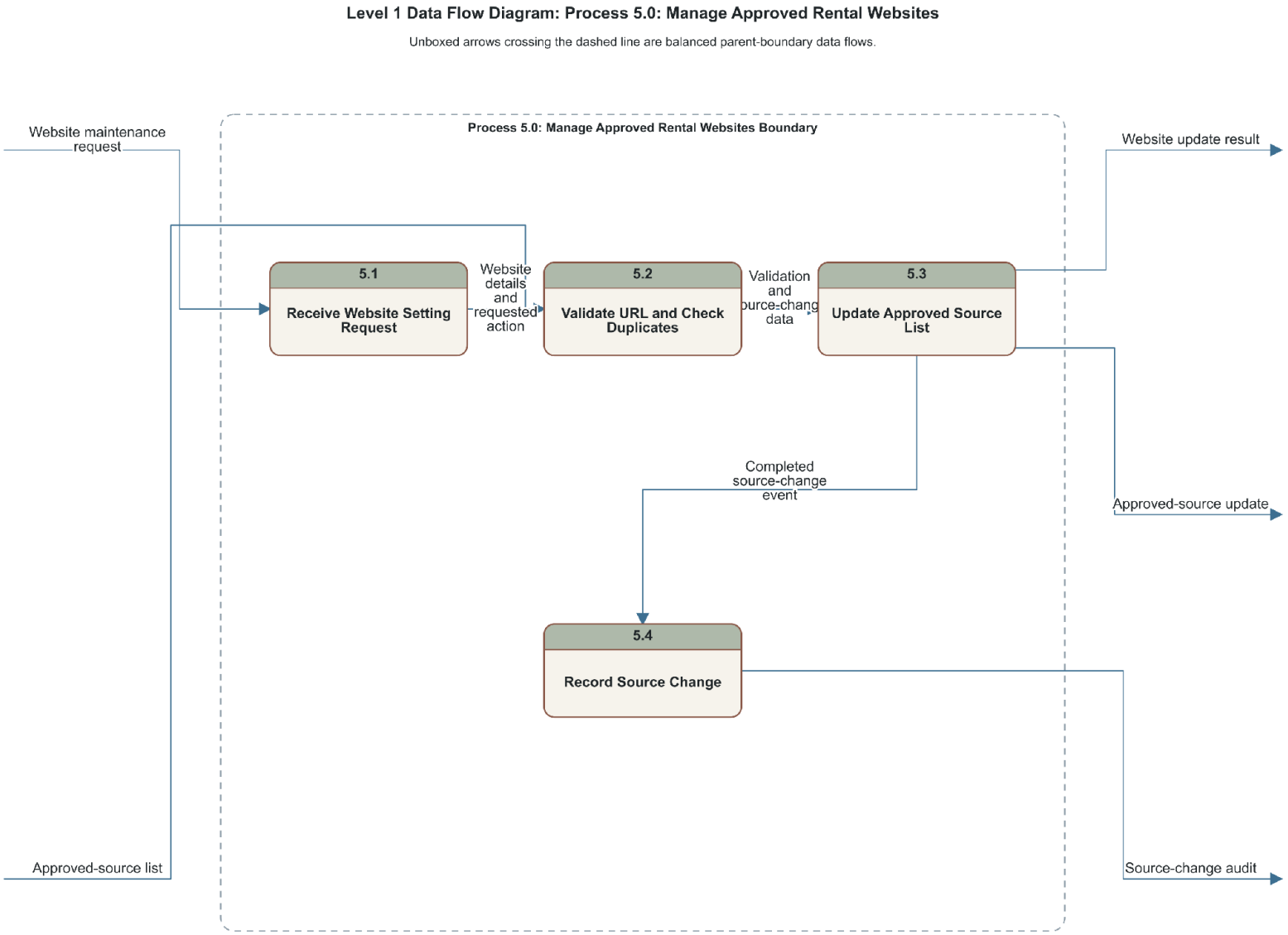


Figure 3.7. Level 1 decomposition of Process 5.0, Manage Approved Rental Websites.

Level 1 Data Flow Diagram for Process 6.0

Boundary labels and directions repeat the parent Level 0 process exactly.

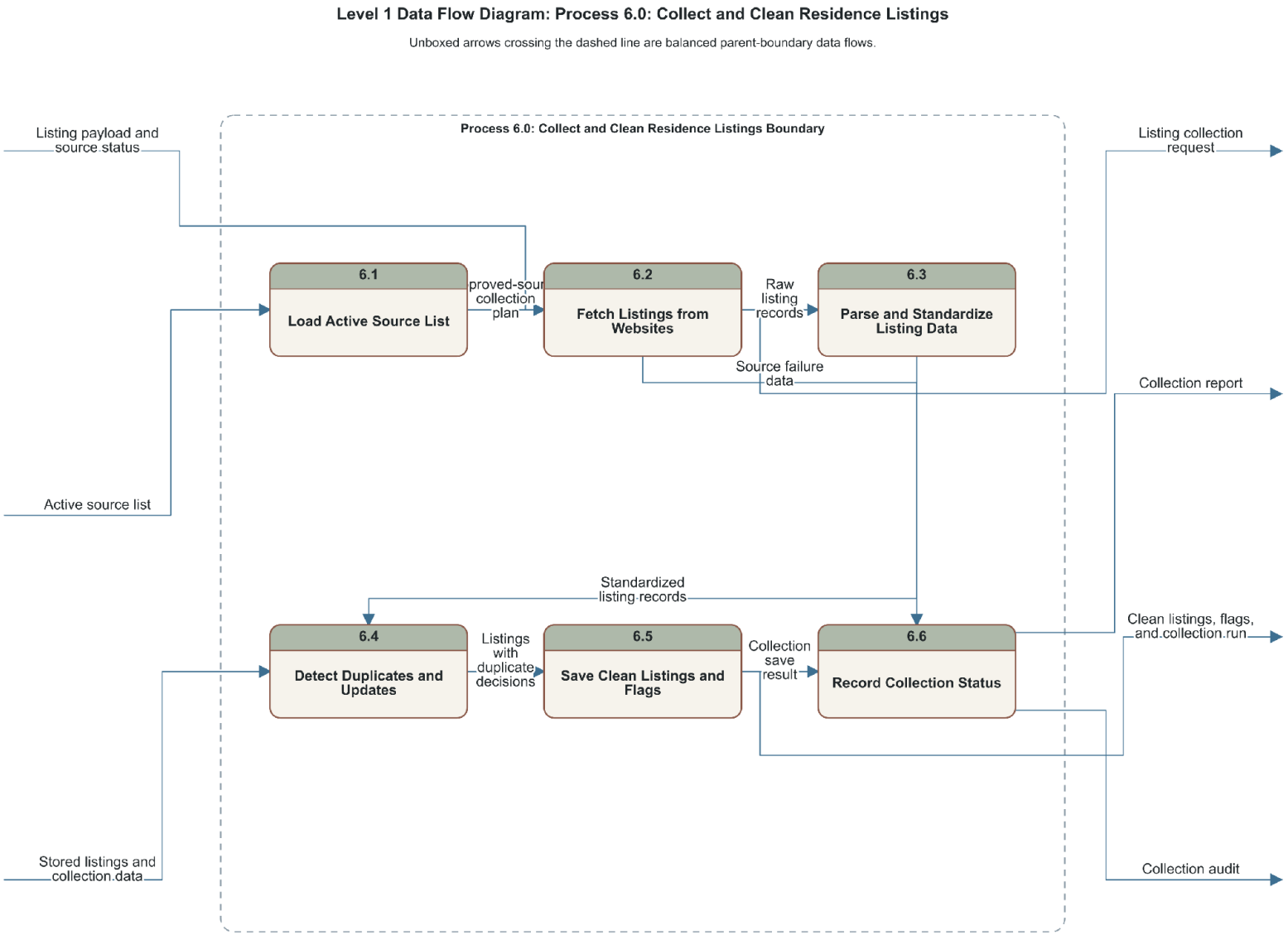


Figure 3.8. Level 1 decomposition of Process 6.0, Collect and Clean Residence Listings.

Level 1 Data Flow Diagram for Process 7.0

Boundary labels and directions repeat the parent Level 0 process exactly.

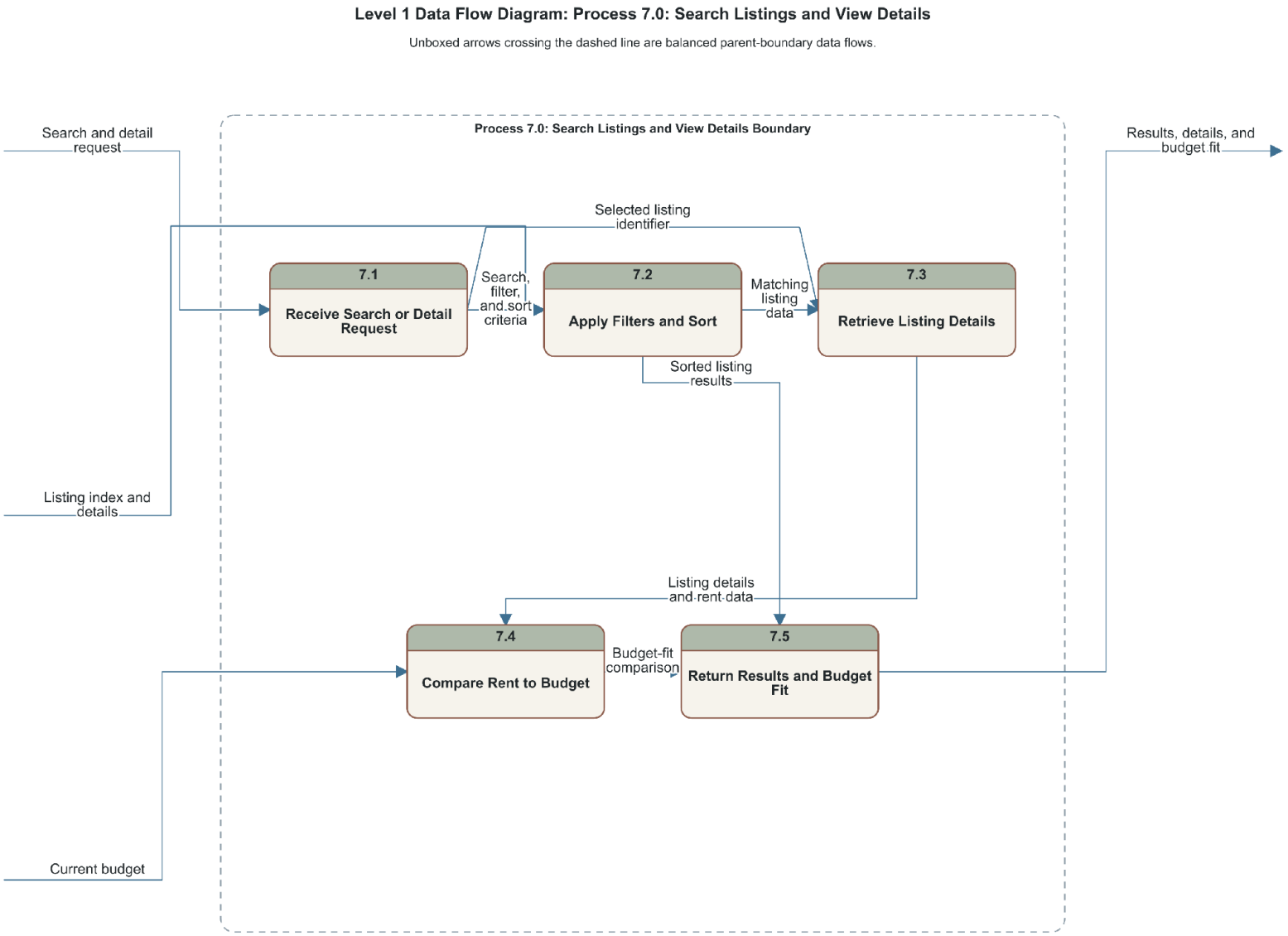


Figure 3.9. Level 1 decomposition of Process 7.0, Search Listings and View Details.

Level 1 Data Flow Diagram for Process 8.0

Boundary labels and directions repeat the parent Level 0 process exactly.

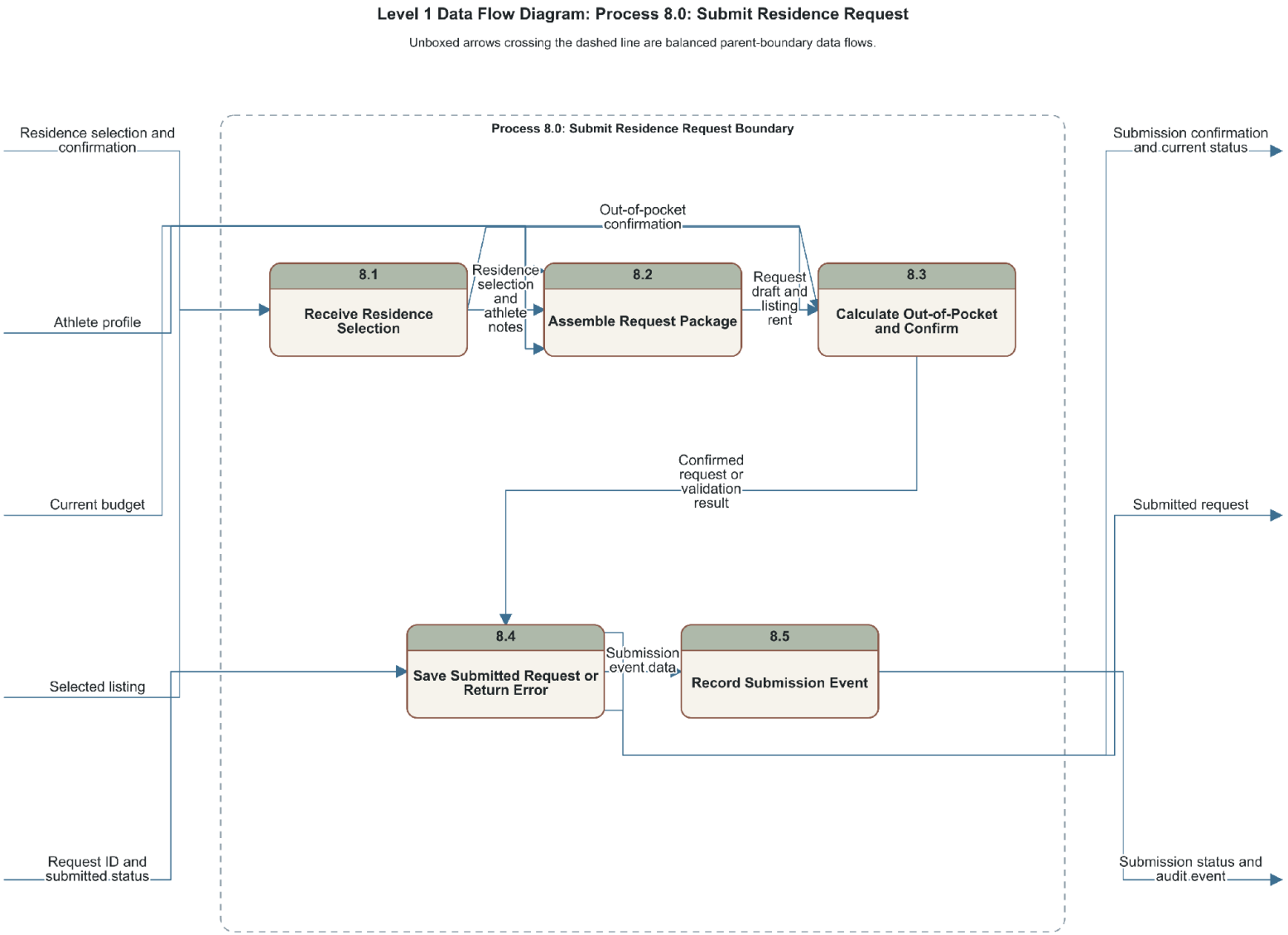


Figure 3.10. Level 1 decomposition of Process 8.0, Submit Residence Request.

Level 1 Data Flow Diagram for Process 9.0

Boundary labels and directions repeat the parent Level 0 process exactly.

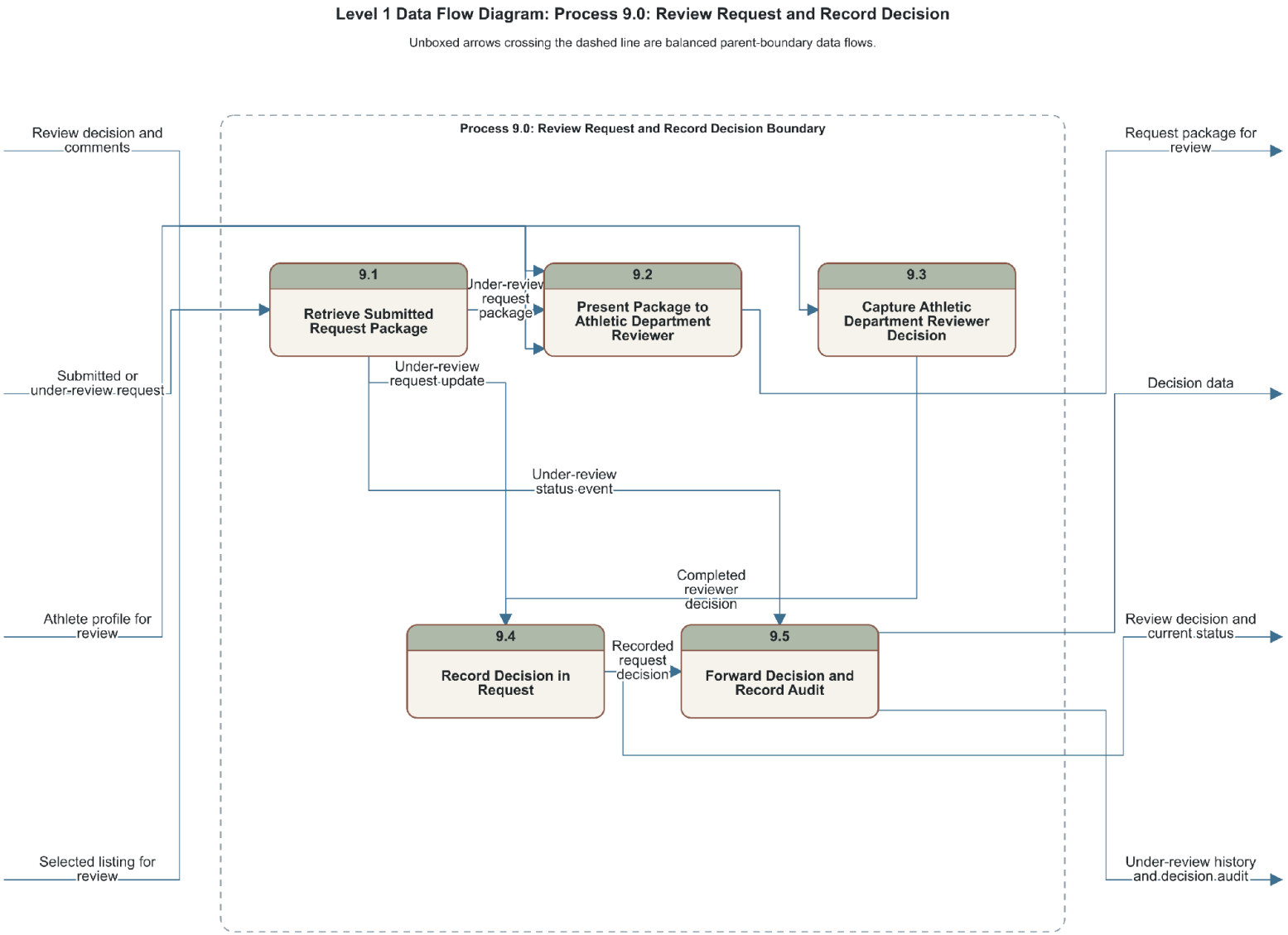


Figure 3.11. Level 1 decomposition of Process 9.0, Review Request and Record Decision.

Level 1 Data Flow Diagram for Process 10.0

Boundary labels and directions repeat the parent Level 0 process exactly.

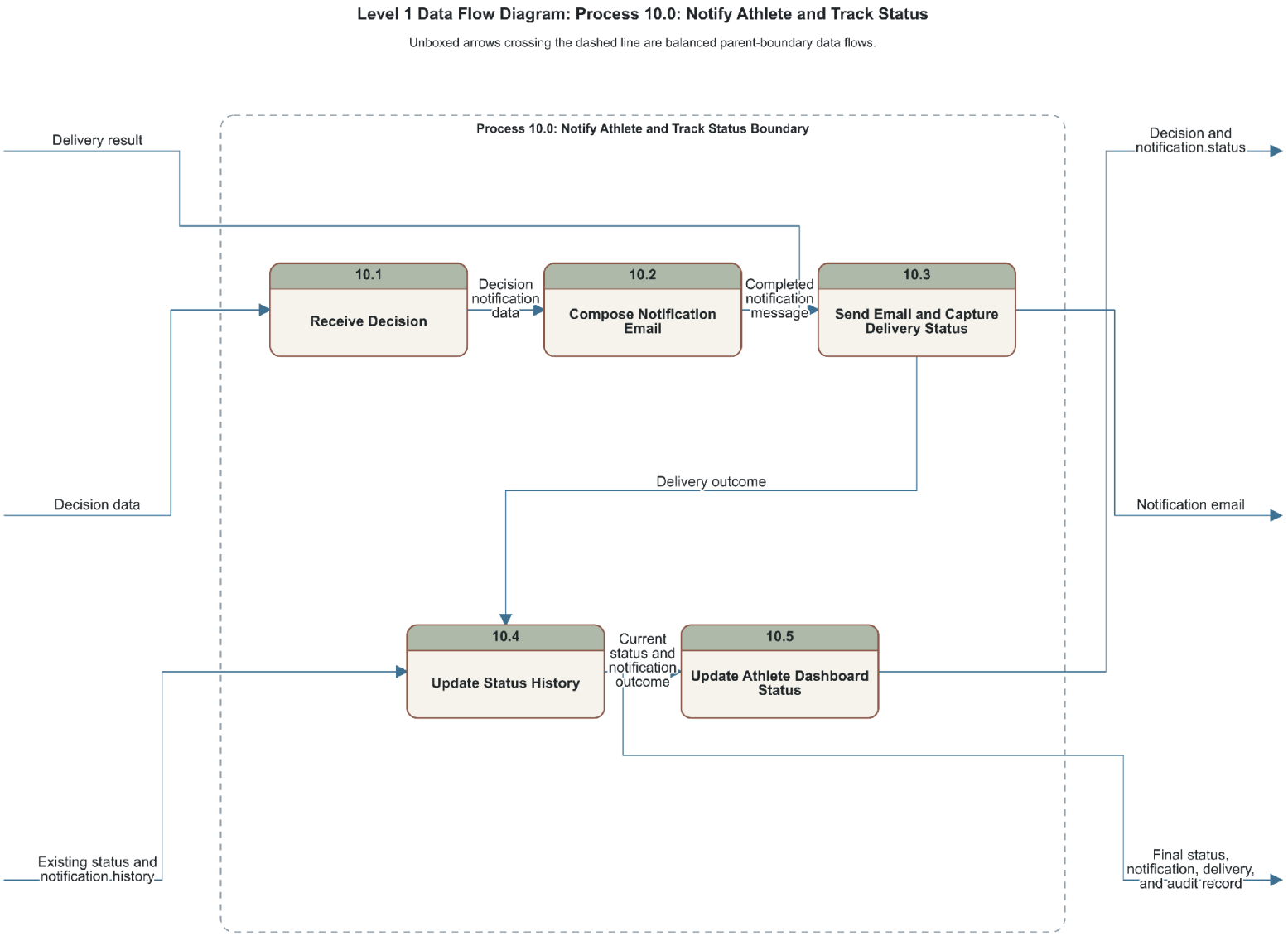


Figure 3.12. Level 1 decomposition of Process 10.0, Notify Athlete and Track Status.

Process Descriptions

This section describes the purpose and behavior of every process shown in the Context, Level 0, and Level 1 diagrams. Each description contains three to five sentences as required.

Process 0: Athlete Housing Search and Residence Approval System

This process represents the complete athlete housing search and residence approval system. It receives UID sign-in data, profile and search requests, residence selections, housing budget data, review decisions, approved website settings, listing data, and email delivery results from the system's external entities. It returns authorized dashboards, athlete profiles, housing budget information, residence search results, request status, administrative results, and decision notifications. The context diagram treats the system as one process and does not display its internal processes or data stores.

Level 0 Process Descriptions

Process 1.0: Authenticate User and Manage Session

This process controls system access for athletes, Athletic Department Reviewers, and System Admin users. It validates each UID through the University UID Authentication Source, reads the user's active status and assigned access from D1 User and Role Records, and manages the related USER_SESSION record. It returns the authorized dashboard or an access result and writes failed authentication and session events as AUDIT_EVENT records in D7 Status and Audit History.

Process 2.0: Manage Athlete Profile

This process maintains the athlete information used throughout the housing workflow. It verifies access through D1 User and Role Records, retrieves or corrects the logical athlete profile in D2 Athlete Profiles, and links that profile to housing budget, residence request, and request status information in D3, D6, and D7. It returns an authorized profile view or update result and records approved corrections as AUDIT_EVENT records in D7 Status and Audit History.

Process 3.0: Manage Housing Budget

This process maintains the current housing budget assigned to each athlete. It accepts a manual change from a System Admin or imports approved records from the Approved University Budget Data Source, validates the athlete and amount, and updates HOUSING_BUDGET in D3 Budgets and Budget Change History. It preserves each completed change in BUDGET_CHANGE, writes an AUDIT_EVENT record in D7 Status and Audit History, and returns the maintenance result to the System Admin.

Process 4.0: Manage Roles, Permissions, and Security

This process controls the access assigned to system users. It validates System Admin changes against the current ROLE, PERMISSION, ROLE_PERMISSION, and USER_ROLE records in D1 User and Role Records and applies only authorized updates. It records each completed change as an AUDIT_EVENT in D7 Status and Audit History and returns either the update result or the requested security audit report.

Process 5.0: Manage Approved Rental Websites

This process maintains the websites that the system may use to collect residence listings. It accepts add, edit, activate, deactivate, or remove requests from a System Admin, checks the website name and web address, and prevents duplicate approved sources. It updates APPROVED_WEBSITE_SOURCE in D4 Approved Website Sources, returns the result, and records each completed source change as an AUDIT_EVENT in D7 Status and Audit History.

Process 6.0: Collect and Clean Residence Listings

This process gathers residence listings from active records in D4 Approved Website Sources. It requests data from each approved rental or residential website, standardizes supported listing fields, and identifies new, changed,

duplicate, or inactive listings. It stores each collection attempt in COLLECTION_RUN and each cleaned listing in RESIDENCE_LISTING within D5 Residence Listings and Source Data, writes an AUDIT_EVENT in D7, and returns a collection report to the System Admin.

Process 7.0: Search Listings and View Details

This process allows an athlete to search, filter, sort, and open active residence listings. It reads address, monthly rent, bedrooms, bathrooms, availability, and approved source information from D5 Residence Listings and Source Data and reads Current_Amount from HOUSING_BUDGET in D3. It returns matching listings and calculates Estimated_Out_Of_Pocket as the greater of zero or Monthly_Rent minus Current_Amount.

Process 8.0: Submit Residence Request

This process creates a residence request for the listing selected by an athlete. It combines the athlete, listing, and housing budget references with optional athlete notes, Budget_Amount_At_Submission, Estimated_Out_Of_Pocket, and Out_Of_Pocket_Confirmed. It saves the request in RESIDENCE_REQUEST within D6 with a Submitted status, returns the Request_ID to the athlete, and records the submission in REQUEST_STATUS_HISTORY and AUDIT_EVENT within D7.

Process 9.0: Review Request and Record Decision

This process supports review by an authorized Athletic Department Reviewer. It retrieves a Submitted request, changes its Current_Status to Under Review when review begins, and presents the athlete, residence, housing budget snapshot, out-of-pocket amount, and optional notes. It stores the Athletic Department Reviewer's action in REQUEST_REVIEW, updates RESIDENCE_REQUEST to Approved or Rejected, records status and audit information in D7, and forwards the decision to Process 10.0.

Process 10.0: Notify Athlete and Track Status

This process notifies an athlete after an Approved or Rejected decision is recorded. It prepares and sends the decision email through the Email Service and stores the delivery outcome in EMAIL_NOTIFICATION within D7 Status and Audit History. It records the final request status in REQUEST_STATUS_HISTORY, writes the notification event to AUDIT_EVENT, and keeps the current decision visible on the athlete dashboard even if email delivery fails.

Level 1 Process Descriptions

Level 1 Decomposition of Process 1.0: Authenticate User and Manage Session

Process 1.1: Receive Login, Logout, or Protected-Page Access Request

This process receives a UID sign-in request, logout command, protected-page request, or active-session request from an Athlete, Athletic Department Reviewer, or System Admin. It identifies the requesting user, requested action, and available session information. It routes UID sign-in data to validation and routes logout or active-session data to session management.

Process 1.2: Validate UID Credentials

This process sends the submitted UID to the University UID Authentication Source for verification. It receives the matched university identity and active-status result. It sends a validated identity to role and permission checking or sends failed validation details to the security event subprocess.

Process 1.3: Check Active Status, Role, and Permissions

This process determines whether the validated user may enter the system. It reads USER, USER_ROLE, ROLE, ROLE_PERMISSION, and PERMISSION data from D1 User and Role Records to confirm active status and allowed access. It sends authorized user information to session management and sends inactive, unassigned, or unauthorized results to access routing and security event recording.

Process 1.4: Create, Refresh, Expire, or End Session

This process manages the user's session after authorization or in response to a logout or timeout. It creates, refreshes, expires, or closes the appropriate USER_SESSION record in D1 User and Role Records. It sends the resulting session state and authorized route to access routing and sends the session event for audit recording.

Process 1.5: Route to Correct Dashboard or Block Access

This process uses the authorization and session results to select the user's next destination. It returns the appropriate athlete, Athletic Department Reviewer, or System Admin dashboard when access is allowed and returns a clear sign-in, timeout, logout, or access-denied result otherwise. It forwards blocked-access information to the security event subprocess without exposing protected data.

Process 1.6: Record Failed Login and Session Security Events

This process receives failed UID validations, blocked-access attempts, and session lifecycle events from the authentication subprocesses. It creates an AUDIT_EVENT record in D7 Status and Audit History with the event type, time, outcome, related object, and available details. It preserves the event for authorized security review without changing the user's assigned access.

Level 1 Decomposition of Process 2.0: Manage Athlete Profile

Process 2.1: Receive Profile Access or Update Request

This process receives an athlete's own profile request, an authorized Athletic Department Reviewer's profile request, or a System Admin correction request. It identifies the target athlete, requested action, requesting user, and available reason for access. It sends the request to permission validation before any athlete information is returned or changed.

Process 2.2: Validate Profile Access Permissions

This process confirms whether the requesting user may view or correct the selected athlete profile. It reads active USER_SESSION, USER_ROLE, ROLE_PERMISSION, and PERMISSION information from D1 User and Role Records and checks any required UID match. It routes authorized views to profile retrieval, authorized corrections to profile maintenance, and denied requests to the response subprocess.

Process 2.3: Retrieve Athlete Profile

This process retrieves the authorized logical athlete profile from D2 Athlete Profiles, which is implemented by the ATHLETE subtype and its related USER identity. It returns the athlete's current profile attributes and the identifiers needed to connect the profile with other housing records. It sends that information to the housing workflow linking subprocess.

Process 2.4: Validate and Save Profile Corrections

This process handles profile corrections submitted by an authorized System Admin. It compares the proposed values with the current ATHLETE data represented by D2 Athlete Profiles and saves only valid changes. It sends the update result to the response subprocess and writes the completed correction as an AUDIT_EVENT in D7 Status and Audit History.

Process 2.5: Link Profile to Housing Workflow Records

This process uses the athlete identifier to connect the profile with the current HOUSING_BUDGET record in D3, RESIDENCE_REQUEST records in D6, and REQUEST_STATUS_HISTORY records in D7. It assembles a current housing summary without copying or changing the related records. It sends the linked profile and housing summary to the response subprocess.

Process 2.6: Return Profile View or Update Result

This process prepares the response for the person who requested profile access or maintenance. It returns the athlete's authorized profile and housing summary, an authorized read-only view, or the result of a System Admin correction. It returns a validation or access-denied message instead of protected profile information when the request is not allowed.

Level 1 Decomposition of Process 3.0: Manage Housing Budget

Process 3.1: Receive Budget Change or Import Command

This process receives a manual housing budget change or an approved import command from the System Admin. A manual request identifies the athlete, proposed Current_Amount, and available change reason, while an import command identifies the approved university data request. It routes manual records to validation and routes import commands to the university budget data subprocess.

Process 3.2: Request and Receive University Budget Data

This process receives an approved import command from the intake subprocess. It requests athlete housing budget records from the Approved University Budget Data Source and receives the authorized import data. It sends the imported athlete identifiers and amounts to validation without changing D3 directly.

Process 3.3: Validate and Match Athlete Budget

This process checks that each manual or imported amount is valid and includes an athlete identifier from the approved command or source record. It uses current HOUSING_BUDGET information from D3 to match an existing assignment or identify a new one. It sends valid matched data to the update subprocess and sends any validation failure to the result subprocess.

Process 3.4: Retrieve Current Budget

This process retrieves the athlete's current HOUSING_BUDGET record and relevant BUDGET_CHANGE entries from D3 Budgets and Budget Change History. It supplies the existing Current_Amount and Budget_ID needed to apply and trace a maintenance action. It sends those records to the update and result path without performing a residence rent comparison.

Process 3.5: Update Current Budget

This process receives a validated housing budget assignment or change. It creates or updates the correct HOUSING_BUDGET record in D3 with the athlete identifier, Current_Amount, Effective_Date, and Budget_Status. It sends the completed old and new values to the change-recording subprocess.

Process 3.6: Record Budget Change and Audit Event

This process preserves the details of each completed housing budget update. It creates the corresponding BUDGET_CHANGE entry in D3 and writes an AUDIT_EVENT record in D7 with the source or acting user, time, outcome, and available reason. It sends the recorded maintenance outcome to the result subprocess.

Process 3.7: Return Budget Update or Import Result

This process receives the validation, update, and audit outcomes from the preceding subprocesses. It prepares a success result for a completed manual update or import and a clear failure result for data that could not be validated or saved. It returns the housing budget maintenance result to the System Admin.

Level 1 Decomposition of Process 4.0: Manage Roles, Permissions, and Security

Process 4.1: Receive Role or Permission Change

This process receives a role assignment, permission configuration change, or security audit report request from the System Admin. A change identifies the target user or role and the requested access, while a report request identifies the event scope needed. It routes changes to validation and sends report criteria to the audit reporting subprocess.

Process 4.2: Validate Against Current Roles

This process checks a proposed access change against USER, ROLE, PERMISSION, USER_ROLE, and ROLE_PERMISSION records in D1 User and Role Records. It confirms that the target exists and that the requested role or permission relationship is valid. It sends an authorized change to the update subprocess and sends a failure result to reporting without altering D1.

Process 4.3: Apply Role and Permission Updates

This process receives an authorized role or permission change. It updates the appropriate USER_ROLE or ROLE_PERMISSION relationship in D1 User and Role Records while preserving the defined ROLE and PERMISSION records. It sends the completed update summary to reporting and sends the change event to audit recording.

Process 4.4: Compile Audit Reports

This process receives a System Admin report request, access update result, or validation failure. It reads relevant AUDIT_EVENT records from D7 Status and Audit History and organizes the authorized information into the requested security report or maintenance response. It returns the report or update result to the System Admin.

Process 4.5: Record Audit Event

This process receives a completed role or permission change from the update subprocess. It creates an AUDIT_EVENT record in D7 Status and Audit History that identifies the acting user, related object, event type, time, outcome, and available details. It preserves the access change for later security review.

Level 1 Decomposition of Process 5.0: Manage Approved Rental Websites

Process 5.1: Receive Website Setting Request

This process receives an add, edit, activate, deactivate, or remove request from the System Admin. It captures the source name, web address, requested active state, and requested action. It sends the proposed website setting to validation before D4 is changed.

Process 5.2: Validate URL and Check Duplicates

This process checks whether the submitted web address is usable and unique. It compares the proposed source with APPROVED_WEBSITE_SOURCE records in D4 Approved Website Sources and verifies that the requested action applies to an existing record when required. It sends a valid source change or a clear validation failure to the update subprocess.

Process 5.3: Update Approved Source List

This process receives the validated website request and requested action. It adds, edits, activates, deactivates, or removes the appropriate APPROVED_WEBSITE_SOURCE record in D4 and preserves the approving System Admin and validation information supported by the record. It returns the source maintenance result to the System Admin and sends completed change details to audit recording.

Process 5.4: Record Source Change

This process receives the completed approved-source change from the update subprocess. It writes an AUDIT_EVENT record in D7 Status and Audit History with the acting user, source identifier, action, time, and outcome. It preserves the event for administrative and security review.

Level 1 Decomposition of Process 6.0: Collect and Clean Residence Listings

Process 6.1: Load Active Source List

This process reads active APPROVED_WEBSITE_SOURCE records from D4 Approved Website Sources. It excludes inactive or unapproved sources and prepares the identifiers needed for collection. It sends the approved collection list to the website retrieval subprocess.

Process 6.2: Fetch Listings from Websites

This process starts run tracking for each approved source and sends a listing collection request to the related Approved Rental or Residential Website. It receives listing data and source results and continues processing other approved sources when one collection attempt fails. It carries the run information with collected data to standardization and sends failure details to collection status recording.

Process 6.3: Parse and Standardize Listing Data

This process converts collected data into the fields supported by RESIDENCE_LISTING. It standardizes the external listing identifier, address, Monthly_Rent, Bedrooms, Bathrooms, Available_Date, and related approved source identifier and marks unavailable values clearly. It sends the standardized records to duplicate and update detection.

Process 6.4: Detect Duplicates and Updates

This process compares each standardized record with existing RESIDENCE_LISTING data in D5 Residence Listings and Source Data. It classifies the record as new, changed, duplicate, or evidence that an existing listing should become inactive. It sends the listing and its Active_Flag or Duplicate_Flag decision to the storage subprocess.

Process 6.5: Save Clean Listings and Flags

This process receives the standardized listings and data quality decisions. It creates or updates RESIDENCE_LISTING records in D5 and completes each COLLECTION_RUN with its Run_Status and available Failure_Details. It sends the combined collection outcome to status recording.

Process 6.6: Record Collection Status

This process receives completed collection outcomes and source failures. It writes the collection event as an AUDIT_EVENT in D7 Status and Audit History while detailed run results remain in COLLECTION_RUN within D5. It returns the combined collection status and failure report to the System Admin.

Level 1 Decomposition of Process 7.0: Search Listings and View Details

Process 7.1: Receive Search or Detail Request

This process receives an athlete's listing search, filter, sort, or detail request. It separates criteria for a set of listings from the identifier for one selected listing. It routes general criteria to filtering and routes the selected identifier to detail retrieval.

Process 7.2: Apply Filters and Sort

This process reads active, nonduplicate RESIDENCE_LISTING records from D5 Residence Listings and Source Data. It applies supported criteria for address, Monthly_Rent, Bedrooms, Bathrooms, Available_Date, and approved source identifier and orders the matches using the athlete's selected sort. It sends the matching records to the detail and result subprocesses.

Process 7.3: Retrieve Listing Details

This process receives the listing records retrieved from D5 by the filter and sort subprocess and selects the requested Listing_ID. It assembles the supported address, Monthly_Rent, Bedrooms, Bathrooms, Available_Date, active status, and approved source identifier. It sends the listing details and Monthly_Rent to the housing budget comparison subprocess.

Process 7.4: Compare Rent to Budget

This process receives Monthly_Rent for the selected listing and reads the athlete's Current_Amount from HOUSING_BUDGET in D3. It calculates Estimated_Out_Of_Pocket as the greater of zero or Monthly_Rent minus Current_Amount and determines whether the rent fits within the assigned amount. It sends the calculation and housing budget fit result to the result subprocess.

Process 7.5: Return Results and Budget Fit

This process combines the matching listing set with any selected listing details and housing budget calculation. It returns only supported address, rent, bedroom, bathroom, availability, approved source, and active-listing information. It presents the housing budget fit and Estimated_Out_Of_Pocket to the athlete without changing the listing or housing budget records.

Level 1 Decomposition of Process 8.0: Submit Residence Request

Process 8.1: Receive Residence Selection

This process receives the Listing_ID selected by the athlete. It also receives optional Athlete_Notes and the athlete's Out_Of_Pocket_Confirmed value. It sends the selection and athlete identifier to request assembly.

Process 8.2: Assemble Request Package

This process links the athlete represented by D2 Athlete Profiles with the selected RESIDENCE_LISTING in D5. It assembles Athlete_ID, Listing_ID, optional Athlete_Notes, and Monthly_Rent for submission validation. It sends the request draft to the calculation and confirmation subprocess, which adds the current housing budget data from D3.

Process 8.3: Calculate Out-of-Pocket and Confirm

This process calculates Estimated_Out_Of_Pocket as the greater of zero or Monthly_Rent minus Current_Amount. It sets Budget_Amount_At_Submission to the current amount and verifies Out_Of_Pocket_Confirmed when the calculated amount is greater than zero. It sends a complete request to storage or returns a validation failure when required confirmation is absent.

Process 8.4: Save Submitted Request or Return Error

This process receives a validated residence request or a validation failure. It creates a RESIDENCE_REQUEST record in D6 with its athlete, listing, and housing budget references, snapshot values, Submitted_At value, and Current_Status of Submitted. It returns the new Request_ID and Submitted status to the athlete or returns the validation error without creating a request.

Process 8.5: Record Submission Event

This process receives the new Request_ID and submission result from request storage. It creates a Submitted entry in REQUEST_STATUS_HISTORY and a related AUDIT_EVENT in D7 Status and Audit History. It preserves the start of the request lifecycle for later review and reporting.

Level 1 Decomposition of Process 9.0: Review Request and Record Decision

Process 9.1: Retrieve Submitted Request Package

This process retrieves a Submitted RESIDENCE_REQUEST and its housing budget snapshot, out-of-pocket amount, optional notes, and related identifiers from D6 Residence Requests. When authorized review begins, it sends an Under Review update to request storage and sends the transition for REQUEST_STATUS_HISTORY and AUDIT_EVENT recording in D7. It sends the Under Review request data to presentation.

Process 9.2: Present Package to Athletic Department Reviewer

This process receives the Under Review request data from D6, the related athlete profile from D2, and selected listing details from D5. It organizes the supported athlete, residence, housing budget snapshot, Estimated_Out_Of_Pocket, Out_Of_Pocket_Confirmed, and optional Athlete_Notes for an authorized Athletic Department Reviewer. It displays the package and requests an approval or rejection decision.

Process 9.3: Capture Athletic Department Reviewer Decision

This process receives an Approved or Rejected decision from the authorized Athletic Department Reviewer. It associates the action with the correct Request_ID and Reviewer_ID and accepts optional Reviewer_Comments, including an optional explanation for a rejection. It sends the complete review action to decision storage.

Process 9.4: Record Decision in Request

This process receives the review-start update and the Athletic Department Reviewer's completed action. It first updates RESIDENCE_REQUEST in D6 to Under Review, then stores Reviewer_ID, Decision, optional Reviewer_Comments, and Decision_Date in REQUEST_REVIEW and updates Current_Status to Approved or Rejected. It sends the recorded final decision to the forwarding and audit subprocess.

Process 9.5: Forward Decision and Record Audit

This process receives the Under Review transition and the recorded Approved or Rejected decision. It writes the Under Review transition to REQUEST_STATUS_HISTORY, records the review action in AUDIT_EVENT, and sends the final decision data to Process 10.0. Process 10.0 records the Approved or Rejected history entry with the notification outcome, which avoids a duplicate final status entry.

Level 1 Decomposition of Process 10.0: Notify Athlete and Track Status

Process 10.1: Receive Decision

This process receives an Approved or Rejected residence request decision from Process 9.0. The decision data includes Request_ID, Athlete_ID, the selected residence reference, Current_Status, Decision_Date, and any optional Reviewer_Comments. It sends the verified decision details to notification composition.

Process 10.2: Compose Notification Email

This process prepares an approval or rejection email for the athlete. It includes the selected residence, final request status, Decision_Date, and any optional comments or rejection explanation provided by the Athletic Department Reviewer. It sends the completed decision message to email delivery.

Process 10.3: Send Email and Capture Delivery Status

This process sends the decision notification through the Email Service. It receives the delivery result and available failure details without changing the Approved or Rejected request decision. It sends the message type, send time, delivery status, and failure details to status and notification recording.

Process 10.4: Update Status History

This process receives the final decision and email outcome. It creates the Approved or Rejected REQUEST_STATUS_HISTORY entry, stores the delivery result in EMAIL_NOTIFICATION, and writes the notification action as an AUDIT_EVENT within D7 Status and Audit History. It sends the current request status and notification outcome to dashboard updating even when delivery fails.

Process 10.5: Update Athlete Dashboard Status

This process receives the current request status and notification outcome. It prepares the Approved or Rejected result for the athlete dashboard using the decision data from Process 9.0 and the related D7 history. It returns the final decision and notification status to the athlete so the result remains available inside the system.

DFD Balance and Final Consistency Check

The Context diagram contains 34 external flows. The Level 0 diagram repeats all 34 labels and directions, contains ten processes, and uses data stores D1 through D7. Each Level 1 diagram repeats every flow that crosses its parent Level 0 process boundary.

Parent process	Parent boundary flows	Level 1 boundary flows	Result
1.0	11	11	Balanced
2.0	13	13	Balanced
3.0	8	8	Balanced
4.0	6	6	Balanced
5.0	5	5	Balanced
6.0	7	7	Balanced
7.0	4	4	Balanced
8.0	8	8	Balanced
9.0	8	8	Balanced

10.0	6	6	Balanced

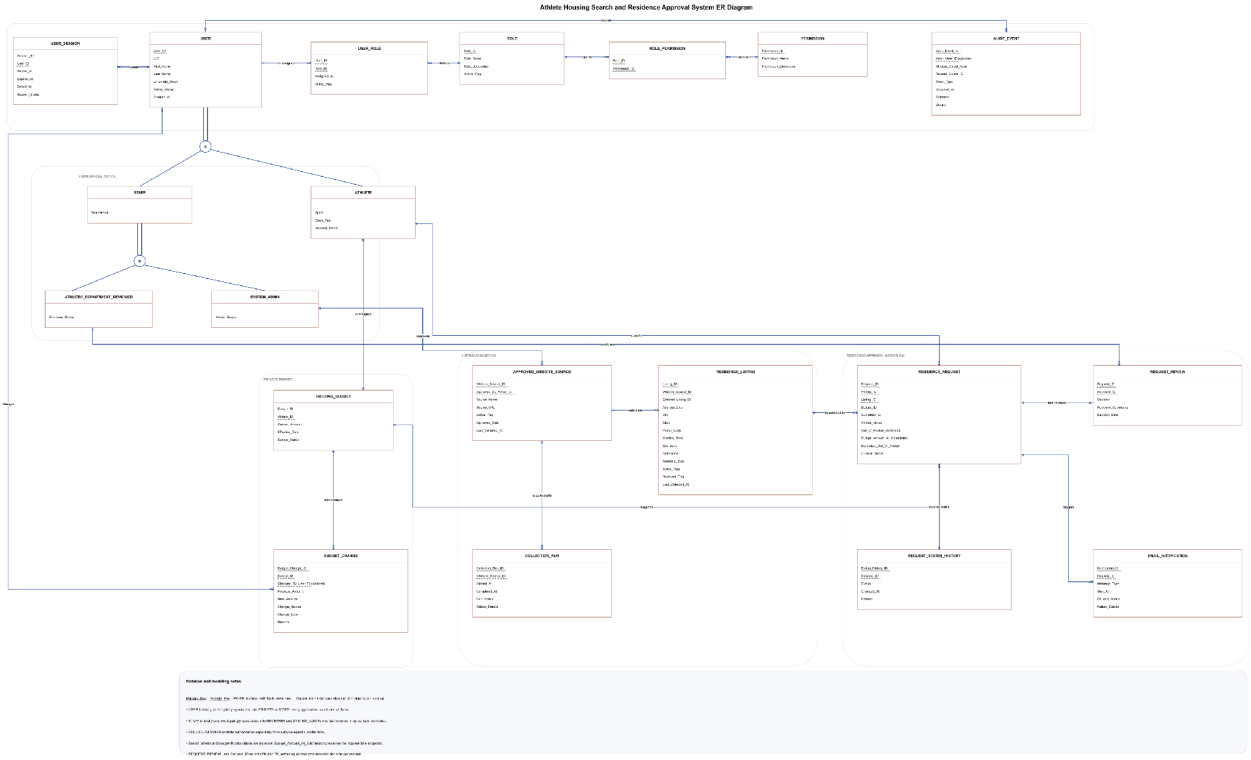
The request lifecycle is Submitted, Under Review, then Approved or Rejected. The diagrams and descriptions use the updated logical design and the final relational tables, including REQUEST_REVIEW, REQUEST_STATUS_HISTORY, EMAIL_NOTIFICATION, AUDIT_EVENT, COLLECTION_RUN, and BUDGET_CHANGE.

4

Milestone 4

Entity Relationship Diagram, Data Dictionary, Domains and Triggers

Final ER Diagram



Trigger Summary

Athlete Housing ERD - Trigger Catalog

Trigger fields follow the course example: user rule, event, entity, condition, and action. These are logical trigger specifications; implementation may use DB triggers, deferred constraints, row-level security, or controlled stored procedures as appropriate.

Trigger ID	Source Process	User Rule	Timing	Event	Entity Name	Condition	Action
TRG-01	1.0 Authenticate User and Manage Session	A session may be opened only for an active USER who has at least one active ROLE assignment.	BEFORE	INSERT	USER_SESSION	Referenced USER is inactive, or the USER has no active USER_ROLE linked to an active ROLE.	Reject the insert and write a LOGIN_FAILED AUDIT_EVENT.
TRG-02	1.0 Authenticate User and Manage Session	Session dates and status must remain internally consistent.	BEFORE	INSERT or UPDATE	USER_SESSION	Expires_At <= Started_At; Ended_At < Started_At; or Session_Status is ACTIVE while Ended_At is populated.	Reject the transaction, or set the correct terminal status when the end event is system-generated.
TRG-03	1.0 Authenticate User and Manage Session	Every login, logout, timeout, expiration, and session termination must be auditable.	AFTER	INSERT, UPDATE, or DELETE	USER_SESSION	A session is created, its status/end time changes, or it is removed.	Insert an AUDIT_EVENT containing the user, session identifier, event type, time, outcome, and details.
TRG-04	1.0 / 2.0 / 4.0 User Classification	USER is a total, disjoint supertype: every USER must be exactly one of ATHLETE or STAFF.	DEFERRED AFTER	INSERT, UPDATE, or DELETE	USER, ATHLETE, STAFF	A USER has neither subtype, or the same USER appears in both ATHLETE and STAFF.	Reject the transaction at commit and preserve the prior valid subtype membership.
TRG-05	4.0 Manage Roles, Permissions, and Security	STAFF is a total, overlapping supertype: every STAFF member must be a reviewer, a system administrator, or both.	DEFERRED AFTER	INSERT, UPDATE, or DELETE	STAFF, ATHLETIC_DEPARTMENT_REVIEWER, SYSTEM_ADMIN	A STAFF record belongs to neither subtype.	Reject the transaction at commit. Membership in both subtypes is allowed.
TRG-06	2.0 Manage Athlete Profile	Approved corrections to user or athlete profile data must be recorded in audit history.	AFTER	UPDATE	USER or ATHLETE	Any profile attribute changes.	Insert an AUDIT_EVENT with actor, affected user, changed attributes, time, outcome, and correction details.
TRG-07	4.0 Manage Roles, Permissions, and Security	A role assignment may reference only an active USER and an active ROLE.	BEFORE	INSERT or UPDATE	USER_ROLE	USER.Active_Status is not ACTIVE, ROLE.Active_Flag is FALSE, or the same active user-role pair already exists.	Reject the transaction.
TRG-08	4.0 Manage Roles, Permissions, and Security	Every role, user-role, permission, or role-permission change must be auditable.	AFTER	INSERT, UPDATE, or DELETE	USER_ROLE, ROLE, ROLE_PERMISSION, PERMISSION	An access-control record is added, changed, activated, deactivated, or removed.	Insert an AUDIT_EVENT containing actor, object type/id, event type, time, outcome, and before/after details.
TRG-09	1.0-10.0 Status and Audit History	Audit history is append-only.	BEFORE	UPDATE or DELETE	AUDIT_EVENT	Any existing AUDIT_EVENT would be changed or removed.	Reject the transaction; corrections must be represented by a new AUDIT_EVENT.
TRG-10	3.0 Manage Housing Budget	An athlete may have only one current active housing budget.	BEFORE	INSERT or UPDATE	HOUSING_BUDGET	Another HOUSING_BUDGET for the same Athlete_ID already has Budget_Status = ACTIVE.	Reject the transaction or supersede the prior budget in the same controlled transaction.
TRG-11	3.0 Manage Housing Budget	A budget may be changed only by an authorized SYSTEM_ADMIN or an approved university import process.	BEFORE	INSERT or UPDATE	HOUSING_BUDGET	The database session actor is not an authorized SYSTEM_ADMIN and the source is not an approved import.	Reject the transaction.
TRG-12	3.0 Manage Housing Budget	Every change to Current_Amount must create a complete budget-change history record.	AFTER	UPDATE	HOUSING_BUDGET	OLD.Current_Amount <> NEW.Current_Amount.	Insert BUDGET_CHANGE with old amount, new amount, source/actor, date, and reason.
TRG-13	3.0 Manage Housing Budget	A BUDGET_CHANGE record must agree with the budget values and with its change source.	BEFORE	INSERT	BUDGET_CHANGE	Previous_Amount does not match the prior amount; New_Amount does not match the resulting budget; or Change_Source and Changed_By_User_ID are inconsistent.	Reject the insert. Changed_By_User_ID may be null only for an approved university import.
TRG-14	3.0 Manage Housing Budget	Every completed budget change must also appear in audit history.	AFTER	INSERT	BUDGET_CHANGE	A valid BUDGET_CHANGE is created.	Insert a BUDGET_CHANGE AUDIT_EVENT with the budget, athlete, actor/source, amounts, date, reason, and success.
TRG-15	3.0 Manage Housing Budget	Budget-change history is append-only.	BEFORE	UPDATE or DELETE	BUDGET_CHANGE	An existing BUDGET_CHANGE would be changed or removed.	Reject the transaction; create a new correcting change record instead.
TRG-16	5.0 Manage Approved Rental Websites	An approved website must have a usable, unique source name and HTTPS URL.	BEFORE	INSERT or UPDATE	APPROVED_WEBSITE_SOURCE	Source_URL is malformed or non-HTTPS, or an active record already uses the same normalized Source_URL or	Reject the transaction and return a validation or duplicate-source error.
TRG-17	5.0 Manage Approved Rental Websites	A website source referenced by listings or collection history may not be physically deleted.	BEFORE	DELETE	APPROVED_WEBSITE_SOURCE	The source is referenced by RESIDENCE_LISTING or COLLECTION_RUN.	Reject the delete and require Active_Flag to be set to FALSE instead.
TRG-18	5.0 Manage Approved Rental Websites	Every add, edit, disable, or permitted removal of an approved website must be auditable.	AFTER	INSERT, UPDATE, or DELETE	APPROVED_WEBSITE_SOURCE	A source record changes.	Insert a WEBSITE_SOURCE_CHANGE AUDIT_EVENT with actor, source, time, outcome, and before/after values.

Athlete Housing ERD - Trigger Catalog

Trigger fields follow the course example: user rule, event, entity, condition, and action. These are logical trigger specifications; implementation may use DB triggers, deferred constraints, row-level security, or controlled stored procedures as appropriate.

Trigger ID	Source Process	User Rule	Timing	Event	Entity Name	Condition	Action
TRG-22	6.0 Collect and Clean Residence Listings	A residence listing may be inserted or refreshed only from its approved source.	BEFORE	INSERT or UPDATE	RESIDENCE_LISTING	Website_Source_ID is not active/approved or does not match the collection source supplying the row.	Reject the transaction.
TRG-23	6.0 Collect and Clean Residence Listings	Duplicate listings must be detected within each approved source.	BEFORE	INSERT or UPDATE	RESIDENCE_LISTING	Another row has the same normalized Website_Source_ID and External_Listing_ID, or matches the configured address/rent duplicate rule.	Set Duplicate_Flag = TRUE and retain the duplicate decision; reject only when the implementation forbids storing duplicate rows.
TRG-24	6.0 Collect and Clean Residence Listings	A completed collection run must stamp refreshed listings and inactivate previously active listings no longer returned by that source.	AFTER	UPDATE	COLLECTION_RUN	Run_Status changes to COMPLETED.	Set Last_Collected_At for rows seen in the run and set Active_Flag = FALSE for previously active source listings not seen in the completed run.
TRG-25	8.0 Submit Residence Request	A residence request may be submitted only by an active athlete for an active, nonduplicate listing using that athlete's current budget.	BEFORE	INSERT	RESIDENCE_REQUEST	ATHLETE/USER is inactive; listing is inactive or duplicate; budget is not ACTIVE; or Budget_ID does not belong to Athlete_ID.	Reject the insert.
TRG-26	7.0 / 8.0 Budget Comparison and Submission	The request must preserve the submission-time budget and calculated out-of-pocket amount.	BEFORE	INSERT	RESIDENCE_REQUEST	The request is otherwise eligible for submission.	Set Budget_Amount_At_Submission from HOUSING_BUDGET.Current_Amount and set Estimated_Out_Of_Pocket = max(Monthly_Rent - budget
TRG-27	8.0 Submit Residence Request	The athlete must explicitly confirm any positive out-of-pocket amount.	BEFORE	INSERT	RESIDENCE_REQUEST	Estimated_Out_Of_Pocket > 0 and Out_Of_Pocket_Confirmed is not TRUE.	Reject the insert and request confirmation.
TRG-28	8.0 Submit Residence Request	A new residence request begins in Pending Review status with a system timestamp.	BEFORE	INSERT	RESIDENCE_REQUEST	Current_Status or Submitted_At is null or supplied with a conflicting value.	Set Current_Status = PENDING_REVIEW and Submitted_At = current UTC timestamp.
TRG-29	8.0 Submit Residence Request	Every submitted request must immediately create status and audit history.	AFTER	INSERT	RESIDENCE_REQUEST	A valid request is inserted.	Insert REQUEST_STATUS_HISTORY(PENDING_REVIEW) and a REQUEST_SUBMITTED AUDIT_EVENT.
TRG-30	9.0 Review Request and Record Decision	Request status may change only through the review-decision workflow and only from Pending Review to Approved or Rejected.	BEFORE	UPDATE	RESIDENCE_REQUEST	Current_Status is changed directly, or the transition is not PENDING_REVIEW -> APPROVED/REJECTED.	Reject the update unless it is executed by the authorized review-decision transaction.
TRG-31	9.0 Review Request and Record Decision	A review may be recorded only by an active reviewer for a pending request that has no prior review.	BEFORE	INSERT	REQUEST_REVIEW	Reviewer_ID is not an active ATHLETIC_DEPARTMENT_REVIEWER; request is not PENDING_REVIEW; or a REQUEST_REVIEW already exists.	Reject the insert.
TRG-32	9.0 Review Request and Record Decision	A review decision must be Approved or Rejected, and rejection requires reviewer comments.	BEFORE	INSERT or UPDATE	REQUEST_REVIEW	Decision is outside the allowed set, or Decision = REJECTED and Reviewer_Comments is null/blank.	Reject the transaction.
TRG-33	9.0 Review Request and Record Decision	A recorded review decision must update the request and both histories atomically.	AFTER	INSERT	REQUEST_REVIEW	A valid decision is inserted.	Update RESIDENCE_REQUEST.Current_Status; insert matching REQUEST_STATUS_HISTORY; insert REQUEST_APPROVED or REQUEST_REJECTED.
TRG-34	9.0 / 10.0 Decision Notification	A recorded decision must create a pending email notification.	AFTER	INSERT	REQUEST_REVIEW	Decision is APPROVED or REJECTED.	Insert EMAIL_NOTIFICATION with matching Message_Type and Delivery_Status = PENDING.
TRG-35	9.0 Review Request and Record Decision	A recorded review decision is immutable.	BEFORE	UPDATE or DELETE	REQUEST_REVIEW	An existing review would be changed or removed.	Reject the transaction; administrative corrections require a separately audited corrective workflow.
TRG-36	8.0-10.0 Request Status Tracking	Request status history is append-only.	BEFORE	UPDATE or DELETE	REQUEST_STATUS_HISTORY	An existing history row would be changed or removed.	Reject the transaction; record a new status-history row instead.
TRG-37	10.0 Notify Athlete and Track Status	A notification may be created only for a decided request, and its message type must match the	BEFORE	INSERT	EMAIL_NOTIFICATION	Request status is not APPROVED/REJECTED, or Message_Type does not correspond to the current decision.	Reject the insert.
TRG-38	10.0 Notify Athlete and Track Status	Email delivery fields must represent a valid state transition.	BEFORE	UPDATE	EMAIL_NOTIFICATION	Delivery_Status does not transition PENDING -> SENT/FAILED; SENT lacks Sent_At; FAILED lacks Failure_Details; or SENT contains failure details.	Reject the update until the delivery data is consistent.
TRG-39	10.0 Notify Athlete and Track Status	Every email outcome must be audited, and delivery failure must not reverse the recorded housing decision.	AFTER	INSERT or UPDATE	EMAIL_NOTIFICATION	A notification is queued or Delivery_Status changes to SENT or FAILED.	Insert EMAIL_QUEUED, EMAIL_SENT, or EMAIL_FAILED AUDIT_EVENT; leave RESIDENCE_REQUEST.Current_Status unchanged when

Attribute Domains

Athlete Housing ERD - Attribute Domain Catalog

Every attribute shown in the logical ERD is listed once. A named domain may be reused by multiple attributes; nullability and uniqueness are specified at the attribute assignment level.

Domain ID	Named Domain	Entity	Attribute Name	Key Role	Meaning	Data Type	Format / Allowed Values / Range	Uniqueness	Null Support	Enforcement Notes
DOM-001	D_SESSION_ID	USER_SESSION	Session_ID	PK	Unique identifier for an authenticated user session.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key.
DOM-002	D_USER_ID	USER_SESSION	User_ID	FK	Identifier of the related USER; in USER it is the user's surrogate primary identifier.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique	Non-null	Foreign key; References USER.User_ID.
DOM-003	D_TIMESTAMP_UTC	USER_SESSION	Started_At	Non-key	UTC date/time when the authenticated session began.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Non-null	
DOM-004	D_TIMESTAMP_UTC	USER_SESSION	Expires_At	Non-key	UTC date/time after which the session is no longer valid.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Non-null	Must be later than Started_At.
DOM-005	D_TIMESTAMP_UTC	USER_SESSION	Ended_At	Non-key	UTC date/time when the session ended, if it has ended.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Nullable while the session is active	When present, must be on/after Started_At.
DOM-006	D_SESSION_STATUS	USER_SESSION	Session_Status	Non-key	Current lifecycle state of the user session.	VARCHAR(20)	One of: ACTIVE, EXPIRED, LOGGED_OUT, TERMINATED	Nonunique	Non-null	
DOM-007	D_USER_ID	USER	User_ID	PK	Identifier of the related USER; in USER it is the user's surrogate primary identifier.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key.
DOM-008	D_UNIVERSITY_UID	USER	UID	Non-key	University-issued identity value used for authentication and identity matching.	VARCHAR(32)	Institution-issued alphanumeric UID, 1-32 characters	Must be unique	Non-null	
DOM-009	D_PERSON_NAME	USER	First_Name	Non-key	User's legal or university-record first name.	VARCHAR(100)	1-100 Unicode characters, trimmed; no control characters	Nonunique	Non-null	
DOM-010	D_PERSON_NAME	USER	Last_Name	Non-key	User's legal or university-record last name.	VARCHAR(100)	1-100 Unicode characters, trimmed; no control characters	Nonunique	Non-null	
DOM-011	D_UNIVERSITY_EMAIL	USER	University_Email	Non-key	User's official university email address.	VARCHAR(254)	Valid email syntax and approved university domain; stored	Must be unique	Non-null	
DOM-012	D_USER_ACTIVE_STATUS	USER	Active_Status	Non-key	Whether the user account is active and eligible to use the system.	VARCHAR(10)	One of: ACTIVE, INACTIVE	Nonunique	Non-null	
DOM-013	D_TIMESTAMP_UTC	USER	Created_At	Non-key	UTC date/time when the USER record was created.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Non-null	
DOM-014	D_USER_ID	USER_ROLE	User_ID	PK/FK	Identifier of the related USER; in USER it is the user's surrogate primary identifier.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique individually; composite PK pair must be unique	Non-null	Primary key and foreign key; References USER.User_ID.
DOM-015	D_ROLE_ID	USER_ROLE	Role_ID	PK/FK	Identifier of the related ROLE.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique individually; composite PK pair must be unique	Non-null	Primary key and foreign key; References ROLE.Role_ID.
DOM-016	D_TIMESTAMP_UTC	USER_ROLE	Assigned_At	Non-key	UTC date/time when the role was assigned to the user.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Non-null	
DOM-017	D_BOOLEAN	USER_ROLE	Active_Flag	Non-key	Boolean indicator that the record is currently active.	BOOLEAN	TRUE or FALSE	Nonunique	Non-null	
DOM-018	D_ROLE_ID	ROLE	Role_ID	PK	Identifier of the related ROLE.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key.
DOM-019	D_ROLE_NAME	ROLE	Role_Name	Non-key	Unique controlled name of a security role.	VARCHAR(60)	Controlled unique role name; initial values include ATHLETE, REVIEWER, SYSTEM_ADMIN.	Must be unique	Non-null	
DOM-020	D_DESCRIPTION_500	ROLE	Role_Description	Non-key	Human-readable explanation of the role.	VARCHAR(500)	0-500 characters	Nonunique	Nullable	
DOM-021	D_BOOLEAN	ROLE	Active_Flag	Non-key	Boolean indicator that the record is currently active.	BOOLEAN	TRUE or FALSE	Nonunique	Non-null	
DOM-022	D_ROLE_ID	ROLE_PERMISSION	Role_ID	PK/FK	Identifier of the related ROLE.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique individually; composite PK pair must be unique	Non-null	Primary key and foreign key; References ROLE.Role_ID.
DOM-023	D_PERMISSION_ID	ROLE_PERMISSION	Permission_ID	PK/FK	Identifier of the related PERMISSION.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique individually; composite PK pair must be unique	Non-null	Primary key and foreign key; References PERMISSION.Permission_ID.
DOM-024	D_PERMISSION_ID	PERMISSION	Permission_ID	PK	Identifier of the related PERMISSION.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key.
DOM-025	D_PERMISSION_NAME	PERMISSION	Permission_Name	Non-key	Unique controlled name of an application permission.	VARCHAR(100)	Controlled unique permission code/name; uppercase snake_case recommended.	Must be unique	Non-null	
DOM-026	D_DESCRIPTION_500	PERMISSION	Permission_Description	Non-key	Human-readable explanation of the permission.	VARCHAR(500)	0-500 characters	Nonunique	Nullable	
DOM-027	D_AUDIT_EVENT_ID	AUDIT_EVENT	Audit_Event_ID	PK	Unique identifier for an audit-history event.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key.
DOM-028	D_USER_ID	AUDIT_EVENT	Actor_User_ID	FK	USER who performed the event; null only for an automated or unidentified event.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique	Nullable for automated/system or unidentified events	Foreign key; References USER.User_ID.
DOM-029	D_OBJECT_TYPE	AUDIT_EVENT	Related_Object_Type	Non-key	Controlled entity/object type affected by the event.	VARCHAR(60)	Controlled ERD entity/object name	Nonunique	Non-null	
DOM-030	D_OBJECT_IDENTIFIER_TEXT	AUDIT_EVENT	Related_Object_ID	Non-key	Identifier of the object affected by the audited event.	VARCHAR(64)	1-64 characters; stores the affected object's identifier as text.	Nonunique	Non-null	
DOM-031	D_AUDIT_EVENT_TYPE	AUDIT_EVENT	Event_Type	Non-key	Controlled code describing the audited activity.	VARCHAR(60)	Controlled code; including LOGIN_SUCCESS, LOGIN_FAILED, LOGOUT, SESSION_TIMEOUT, PROFILE_UPDATE, BUDGET_CHANGE, ROLE_CHANGE, PERMISSION_CHANGE, WEBSITE_SOURCE_CHANGE, COLLECTION_SUCCESS, COLLECTION_FAILURE, REQUEST_SUBMITTED, APPLICATION_APPROVED, APPLICATION_REJECTED	Nonunique	Non-null	
DOM-032	D_TIMESTAMP_UTC	AUDIT_EVENT	Occurred_At	Non-key	UTC date/time when the audited event occurred.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Non-null	
DOM-033	D_AUDIT_OUTCOME	AUDIT_EVENT	Outcome	Non-key	Controlled result of the audited activity.	VARCHAR(15)	One of: SUCCESS, FAILURE, REJECTED	Nonunique	Non-null	
DOM-034	D_LONG_TEXT	AUDIT_EVENT	Details	Non-key	Optional explanatory or before/after details for the event.	VARCHAR(4000)	0-4,000 characters	Nonunique	Nullable	
DOM-035	D_USER_ID	ATHLETE	User_ID	PK/FK	Identifier of the related USER; in USER it is the user's surrogate primary identifier.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key and foreign key; References USER.User_ID; total/disjoint subtype.
DOM-036	D_UNIVERSITY_SPORT	ATHLETE	Sport	Non-key	University sport associated with the athlete.	VARCHAR(80)	Value from the university-maintained list of sponsored sports	Nonunique	Non-null	
DOM-037	D_CLASS_YEAR	ATHLETE	Class_Year	Non-key	Athlete's academic class-year classification.	VARCHAR(12)	One of: FRESHMAN, SOPHOMORE, JUNIOR, SENIOR.	Nonunique	Non-null	
DOM-038	D_HOUSING_STATUS	ATHLETE	Housing_Status	Non-key	Athlete's current state in the housing workflow.	VARCHAR(20)	One of: NOT_STARTED, SEARCHING, REQUEST_PENDING, APPROVED, REJECTED, HOUSED	Nonunique	Non-null	

Athlete Housing ERD - Attribute Domain Catalog

Every attribute shown in the logical ERD is listed once. A named domain may be reused by multiple attributes; nullability and uniqueness are specified at the attribute assignment level.

Domain ID	Named Domain	Entity	Attribute Name	Key Role	Meaning	Data Type	Format / Allowed Values / Range	Uniqueness	Null Support	Enforcement Notes
DOM-038	D_HOUSING_STATUS	ATHLETE	Housing_Status	Non-key	Athlete's current state in the housing workflow.	VARCHAR(20)	One of: NOT_SUBMITTED, SUBMITTED, REQUEST_CLARIFY, APPROVED, REJECTED, HOUSED	Nonunique	Non-null	
DOM-039	D_USER_ID	STAFF	User_ID	PK/FK	Identifier of the related USER; in USER it is the user's surrogate primary identifier.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key and foreign key. References USER.User_ID; total/dissent subtype.
DOM-040	D_DEPARTMENT	STAFF	Department	Non-key	University department employing or sponsoring the staff member.	VARCHAR(120)	Value from the university department list; 1-120 characters	Nonunique	Non-null	
DOM-041	D_USER_ID	ATHLETIC_DEPARTMENT_REVIEWER	User_ID	PK/FK	Identifier of the related USER; in USER it is the user's surrogate primary identifier.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key and foreign key. References STAFF.User_ID; total/overlapping subtype.
DOM-042	D_REVIEWER_STATUS	ATHLETIC_DEPARTMENT_REVIEWER	Reviewer_Status	Non-key	Whether the reviewer is currently authorized to make housing decisions.	VARCHAR(10)	One of: ACTIVE, INACTIVE	Nonunique	Non-null	
DOM-043	D_USER_ID	SYSTEM_ADMIN	User_ID	PK/FK	Identifier of the related USER; in USER it is the user's surrogate primary identifier.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key and foreign key. References STAFF.User_ID; total/overlapping subtype.
DOM-044	D_ADMIN_SCOPE	SYSTEM_ADMIN	Admin_Scope	Non-key	Controlled administrative scope assigned to the system administrator.	VARCHAR(20)	One of: GLOBAL, HOUSING, SECURITY, DATA, AUDIT	Nonunique	Non-null	
DOM-045	D_BUDGET_ID	HOUSING_BUDGET	Budget_ID	PK	Identifier of the related HOUSING_BUDGET.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key.
DOM-046	D_USER_ID	HOUSING_BUDGET	Athlete_ID	FK	USER identifier of the related ATHLETE.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique for the active/current budget	Non-null	Foreign key. References ATHLETE.User_ID.
DOM-047	D_USD_NONNEGATIVE	HOUSING_BUDGET	Current_Amount	Non-key	Current university-approved monthly housing budget amount.	DECIMAL(12,2)	$0.00 to $9,999,999,999.99; exactly 2 decimal places	Nonunique	Non-null	
DOM-048	D_CALENDAR_DATE	HOUSING_BUDGET	Effective_Date	Non-key	Date on which the current budget became effective.	DATE	ISO date; YYYY-MM-DD	Nonunique	Non-null	For the current budget, normally not later than the current date.
DOM-049	D_BUDGET_STATUS	HOUSING_BUDGET	Budget_Status	Non-key	Lifecycle state of the housing budget.	VARCHAR(15)	One of: ACTIVE, INACTIVE, SUPERSEDED	Nonunique	Non-null	
DOM-050	D_BUDGET_CHANGE_ID	BUDGET_CHANGE	Budget_Change_ID	PK	Unique identifier for a budget-change history record.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key.
DOM-051	D_BUDGET_ID	BUDGET_CHANGE	Budget_ID	FK	Identifier of the related HOUSING_BUDGET.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique	Non-null	Foreign key. References HOUSING_BUDGET.Budget_ID.
DOM-052	D_USER_ID	BUDGET_CHANGE	Changed_By_User_ID	FK	USER who made the change; null only for an approved automated import.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique	Nullable only when Change_Source = UNIVERSITY_IMPORT	Foreign key. References USER.User_ID.
DOM-053	D_USD_NONNEGATIVE	BUDGET_CHANGE	Previous_Amount	Non-key	Budget amount immediately before the recorded change.	DECIMAL(12,2)	$0.00 to $9,999,999,999.99; exactly 2 decimal places	Nonunique	Non-null	Must equal the budget value immediately before the change.
DOM-054	D_USD_NONNEGATIVE	BUDGET_CHANGE	New_Amount	Non-key	Budget amount immediately after the recorded change.	DECIMAL(12,2)	$0.00 to $9,999,999,999.99; exactly 2 decimal places	Nonunique	Non-null	Must equal the resulting HOUSING_BUDGET.Current_Amount.
DOM-055	D_BUDGET_CHANGE_SOURCE	BUDGET_CHANGE	Change_Source	Non-key	Controlled source of the budget change.	VARCHAR(20)	One of: MANUAL_ADMIN, UNIVERSITY_IMPORT	Nonunique	Non-null	
DOM-056	D_TIMESTAMP_UTC	BUDGET_CHANGE	Change_Date	Non-key	UTC date/time when the budget change was applied.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Non-null	
DOM-057	D_REASON_TEXT	BUDGET_CHANGE	Reason	Non-key	Required business reason for changing the athlete's	VARCHAR(500)	1-500 characters; trimmed	Nonunique	Non-null	
DOM-058	D_WEBSITE_SOURCE_ID	APPROVED_WEBSITE_SOURCE	Website_Source_ID	PK	Identifier of the related approved website source.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key.
DOM-059	D_USER_ID	APPROVED_WEBSITE_SOURCE	Approved_By_Admin_ID	FK	SYSTEM_ADMIN user who approved the website	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique	Non-null	Foreign key. References SYSTEM_ADMIN.User_ID.
DOM-060	D_SOURCE_NAME	APPROVED_WEBSITE_SOURCE	Source_Name	Non-key	Unique display name of the approved website source.	VARCHAR(150)	1-150 characters; normalized for duplicate comparison	Must be unique among active sources	Non-null	
DOM-061	D_HTTPS_URL	APPROVED_WEBSITE_SOURCE	Source_URL	Non-key	Canonical URL used to identify or access the approved source.	VARCHAR(2048)	Valid absolute HTTPS URL; normalized canonical form	Must be unique after URL normalization	Non-null	
DOM-062	D_BOOLEAN	APPROVED_WEBSITE_SOURCE	Active_Flag	Non-key	Boolean indicator that the record is currently active.	BOOLEAN	TRUE or FALSE	Nonunique	Non-null	
DOM-063	D_CALENDAR_DATE	APPROVED_WEBSITE_SOURCE	Approved_Date	Non-key	Date on which the source was approved.	DATE	ISO date; YYYY-MM-DD	Nonunique	Non-null	
DOM-064	D_TIMESTAMP_UTC	APPROVED_WEBSITE_SOURCE	Last_Validated_At	Non-key	UTC date/time of the most recent successful source validation.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Nullable until the first successful validation	
DOM-065	D_COLLECTION_RUN_ID	COLLECTION_RUN	Collection_Run_ID	PK	Unique identifier for a listing collection run.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key.
DOM-066	D_WEBSITE_SOURCE_ID	COLLECTION_RUN	Website_Source_ID	FK	Identifier of the related approved website source.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique	Non-null	Foreign key. References APPROVED_WEBSITE_SOURCE.Website_Source_ID.
DOM-067	D_TIMESTAMP_UTC	COLLECTION_RUN	Started_At	Non-key	UTC date/time when listing collection began.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Non-null	
DOM-068	D_TIMESTAMP_UTC	COLLECTION_RUN	Completed_At	Non-key	UTC date/time when listing collection ended.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Nullable while Run_Status = RUNNING	
DOM-069	D_COLLECTION_RUN_STATUS	COLLECTION_RUN	Run_Status	Non-key	Lifecycle/result state of a collection run.	VARCHAR(12)	One of: RUNNING, COMPLETED, PARTIAL, FAILED	Nonunique	Non-null	
DOM-070	D_FAILURE_DETAILS	COLLECTION_RUN	Failure_Details	Non-key	Collection error details when a run is partial or failed.	VARCHAR(2000)	0-2,000 characters; required when the related status is FAILED	Nonunique	Nullable unless Run_Status = PARTIAL or FAILED	
DOM-071	D_LISTING_ID	RESIDENCE_LISTING	Listing_ID	PK	Identifier of the related RESIDENCE_LISTING.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key.
DOM-072	D_WEBSITE_SOURCE_ID	RESIDENCE_LISTING	Website_Source_ID	FK	Identifier of the related approved website source.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique	Non-null	Foreign key. References APPROVED_WEBSITE_SOURCE.Website_Source_ID.
DOM-073	D_EXTERNAL_LISTING_ID	RESIDENCE_LISTING	External_Listing_ID	Non-key	Identifier assigned to the listing by the source website.	VARCHAR(150)	1-150 source-assigned characters; trimmed	Unique within Website_Source_ID	Non-null	
DOM-074	D_ADDRESS_LINE	RESIDENCE_LISTING	Address_Line	Non-key	Street-address line for the residence.	VARCHAR(200)	1-200 characters	Nonunique	Non-null	
DOM-075	D_CITY	RESIDENCE_LISTING	City	Non-key	City in which the residence is located.	VARCHAR(100)	1-100 characters	Nonunique	Non-null	
DOM-076	D_US_STATE_CODE	RESIDENCE_LISTING	State	Non-key	Two-letter U.S. state or territory code.	CHAR(2)	Two uppercase letters from the USPS state/territory code list	Nonunique	Non-null	
DOM-077	D_US_POSTAL_CODE	RESIDENCE_LISTING	Postal_Code	Non-key	U.S. ZIP Code for the residence.	VARCHAR(10)	##### or #####-####	Nonunique	Non-null	
DOM-078	D_USD_NONNEGATIVE	RESIDENCE_LISTING	Monthly_Rent	Non-key	Monthly rental charge shown for the listing.	DECIMAL(12,2)	$0.00 to $9,999,999,999.99; exactly 2 decimal places	Nonunique	Non-null	
DOM-079	D_BEDROOM_COUNT	RESIDENCE_LISTING	Bedrooms	Non-key	Number of bedrooms in the residence.	SMALLINT	Whole number 0-20	Nonunique	Non-null	
DOM-080	D_BATHROOM_COUNT	RESIDENCE_LISTING	Bathrooms	Non-key	Number of bathrooms in the residence.	DECIMAL(3,1)	0.0-20.0 in 0.5 increments	Nonunique	Non-null	
DOM-081	D_CALENDAR_DATE	RESIDENCE_LISTING	Available_Date	Non-key	Date the residence becomes available.	DATE	ISO date; YYYY-MM-DD	Nonunique	Non-null	

Athlete Housing ERD - Attribute Domain Catalog

Every attribute shown in the logical ERD is listed once. A named domain may be reused by multiple attributes; nullability and uniqueness are specified at the attribute assignment level.

Domain ID	Named Domain	Entity	Attribute Name	Key Role	Meaning	Data Type	Format / Allowed Values / Range	Uniqueness	Null Support	Enforcement Notes
DOM-081	D_CALENDAR_DATE	RESIDENCE_LISTING	Available_Date	Non-key	Date the residence becomes available.	DATE	ISO date; YYYY-MM-DD	Nonunique	Non-null	
DOM-082	D_BOOLEAN	RESIDENCE_LISTING	Active_Flag	Non-key	Boolean indicator that the record is currently active.	BOOLEAN	TRUE or FALSE	Nonunique	Non-null	
DOM-083	D_BOOLEAN	RESIDENCE_LISTING	Duplicate_Flag	Non-key	Boolean indicator that the listing was classified as a duplicate.	BOOLEAN	TRUE or FALSE	Nonunique	Non-null	Set by the duplicate-detection rule.
DOM-084	D_TIMESTAMP_UTC	RESIDENCE_LISTING	Last_Collected_At	Non-key	UTC date/time when the listing was most recently collected or refreshed.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Non-null	
DOM-085	D_REQUEST_ID	RESIDENCE_REQUEST	Request_ID	PK	Identifier of the related RESIDENCE_REQUEST.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key.
DOM-086	D_USER_ID	RESIDENCE_REQUEST	Athlete_ID	FK	USER identifier of the related ATHLETE.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique	Non-null	Foreign key. References ATHLETE.User_ID.
DOM-087	D_LISTING_ID	RESIDENCE_REQUEST	Listing_ID	FK	Identifier of the related RESIDENCE_LISTING.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique	Non-null	Foreign key. References RESIDENCE_LISTING.Listing_ID.
DOM-088	D_BUDGET_ID	RESIDENCE_REQUEST	Budget_ID	FK	Identifier of the related HOUSING_BUDGET.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique	Non-null	Foreign key. References HOUSING_BUDGET.Budget_ID.
DOM-089	D_TIMESTAMP_UTC	RESIDENCE_REQUEST	Submitted_At	Non-key	UTC date/time when the athlete submitted the residence request.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Non-null	
DOM-090	D_ATHLETE_NOTES	RESIDENCE_REQUEST	Athlete_Notes	Non-key	Optional notes supplied by the athlete with the request.	VARCHAR(2000)	0-2,000 characters	Nonunique	Nullable	
DOM-091	D_BOOLEAN	RESIDENCE_REQUEST	Out_Of_Pocket_Confirmed	Non-key	Whether the athlete confirmed a positive estimated out-of-pocket amount.	BOOLEAN	TRUE or FALSE	Nonunique	Non-null	
DOM-092	D_USD_NONNEGATIVE	RESIDENCE_REQUEST	Budget_Amount_At_Submission	Non-key	Snapshot of the athlete's budget when the request was submitted.	DECIMAL(12,2)	$0.00 to $9,999,999,999.99; exactly 2 decimal places	Nonunique	Non-null	System-generated snapshot; not athlete-entered.
DOM-093	D_USD_NONNEGATIVE	RESIDENCE_REQUEST	Estimated_Out_Of_Pocket	Non-key	Calculated amount by which monthly rent exceeds the submission-time budget, floored at zero.	DECIMAL(12,2)	$0.00 to $9,999,999,999.99; exactly 2 decimal places	Nonunique	Non-null	System-calculated as max(Monthly_Rent - Budget_Amount_At_Submission, 0).
DOM-094	D_REQUEST_STATUS	RESIDENCE_REQUEST	Current_Status	Non-key	Current decision/workflow state of the request.	VARCHAR(20)	One of: PENDING_REVIEW, APPROVED, REJECTED	Nonunique	Non-null	System initializes to PENDING_REVIEW; only the review workflow may change it.
DOM-095	D_REQUEST_ID	REQUEST_REVIEW	Request_ID	PK/FK	Identifier of the related RESIDENCE_REQUEST.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key and foreign key. References RESIDENCE_REQUEST.Request_ID; enforces at most one review per request.
DOM-096	D_USER_ID	REQUEST_REVIEW	Reviewer_ID	FK	USER identifier of the ATHLETIC_DEPARTMENT_REVIEWER who decided the	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique	Non-null	Foreign key. References ATHLETIC_DEPARTMENT_REVIEWER.User_ID.
DOM-097	D_REVIEW_DECISION	REQUEST_REVIEW	Decision	Non-key	Reviewer decision for the residence request.	VARCHAR(10)	One of: APPROVED, REJECTED	Nonunique	Non-null	
DOM-098	D_REVIEWER_COMMENTS	REQUEST_REVIEW	Reviewer_Comments	Non-key	Reviewer explanation; required for a rejection.	VARCHAR(2000)	0-2,000 characters; required when Decision = REJECTED	Nonunique	Nullable for approval; non-null for	
DOM-099	D_TIMESTAMP_UTC	REQUEST_REVIEW	Decision_Date	Non-key	UTC date/time when the decision was recorded.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Non-null	
DOM-100	D_STATUS_HISTORY_ID	REQUEST_STATUS_HISTORY	Status_History_ID	PK	Unique identifier for a request-status history row.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key.
DOM-101	D_REQUEST_ID	REQUEST_STATUS_HISTORY	Request_ID	FK	Identifier of the related RESIDENCE_REQUEST.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique	Non-null	Foreign key. References RESIDENCE_REQUEST.Request_ID.
DOM-102	D_REQUEST_STATUS	REQUEST_STATUS_HISTORY	Status	Non-key	Request status represented by the history row.	VARCHAR(20)	One of: PENDING_REVIEW, APPROVED, REJECTED	Nonunique	Non-null	Must match the status created by the submission or review transaction.
DOM-103	D_TIMESTAMP_UTC	REQUEST_STATUS_HISTORY	Changed_At	Non-key	UTC date/time when the status became effective.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Non-null	
DOM-104	D_REASON_TEXT	REQUEST_STATUS_HISTORY	Reason	Non-key	Explanation for the request status entry.	VARCHAR(500)	1-500 characters; trimmed	Nonunique	Non-null	
DOM-105	D_NOTIFICATION_ID	EMAIL_NOTIFICATION	Notification_ID	PK	Unique identifier for an email-notification record.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Must be unique	Non-null	Primary key.
DOM-106	D_REQUEST_ID	EMAIL_NOTIFICATION	Request_ID	FK	Identifier of the related RESIDENCE_REQUEST.	BIGINT	Positive integer; 1 to 9,223,372,036,854,775,807	Nonunique	Non-null	Foreign key. References RESIDENCE_REQUEST.Request_ID.
DOM-107	D_MESSAGE_TYPE	EMAIL_NOTIFICATION	Message_Type	Non-key	Controlled message category generated from the review decision.	VARCHAR(10)	One of: APPROVAL, REJECTION	Nonunique	Non-null	Must match the request decision.
DOM-108	D_TIMESTAMP_UTC	EMAIL_NOTIFICATION	Sent_At	Non-key	UTC date/time when the email was successfully sent.	TIMESTAMP WITH TIME ZONE	ISO 8601 UTC timestamp; YYYY-MM-DDThh:mm:ssZ	Nonunique	Nullable until Delivery_Status = SENT	
DOM-109	D_DELIVERY_STATUS	EMAIL_NOTIFICATION	Delivery_Status	Non-key	Current delivery state of the email notification.	VARCHAR(10)	One of: PENDING, SENT, FAILED	Nonunique	Non-null	
DOM-110	D_FAILURE_DETAILS	EMAIL_NOTIFICATION	Failure_Details	Non-key	Provider or delivery error details when the message	VARCHAR(2000)	0-2,000 characters; required when the related status is FAILED	Nonunique	Nullable unless Delivery_Status =	

5

Data TablesData Tables & Activity Diagrams

Primary Keys, Foreign Keys, and Sample Data

Athlete Housing Relational Database Workbook

Sample relational data derived from the Athlete Housing Search and Residence Approval System DFD and the associated ERD design.

Item	Explanation	Grading / Modeling Note	Source
Key notation	(PK) = Primary Key; (FK) = Foreign Key; (PK/FK) = subtype key inherited from Users.	PK columns use navy headers, FK columns orange, and inherited PK/FK columns purple.	dfd_athlete_housing.docx
Specialization	Users is the supertype. Athletes, Reviewers, and System_Admins are total and disjoint subtypes.	Every sample user appears in exactly one subtype table.	ERD design assumption
Sample data	All identifiers, people, addresses, and transactions are synthetic classroom examples.	Every entity table contains at least five sample rows.	Generated for this workbook
Referential integrity	All foreign keys were programmatically checked against their parent primary keys.	See Validation_Checks for PASS results.	Workbook validation
Formula field	Residence_Requests.estimated_out_of_pocket = MAX(0, submitted_rent - budget_at_submission).	The formula is stored in each sample request row.	Residence request business rule
Workbook navigation	Use Schema_Overview for entities, Relationships for cardinalities, Domains for allowed values, and Triggers for integrity logic.	Each entity is stored as a separate Excel table with filters.	Workbook structure

Schema Overview

Entity inventory, identifiers, foreign keys, and formula-driven sample row counts.

Entity	Purpose	Primary Key	Foreign Keys	Sample Row Count	Table Name
Users	Supertype for every authenticated system user. Total/disjoint specialization into Athlete, Reviewer, or System Admin.	user_id	None	15	tblUsers
Athletes	Athlete subtype containing athlete-specific profile attributes.	athlete_id	athlete_id → Users.user_id	5	tblAthletes
Reviewers	Athletic Department Reviewer subtype authorized to evaluate residence requests.	reviewer_id	reviewer_id → Users.user_id	5	tblReviewers
System_Admins	System Administrator subtype with privileges for users, budgets, websites, roles, and audit functions.	admin_id	admin_id → Users.user_id	5	tblSystemAdmins
Roles	Named access-control roles.	role_id	None	5	tblRoles
Permissions	Atomic permissions assigned to roles.	permission_id	None	8	tblPermissions
User_Roles	Associative entity resolving the many-to-many relationship between Users and Roles.	user_role_id	user_id → Users.user_id; role_id → Roles.role_id	18	tblUserRoles
Role_Permissions	Associative entity resolving the many-to-many relationship between Roles and Permissions.	role_permission_id	role_id → Roles.role_id; permission_id → Permissions.permission_id	13	tblRolePermissions
Sessions	Authentication sessions and session termination events.	session_id	user_id → Users.user_id	8	tblSessions
Housing_Budgets	Current monthly housing budget assigned to each athlete.	budget_id	athlete_id → Athletes.athlete_id	5	tblHousingBudgets
Budget_Changes	History of athlete budget changes with old/new amounts and administrative reason.	budget_change_id	budget_id → Housing_Budgets.budget_id; changed_by_admin_id → System_Admins.admin_id	7	tblBudgetChanges
Approved_Websites	Rental websites authorized as listing sources.	website_id	added_by_admin_id → System_Admins.admin_id	5	tblApprovedWebsites
Residence_Listings	Cleaned residence listings collected from approved websites.	listing_id	website_id → Approved_Websites.website_id	10	tblResidenceListings
Residence_Requests	Residence selections submitted by athletes for review. Estimated out-of-pocket is formula-driven.	request_id	athlete_id → Athletes.athlete_id; listing_id → Residence_Listings.listing_id; budget_id → Housing_Budgets.budget_id	5	tblResidenceRequests
Request_Reviews	Reviewer decisions and comments for residence requests.	review_id	request_id → Residence_Requests.request_id; reviewer_id → Reviewers.reviewer_id	5	tblRequestReviews
Status_History	Chronological request status transitions.	status_history_id	request_id → Residence_Requests.request_id; changed_by_user_id → Users.user_id	10	tblStatusHistory
Audit_Log	Security, administrative, collection, and workflow audit events.	audit_id	user_id → Users.user_id	10	tblAuditLog
Notifications	Decision notifications sent to athletes and tracked for delivery outcome.	notification_id	request_id → Residence_Requests.request_id; athlete_id → Athletes.athlete_id	5	tblNotifications

Schema Overview

Entity inventory, identifiers, foreign keys, and formula-driven sample row counts.

Entity	Purpose	Primary Key	Foreign Keys	Sample Row Count	Table Name
Users	Supertype for every authenticated system user. Total/disjoint specialization into Athlete, Reviewer, or System Admin.	user_id	None	15	tblUsers
Athletes	Athlete subtype containing athlete-specific profile attributes.	athlete_id	athlete_id → Users.user_id	5	tblAthletes
Reviewers	Athletic Department Reviewer subtype authorized to evaluate residence requests.	reviewer_id	reviewer_id → Users.user_id	5	tblReviewers
System_Admins	System Administrator subtype with privileges for users, budgets, websites, roles, and audit functions.	admin_id	admin_id → Users.user_id	5	tblSystemAdmins
Roles	Named access-control roles.	role_id	None	5	tblRoles
Permissions	Atomic permissions assigned to roles.	permission_id	None	8	tblPermissions
User_Roles	Associative entity resolving the many-to-many relationship between Users and Roles.	user_role_id	user_id → Users.user_id; role_id → Roles.role_id	18	tblUserRoles
Role_Permissions	Associative entity resolving the many-to-many relationship between Roles and Permissions.	role_permission_id	role_id → Roles.role_id; permission_id → Permissions.permission_id	13	tblRolePermissions
Sessions	Authentication sessions and session termination events.	session_id	user_id → Users.user_id	8	tblSessions
Housing_Budgets	Current monthly housing budget assigned to each athlete.	budget_id	athlete_id → Athletes.athlete_id	5	tblHousingBudgets
Budget_Changes	History of athlete budget changes with old/new amounts and administrative reason.	budget_change_id	budget_id → Housing_Budgets.budget_id; changed_by_admin_id → System_Admins.admin_id	7	tblBudgetChanges
Approved_Websites	Rental websites authorized as listing sources.	website_id	added_by_admin_id → System_Admins.admin_id	5	tblApprovedWebsites
Residence_Listings	Cleaned residence listings collected from approved websites.	listing_id	website_id → Approved_Websites.website_id	10	tblResidenceListings
Residence_Requests	Residence selections submitted by athletes for review. Estimated out-of-pocket is formula-driven.	request_id	athlete_id → Athletes.athlete_id; listing_id → Residence_Listings.listing_id; budget_id → Housing_Budgets.budget_id	5	tblResidenceRequests
Request_Reviews	Reviewer decisions and comments for residence requests.	review_id	request_id → Residence_Requests.request_id; reviewer_id → Reviewers.reviewer_id	5	tblRequestReviews
Status_History	Chronological request status transitions.	status_history_id	request_id → Residence_Requests.request_id; changed_by_user_id → Users.user_id	10	tblStatusHistory
Audit_Log	Security, administrative, collection, and workflow audit events.	audit_id	user_id → Users.user_id	10	tblAuditLog
Notifications	Decision notifications sent to athletes and tracked for delivery outcome.	notification_id	request_id → Residence_Requests.request_id; athlete_id → Athletes.athlete_id	5	tblNotifications

PK–FK Relationships

Logical relationships and crow's-foot cardinality equivalents used by the workbook.

Parent Table	Parent PK	Child Table	Child FK	Cardinality	Business Rule
Users	user_id	Athletes	athlete_id	1 to 0..1	Total/disjoint subtype; subtype key is also FK.
Users	user_id	Reviewers	reviewer_id	1 to 0..1	Total/disjoint subtype; subtype key is also FK.
Users	user_id	System_Admins	admin_id	1 to 0..1	Total/disjoint subtype; subtype key is also FK.
Users	user_id	User_Roles	user_id	1 to 0..*	A user may hold one or more roles.
Roles	role_id	User_Roles	role_id	1 to 0..*	A role may be assigned to many users.
Roles	role_id	Role_Permissions	role_id	1 to 0..*	A role contains multiple permissions.
Permissions	permission_id	Role_Permissions	permission_id	1 to 0..*	A permission can belong to multiple roles.
Users	user_id	Sessions	user_id	1 to 0..*	A user may create many sessions.
Athletes	athlete_id	Housing_Budgets	athlete_id	1 to 1..*	Budget records may change over time; only one is current.
Housing_Budgets	budget_id	Budget_Changes	budget_id	1 to 0..*	Each change belongs to one budget record.
System_Admins	admin_id	Budget_Changes	changed_by_admin_id	1 to 0..*	An administrator records budget changes.
System_Admins	admin_id	Approved_Websites	added_by_admin_id	1 to 0..*	An administrator approves website sources.
Approved_Websites	website_id	Residence_Listings	website_id	1 to 0..*	A source can provide many listings.
Athletes	athlete_id	Residence_Requests	athlete_id	1 to 0..*	An athlete may submit multiple requests.
Residence_Listings	listing_id	Residence_Requests	listing_id	1 to 0..*	A listing may be selected in requests over time.
Housing_Budgets	budget_id	Residence_Requests	budget_id	1 to 0..*	The budget used at submission is retained.
Residence_Requests	request_id	Request_Reviews	request_id	1 to 0..*	A request may be reviewed or re-reviewed.
Reviewers	reviewer_id	Request_Reviews	reviewer_id	1 to 0..*	A reviewer may process many requests.
Residence_Requests	request_id	Status_History	request_id	1 to 1..*	Every request receives status history.
Users	user_id	Status_History	changed_by_user_id	1 to 0..*	Status changes identify the responsible user.
Users	user_id	Audit_Log	user_id	1 to 0..*	User actions are written to the audit log.
Residence_Requests	request_id	Notifications	request_id	1 to 0..*	A request may generate multiple notifications.
Athletes	athlete_id	Notifications	athlete_id	1 to 0..*	Notifications are addressed to the athlete.

Users

Supertype for every authenticated system user. Total/disjoint specialization into Athlete, Reviewer, or System Admin. PK: user_id. FK: No foreign keys.

user_id (PK)	university_uid	university_email	first_name	last_name	account_status	created_at
U001	A10001	jordan.lee@university.edu	Jordan	Lee	Active	2026-01-08 09:15
U002	A10002	maya.thompson@university.edu	Maya	Thompson	Active	2026-01-08 09:18
U003	A10003	ethan.brooks@university.edu	Ethan	Brooks	Active	2026-01-08 09:21
U004	A10004	sofia.ramirez@university.edu	Sofia	Ramirez	Active	2026-01-08 09:24
U005	A10005	noah.williams@university.edu	Noah	Williams	Active	2026-01-08 09:27
U101	R20001	olivia.chen@university.edu	Olivia	Chen	Active	2026-01-05 10:00
U102	R20002	marcus.green@university.edu	Marcus	Green	Active	2026-01-05 10:05
U103	R20003	priya.patel@university.edu	Priya	Patel	Active	2026-01-05 10:10
U104	R20004	daniel.kim@university.edu	Daniel	Kim	Active	2026-01-05 10:15
U105	R20005	aisha.jackson@university.edu	Aisha	Jackson	Active	2026-01-05 10:20
U201	S30001	liam.carter@university.edu	Liam	Carter	Active	2026-01-02 08:00
U202	S30002	emma.davis@university.edu	Emma	Davis	Active	2026-01-02 08:05
U203	S30003	lucas.martin@university.edu	Lucas	Martin	Active	2026-01-02 08:10
U204	S30004	ava.robinson@university.edu	Ava	Robinson	Active	2026-01-02 08:15
U205	S30005	mason.clark@university.edu	Mason	Clark	Active	2026-01-02 08:20

Athletes

Athlete subtype containing athlete-specific profile attributes. PK: athlete_id. FK: athlete_id → Users.user_id.

athlete_id (PK/FK)	sport	class_year	housing_status	profile_last_updated
U001	Basketball	Junior	Searching	2026-07-22 14:10
U002	Soccer	Senior	Request Submitted	2026-07-25 09:30
U003	Football	Sophomore	**Approved**	2026-07-26 16:45
U004	Volleyball	Junior	Searching	2026-07-24 11:20
U005	Track and Field	Senior	**Rejected**	2026-07-27 13:05

Reviewers

Athletic Department Reviewer subtype authorized to evaluate residence requests. PK: reviewer_id. FK: reviewer_id → Users.user_id.

reviewer_id (PK/FK)	department	approval_limit	reviewer_status	assigned_region
U101	Athletic Compliance	$2,200.00	**Active**	Central Denver
U102	Student Housing	$2,000.00	**Active**	Aurora
U103	Athlete Services	$2,100.00	**Active**	Lakewood
U104	Financial Compliance	$1,900.00	**Active**	Englewood
U105	Student Affairs	$2,300.00	**Active**	Metro Area

System Admins

System Administrator subtype with privileges for users, budgets, websites, roles, and audit functions. PK: admin_id. FK: admin_id → Users.user_id.

admin_id (PK/FK)	admin_level	support_area	admin_status	last_privilege_review
U201	Full	Identity and Access	Active	2026-07-01
U202	Security	Security Operations	Active	2026-07-02
U203	Operations	Listing Collection	Active	2026-07-03
U204	Data	Budget Administration	Active	2026-07-04
U205	Audit	Compliance Reporting	Active	2026-07-05

Roles

Named access-control roles. PK: role_id. FK: No foreign keys.

role_id (PK)	role_name	role_description	is_active	created_at
R01	Athlete	Search listings, view budget, and submit residence requests	☑	2026-01-01 08:00
R02	Reviewer	Review pending residence requests and record decisions	☑	2026-01-01 08:05
R03	System Admin	Manage users, budgets, websites, security, and audits	☑	2026-01-01 08:10
R04	Audit Viewer	Read security and workflow audit history	☑	2026-01-01 08:15
R05	Listing Manager	Manage approved sources and listing collection	☑	2026-01-01 08:20

Permissions

Atomic permissions assigned to roles. PK: permission_id. FK: No foreign keys.

permission_id (PK)	permission_code	permission_description	resource	action
P01	VIEW_LISTINGS	View and search approved residence listings	Residence Listing	SELECT
P02	SUBMIT_REQUEST	Create a residence approval request	Residence Request	INSERT
P03	REVIEW_REQUEST	Approve or reject a pending request	Residence Request	UPDATE
P04	MANAGE_USERS	Correct profiles and manage user access	User	UPDATE
P05	MANAGE_WEBSITES	Add, edit, disable, or remove approved websites	Approved Website	ALL
P06	MANAGE_BUDGETS	Import or manually update athlete budgets	Housing Budget	UPDATE
P07	VIEW_AUDIT	View audit and status history	Audit Log	SELECT
P08	MANAGE_ROLES	Assign roles and permissions	Role	ALL

User Roles

Associative entity resolving the many-to-many relationship between Users and Roles. PK: user_role_id. FK: user_id → Users.user_id; role_id → Roles.role_id.

user_role_id (PK)	user_id (FK)	role_id (FK)	assigned_date	is_active
UR001	U001	R01	2026-01-08	☑
UR002	U002	R01	2026-01-08	☑
UR003	U003	R01	2026-01-08	☑
UR004	U004	R01	2026-01-08	☑
UR005	U005	R01	2026-01-08	☑
UR006	U101	R02	2026-01-05	☑
UR007	U102	R02	2026-01-05	☑
UR008	U103	R02	2026-01-05	☑
UR009	U104	R02	2026-01-05	☑
UR010	U105	R02	2026-01-05	☑
UR011	U201	R03	2026-01-02	☑
UR012	U202	R03	2026-01-02	☑
UR013	U203	R03	2026-01-02	☑
UR014	U204	R03	2026-01-02	☑
UR015	U205	R03	2026-01-02	☑
UR016	U101	R04	2026-02-01	☑
UR017	U203	R05	2026-02-01	☑
UR018	U205	R04	2026-02-01	☑

Role Permissions

Associative entity resolving the many-to-many relationship between Roles and Permissions. PK: role_permission_id. FK: role_id → Roles.role_id; permission_id →

role_permission_id (PK)	role_id (FK)	permission_id (FK)	granted_date
RP001	R01	P01	2026-01-01
RP002	R01	P02	2026-01-01
RP003	R02	P01	2026-01-01
RP004	R02	P03	2026-01-01
RP005	R02	P07	2026-01-01
RP006	R03	P04	2026-01-01
RP007	R03	P05	2026-01-01
RP008	R03	P06	2026-01-01
RP009	R03	P07	2026-01-01
RP010	R03	P08	2026-01-01
RP011	R04	P07	2026-01-01
RP012	R05	P01	2026-01-01
RP013	R05	P05	2026-01-01

Sessions

Authentication sessions and session termination events. PK: session_id. FK: user_id → Users.user_id.

session_id (PK)	user_id (FK)	token_hash	started_at	expires_at	ended_at	session_status	end_reason
S001	U001	hash_8f1a0c	2026-07-29 08:00	2026-07-29 16:00	2026-07-29 09:10	Ended	Logout
S002	U002	hash_9b2d1e	2026-07-29 08:15	2026-07-29 16:15		Active	
S003	U101	hash_a31f22	2026-07-29 09:00	2026-07-29 17:00	2026-07-29 12:30	Ended	Timeout
S004	U201	hash_b4423f	2026-07-29 07:45	2026-07-29 15:45		Active	
S005	U005	hash_c55340	2026-07-28 20:00	2026-07-29 04:00	2026-07-28 20:02	Failed	Invalid UID
S006	U003	hash_d66451	2026-07-29 10:30	2026-07-29 18:30		Active	
S007	U103	hash_e77562	2026-07-29 11:00	2026-07-29 19:00	2026-07-29 13:05	Ended	Logout
S008	U203	hash_f88673	2026-07-29 06:30	2026-07-29 14:30	2026-07-29 14:30	Ended	Expired

Housing Budgets

Current monthly housing budget assigned to each athlete. PK: budget_id. FK: athlete_id → Athletes.athlete_id.

budget_id (PK)	athlete_id (FK)	monthly_budget	effective_date	budget_source	is_current
B001	U001	$1,750.00	2026-07-01	University Import	☑
B002	U002	$1,650.00	2026-07-01	University Import	☑
B003	U003	$1,900.00	2026-07-01	Manual Adjustment	☑
B004	U004	$1,600.00	2026-07-01	University Import	☑
B005	U005	$1,800.00	2026-07-01	Manual Adjustment	☑

Budget Changes

History of athlete budget changes with old/new amounts and administrative reason. PK: budget_change_id. FK: budget_id → Housing_Budgets.budget_id; changed_by_admin_id → System_Admins.admin_id.

budget_change_id (PK)	budget_id (FK)	changed_by_admin_id (FK)	previous_amount	new_amount	change_reason	change_date
BC001	B001	U204	$1,700.00	$1,750.00	Annual university budget update	2026-07-01 08:00
BC002	B002	U204	$1,600.00	$1,650.00	Annual university budget update	2026-07-01 08:05
BC003	B003	U204	$1,800.00	$1,900.00	Documented accessibility need	2026-07-02 10:30
BC004	B004	U204	$1,550.00	$1,600.00	Annual university budget update	2026-07-01 08:10
BC005	B005	U204	$1,700.00	$1,800.00	Approved cost-of-living adjustment	2026-07-03 14:00
BC006	B001	U205	$1,650.00	$1,700.00	Prior-year correction	2026-01-15 11:00
BC007	B003	U204	$1,750.00	$1,800.00	Midyear review	2026-03-10 09:20

Approved Websites

Rental websites authorized as listing sources. PK: website_id. FK: added_by_admin_id → System_Admins.admin_id.

website_id (PK)	website_name	base_url	source_status	added_by_admin_id (FK)	added_date	last_verified_date
W001	Zillow	https://www.zillow.com	Active	U203	2026-01-10	2026-07-25
W002	Apartments.com	https://www.apartments.com	Active	U203	2026-01-10	2026-07-25
W003	Rent.com	https://www.rent.com	Active	U203	2026-01-11	2026-07-24
W004	Realtor.com	https://www.realtor.com	Active	U203	2026-01-11	2026-07-24
W005	HotPads	https://hotpads.com	Active	U203	2026-01-12	2026-07-23

Residence Listings

Cleaned residence listings collected from approved websites. PK: listing_id. FK: website_id → Approved_Websites.website_id.

listing_id (PK)	website_id (FK)	external_listing_id	street_address	city	state	zip_code	bedrooms	bathrooms	monthly_rent	available_date	listing_status	last_collected_at
L001	W001	Z-41001	1450 Blake St Apt 208	Denver	CO	80202	1	1	$1,680.00	2026-08-01	Available	2026-07-29 05:10
L002	W001	Z-41002	780 S Pearl St Unit 4	Denver	CO	80209	1	1	$1,825.00	2026-08-15	Available	2026-07-29 05:10
L003	W002	A-52001	990 N Logan St Apt 315	Denver	CO	80203	2	1	$1,950.00	2026-08-05	Available	2026-07-29 05:20
L004	W002	A-52002	2121 E Colfax Ave Apt 7	Denver	CO	80206	1	1	$1,575.00	2026-08-01	Available	2026-07-29 05:20
L005	W003	R-63001	1550 Sable Blvd Unit 12	Aurora	CO	80011	2	2	$1,740.00	2026-08-10	Available	2026-07-29 05:30
L006	W003	R-63002	3200 W 20th Ave Apt 9	Denver	CO	80211	1	1	$1,495.00	2026-08-01	Available	2026-07-29 05:30
L007	W004	RE-74001	440 Benton St Unit B	Lakewood	CO	80226	2	1	$1,850.00	2026-08-20	Available	2026-07-29 05:40
L008	W004	RE-74002	88 W Floyd Ave Apt 2	Englewood	CO	80110	1	1	$1,625.00	2026-08-03	Available	2026-07-29 05:40
L009	W005	H-85001	1735 N Gaylord St Apt 6	Denver	CO	80206	1	1	$1,775.00	2026-08-12	Available	2026-07-29 05:50
L010	W005	H-85002	6100 E 8th Ave Unit 114	Denver	CO	80220	2	1.5	$2,050.00	2026-08-15	Inactive	2026-07-29 05:50

Residence Requests

Residence selections submitted by athletes for review. Estimated out-of-pocket is formula-driven. PK: request_id. FK: athlete_id → Athletes.athlete_id; listing_id → Residence_Listings.listing_id; budget_id → Housing_Budgets.budget_id.

request_id (PK)	athlete_id (FK)	listing_id (FK)	budget_id (FK)	submitted_at	athlete_note	out_of_pocket_confirmed	submitted_rent	budget_at_submission	estimated_out_of_pocket	request_status
Q001	U001	L001	B001	2026-07-24 10:15	Close to campus and within budget.	☐	$1,680.00	$1,750.00	$0.00	Pending Review
Q002	U002	L002	B002	2026-07-25 09:30	I can cover the monthly difference.	☑	$1,825.00	$1,650.00	$175.00	Pending Review
Q003	U003	L003	B003	2026-07-23 16:00	Room for a training partner.	☑	$1,950.00	$1,900.00	$50.00	Approved
Q004	U004	L004	B004	2026-07-26 12:40	Preferred location near practice facilities.	☐	$1,575.00	$1,600.00	$0.00	Approved
Q005	U005	L009	B005	2026-07-27 13:05	Requested because of transit access.	☐	$1,775.00	$1,800.00	$0.00	Rejected

Request Reviews

Reviewer decisions and comments for residence requests. PK: review_id. FK: request_id → Residence_Requests.request_id; reviewer_id → Reviewers.reviewer_id.

review_id (PK)	request_id (FK)	reviewer_id (FK)	reviewed_at	decision	reviewer_comments
RV001	Q001	U101	2026-07-29 09:00	Pending	Awaiting verification of lease terms.
RV002	Q002	U102	2026-07-29 10:20	Pending	Confirming out-of-pocket acknowledgement.
RV003	Q003	U103	2026-07-26 16:45	Approved	Budget difference is documented and acceptable.
RV004	Q004	U104	2026-07-27 11:15	Approved	Rent is within the assigned monthly budget.
RV005	Q005	U105	2026-07-28 14:30	Rejected	Listing source terms could not be verified.

Status History

Chronological request status transitions. PK: status_history_id. FK: request_id → Residence_Requests.request_id; changed_by_user_id → Users.user_id.

status_history_id (PK)	request_id (FK)	old_status	new_status	changed_by_user_id (FK)	changed_at	status_note
SH001	Q001		Pending Review	U001	2026-07-24 10:15	Request submitted.
SH002	Q001	Pending Review	Pending Review	U101	2026-07-29 09:00	Review opened.
SH003	Q002		Pending Review	U002	2026-07-25 09:30	Request submitted.
SH004	Q002	Pending Review	Pending Review	U102	2026-07-29 10:20	Additional confirmation requested.
SH005	Q003		Pending Review	U003	2026-07-23 16:00	Request submitted.
SH006	Q003	Pending Review	Approved	U103	2026-07-26 16:45	Reviewer approved request.
SH007	Q004		Pending Review	U004	2026-07-26 12:40	Request submitted.
SH008	Q004	Pending Review	Approved	U104	2026-07-27 11:15	Reviewer approved request.
SH009	Q005		Pending Review	U005	2026-07-27 13:05	Request submitted.
SH010	Q005	Pending Review	Rejected	U105	2026-07-28 14:30	Reviewer rejected request.

Audit Log

Security, administrative, collection, and workflow audit events. PK: audit_id. FK: user_id → Users.user_id.

audit_id (PK)	user_id (FK)	action_type	entity_name	entity_id	event_at	result	details
A001	U001	LOGIN	Session	S001	2026-07-29 08:00	Success	UID authenticated.
A002	U005	LOGIN	Session	S005	2026-07-28 20:02	Failure	Inactive or invalid UID response.
A003	U204	UPDATE	Housing Budget	B003	2026-07-02 10:30	Success	Budget updated from $1,800 to $1,900.
A004	U203	COLLECT	Residence Listing	L001	2026-07-29 05:10	Success	Listing collected and standardized.
A005	U001	INSERT	Residence Request	Q001	2026-07-24 10:15	Success	Pending review request created.
A006	U103	UPDATE	Residence Request	Q003	2026-07-26 16:45	Success	Request approved.
A007	U105	UPDATE	Residence Request	Q005	2026-07-28 14:30	Success	Request rejected.
A008	U202	UPDATE	Role	R05	2026-02-01 09:00	Success	Listing Manager role activated.
A009	U203	VERIFY	Approved Website	W005	2026-07-23 15:00	Success	Source URL verified.
A010	U201	REPORT	Audit Log	AUD-0729	2026-07-29 14:00	Success	Daily audit report generated.

Notifications

Decision notifications sent to athletes and tracked for delivery outcome. PK: notification_id. FK: request_id → Residence_Requests.request_id; athlete_id → Athletes.athlete_id.

notification_id (PK)	request_id (FK)	athlete_id (FK)	delivery_channel	recipient_email	message_type	sent_at	delivery_status	external_message_id
N001	Q001	U001	Email	jordan.lee@university.edu	Review Pending	2026-07-24 10:16	Delivered	MSG-70001
N002	Q002	U002	Email	maya.thompson@university.edu	Review Pending	2026-07-25 09:31	Delivered	MSG-70002
N003	Q003	U003	Email	ethan.brooks@university.edu	Approval	2026-07-26 16:46	Delivered	MSG-70003
N004	Q004	U004	Email	sofia.ramirez@university.edu	Approval	2026-07-27 11:16	Failed	MSG-70004
N005	Q005	U005	Email	noah.williams@university.edu	Rejection	2026-07-28 14:31	Delivered	MSG-70005

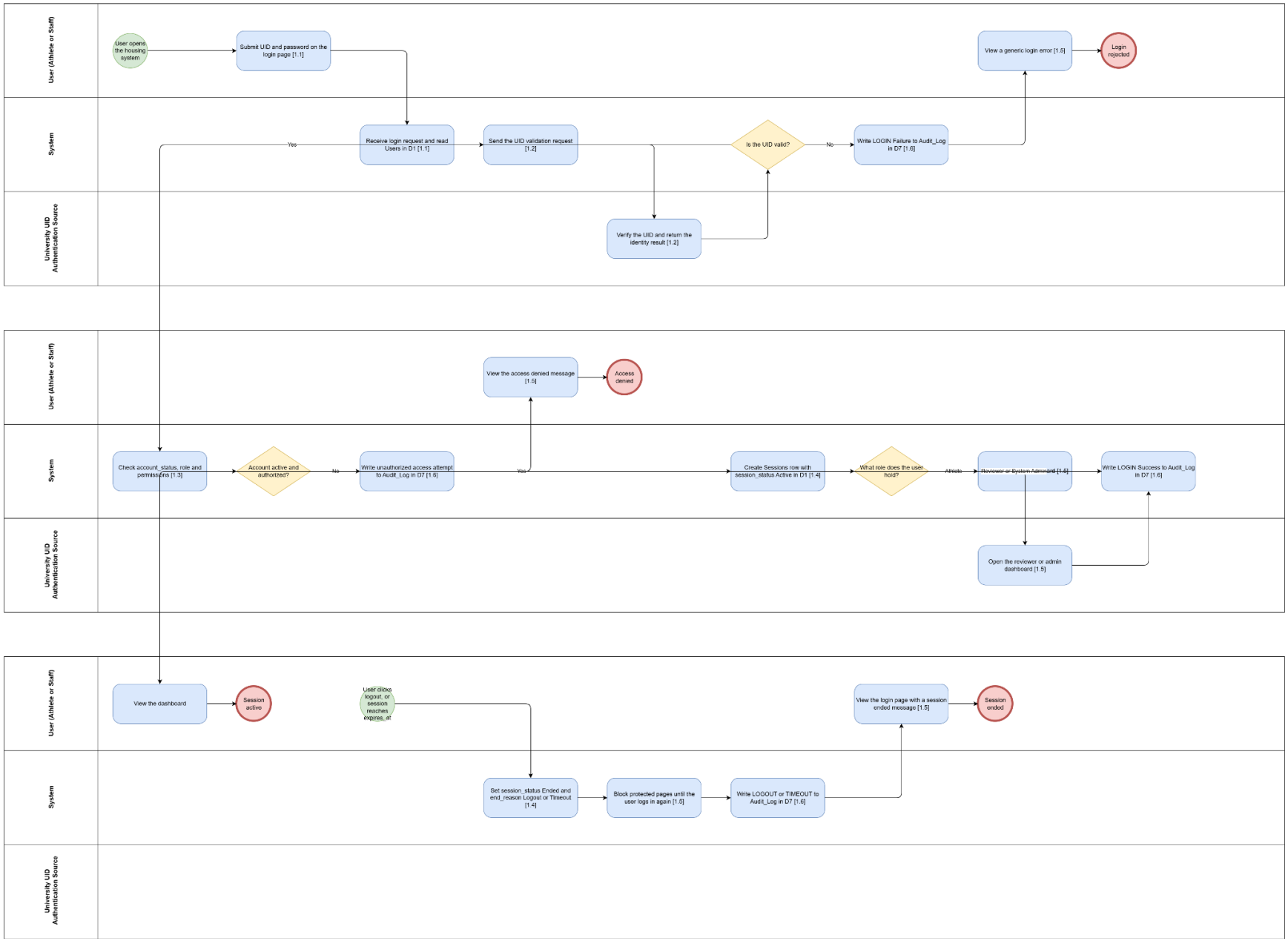
Validation Checks

Programmatic PK uniqueness, FK referential integrity, and specialization checks performed before export.

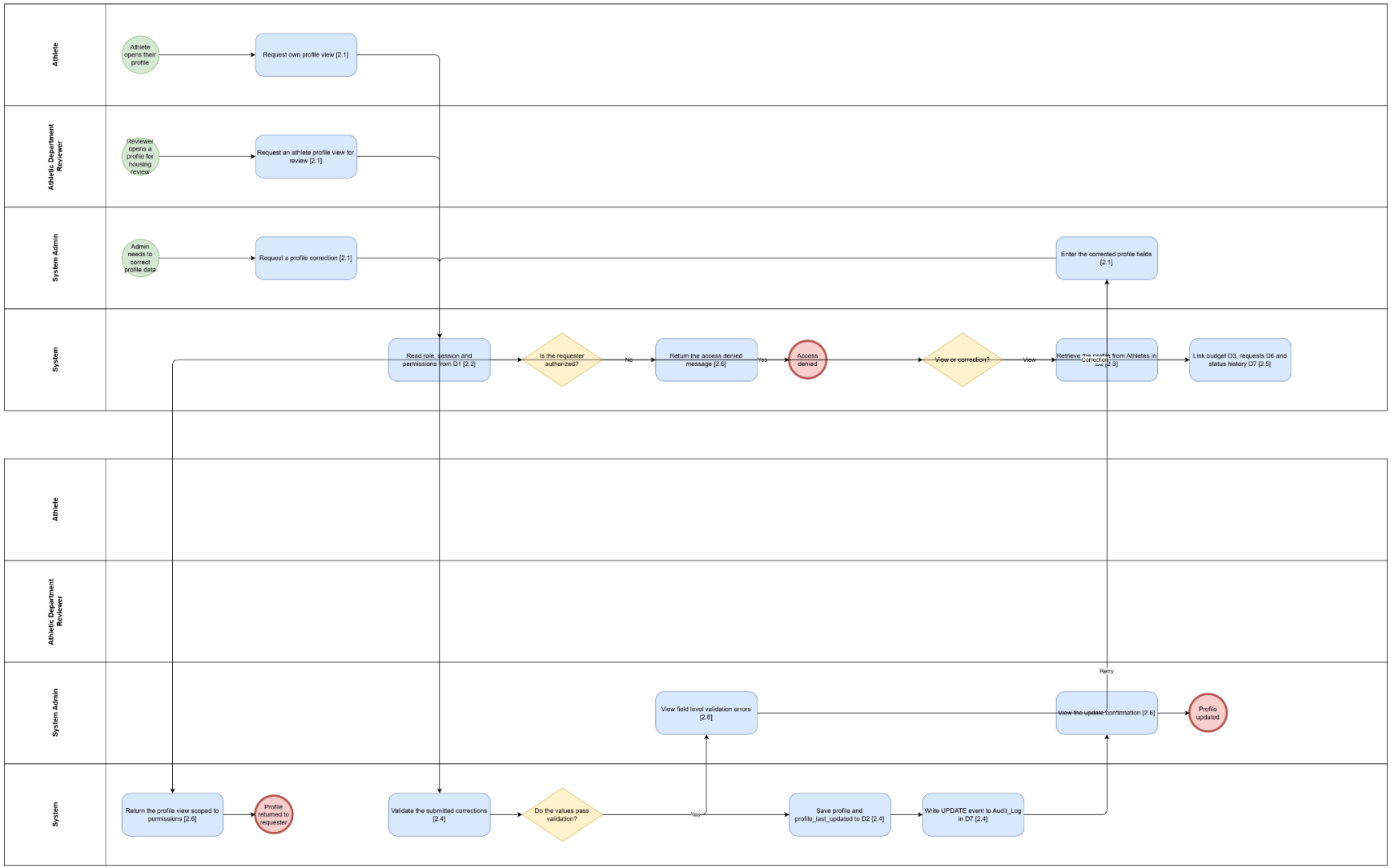
Check	Parent / Rule	Rows Tested	Orphans / Violations	Result
Athletes.athlete_id	Users.user_id	5	0	PASS
Reviewers.reviewer_id	Users.user_id	5	0	PASS
System_Admins.admin_id	Users.user_id	5	0	PASS
User_Roles.user_id	Users.user_id	18	0	PASS
User_Roles.role_id	Roles.role_id	18	0	PASS
Role_Permissions.role_id	Roles.role_id	13	0	PASS
Role_Permissions.permission_id	Permissions.permission_id	13	0	PASS
Sessions.user_id	Users.user_id	8	0	PASS
Housing_Budgets.athlete_id	Athletes.athlete_id	5	0	PASS
Budget_Changes.budget_id	Housing_Budgets.budget_id	7	0	PASS
Budget_Changes.changed_by_admin_id	System_Admins.admin_id	7	0	PASS
Approved_Websites.added_by_admin_id	System_Admins.admin_id	5	0	PASS
Residence_Listings.website_id	Approved_Websites.website_id	10	0	PASS
Residence_Requests.athlete_id	Athletes.athlete_id	5	0	PASS
Residence_Requests.listing_id	Residence_Listings.listing_id	5	0	PASS
Residence_Requests.budget_id	Housing_Budgets.budget_id	5	0	PASS
Request_Reviews.request_id	Residence_Requests.request_id	5	0	PASS
Request_Reviews.reviewer_id	Reviewers.reviewer_id	5	0	PASS
Status_History.request_id	Residence_Requests.request_id	10	0	PASS
Status_History.changed_by_user_id	Users.user_id	10	0	PASS
Audit_Log.user_id	Users.user_id	10	0	PASS
Notifications.request_id	Residence_Requests.request_id	5	0	PASS
Notifications.athlete_id	Athletes.athlete_id	5	0	PASS
Users total specialization	Every Users.user_id exists in exactly one subtype	15	0	PASS
Users disjoint specialization	Athlete, Reviewer, and System Admin subtype key sets do not overlap	15	0	PASS
Primary key uniqueness	All entity primary keys unique	144	0	PASS
Minimum sample rows	Every entity table contains at least five rows	18	0	PASS

Activity Diagrams

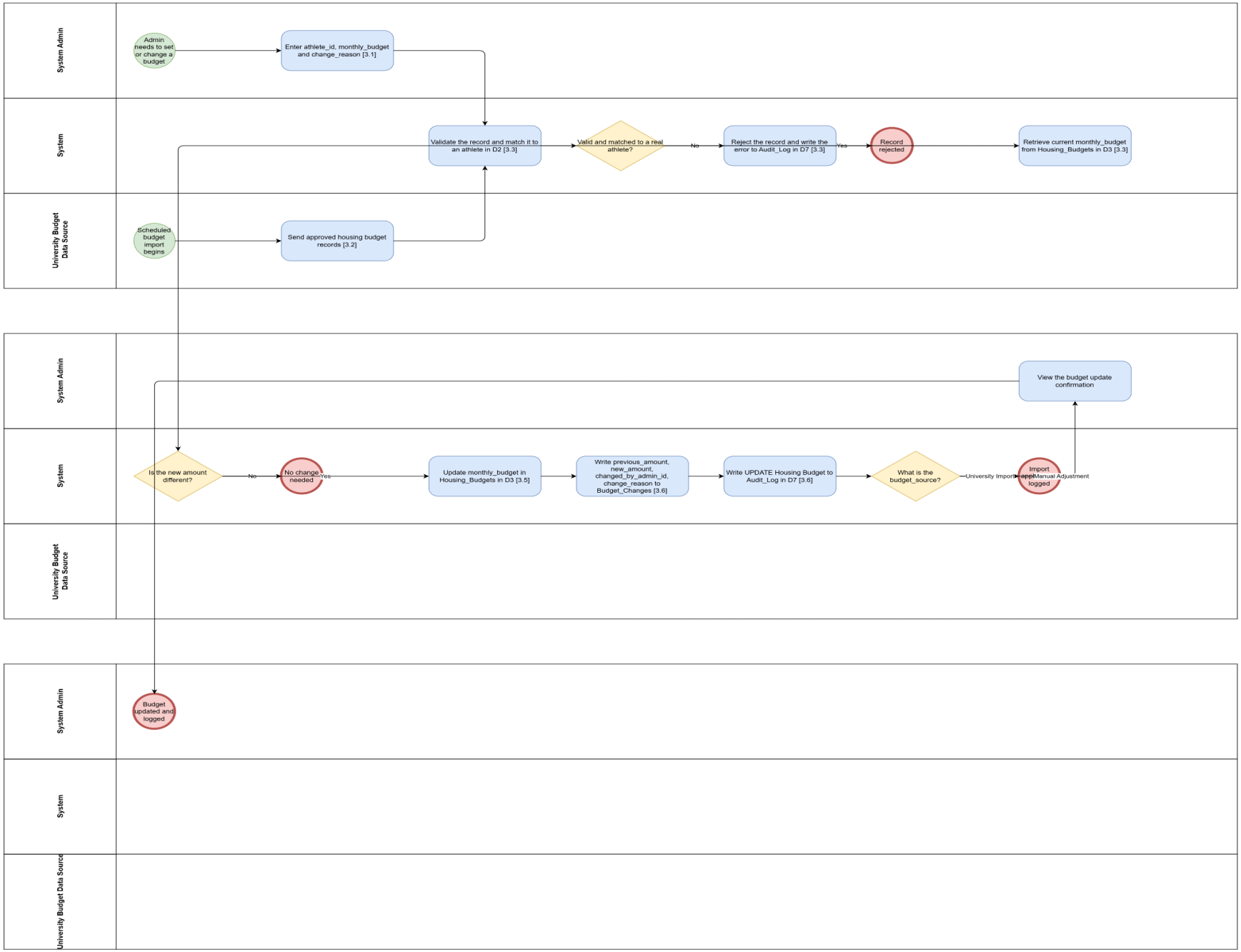
Activity Diagram 1.0 Authenticate User and Manage Session



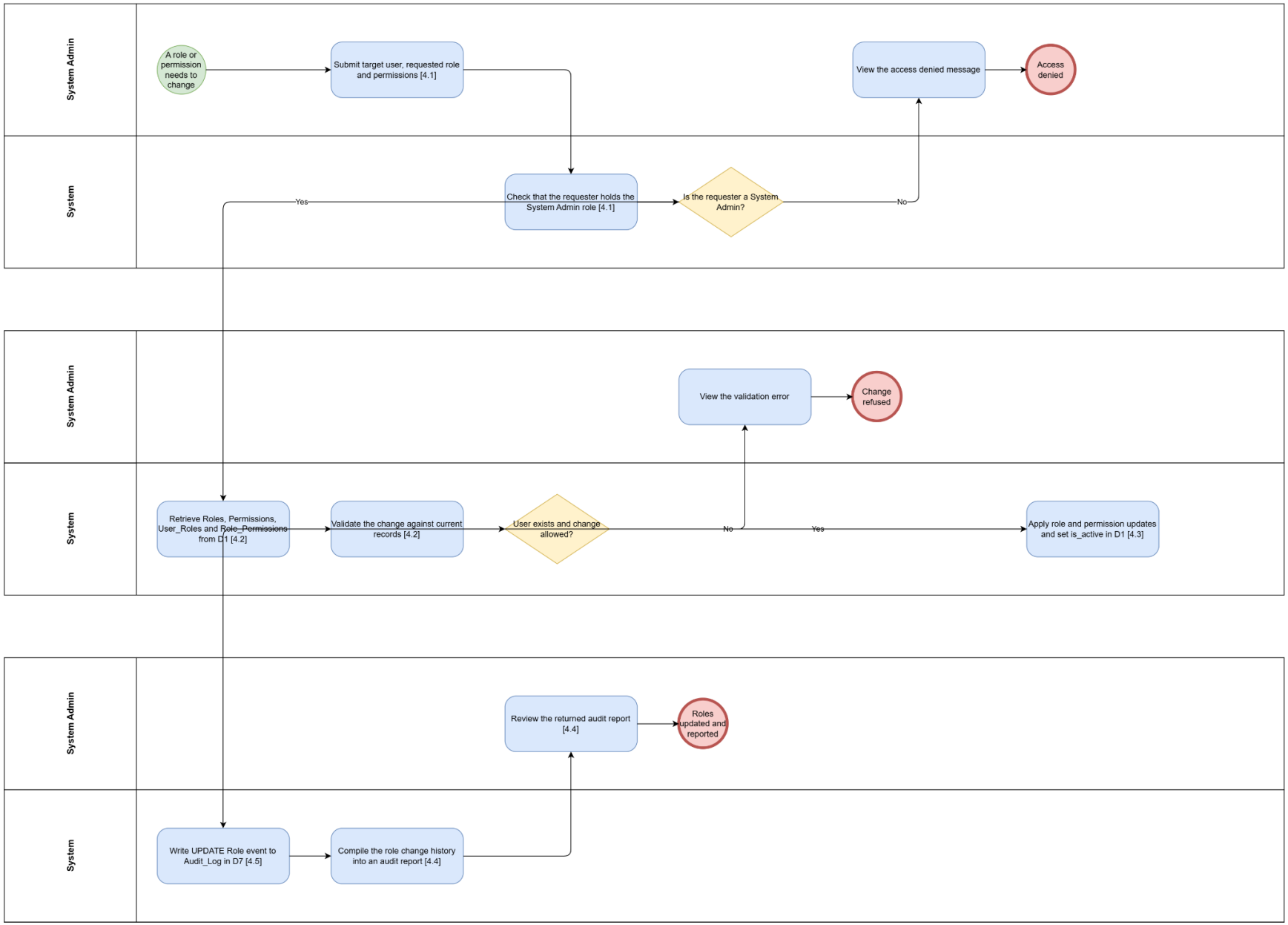
Activity Diagram 2.0 Manage Athlete Profile



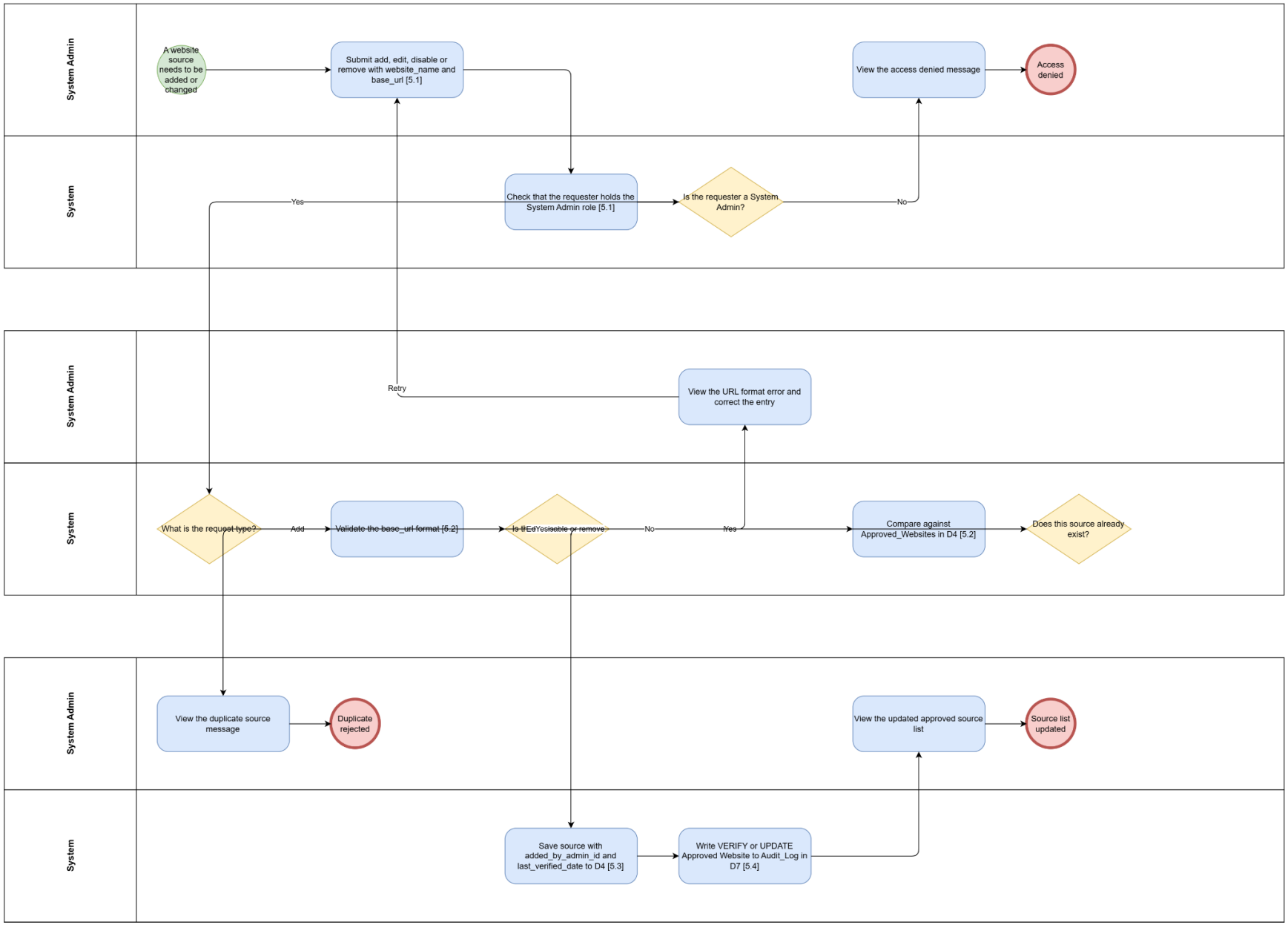
Activity Diagram 3.0 Manage Housing Budget



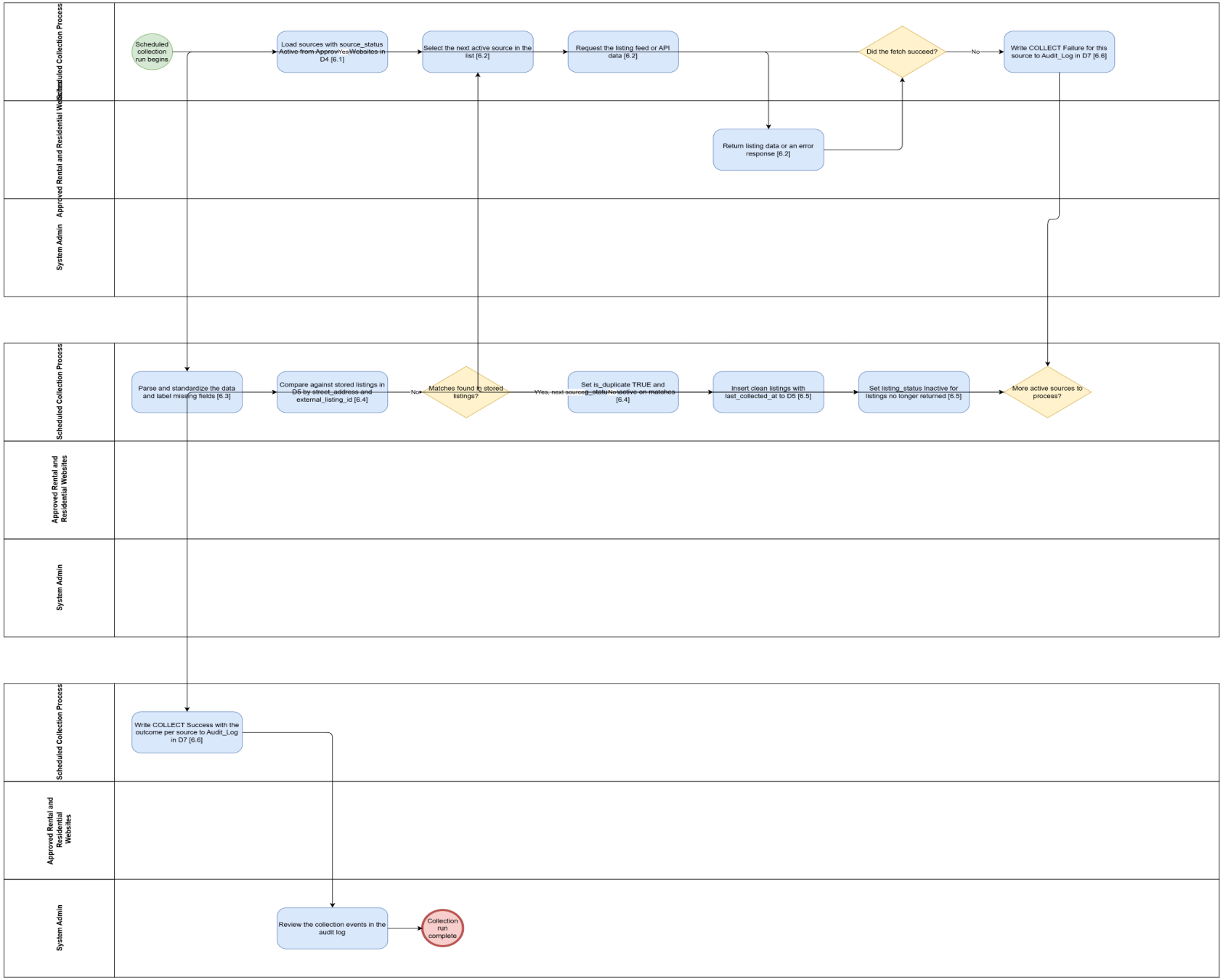
Activity Diagram 4.0 Manage Roles, Permissions and Security



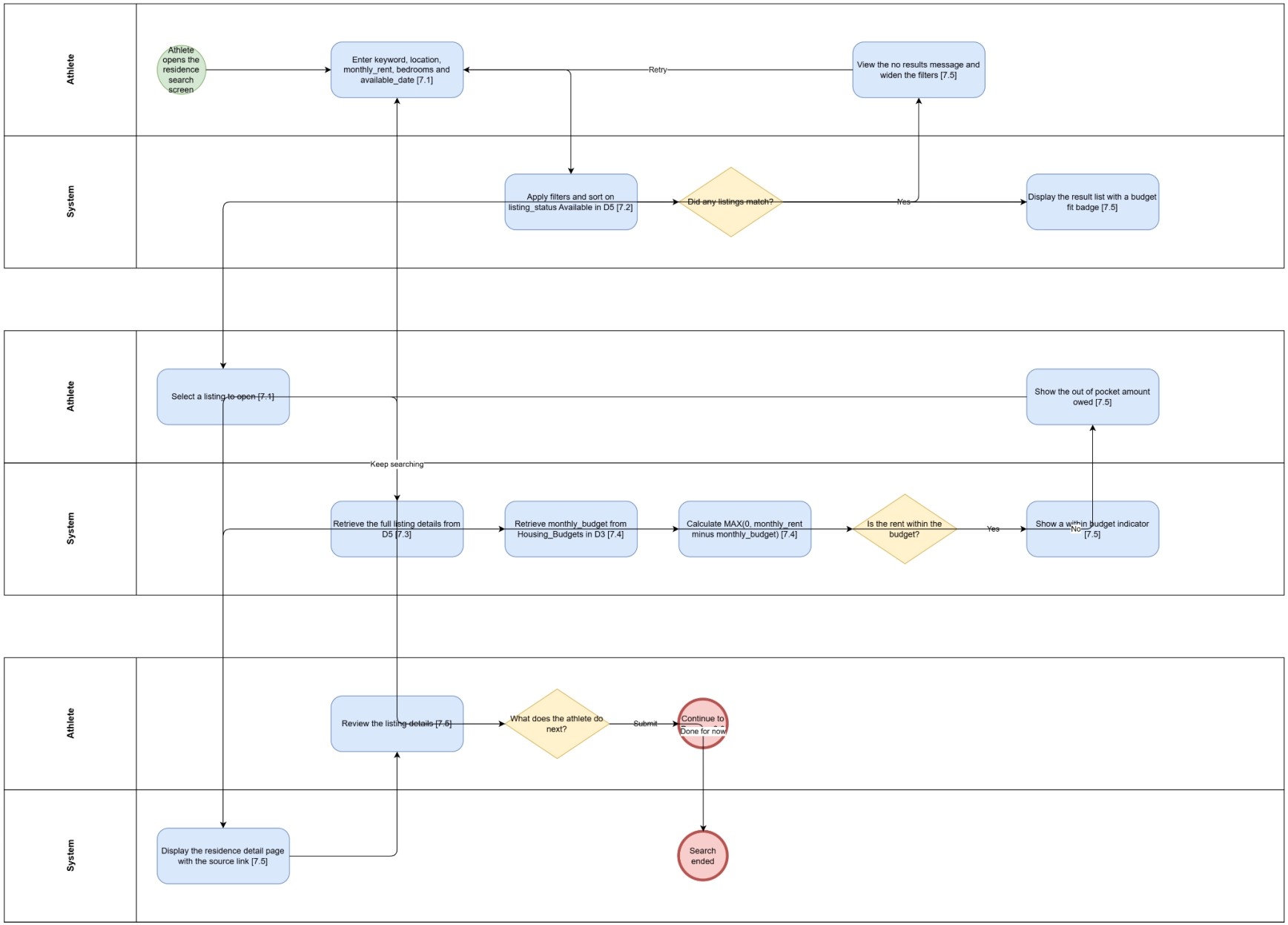
Activity Diagram 5.0 Manage Approved Rental Websites



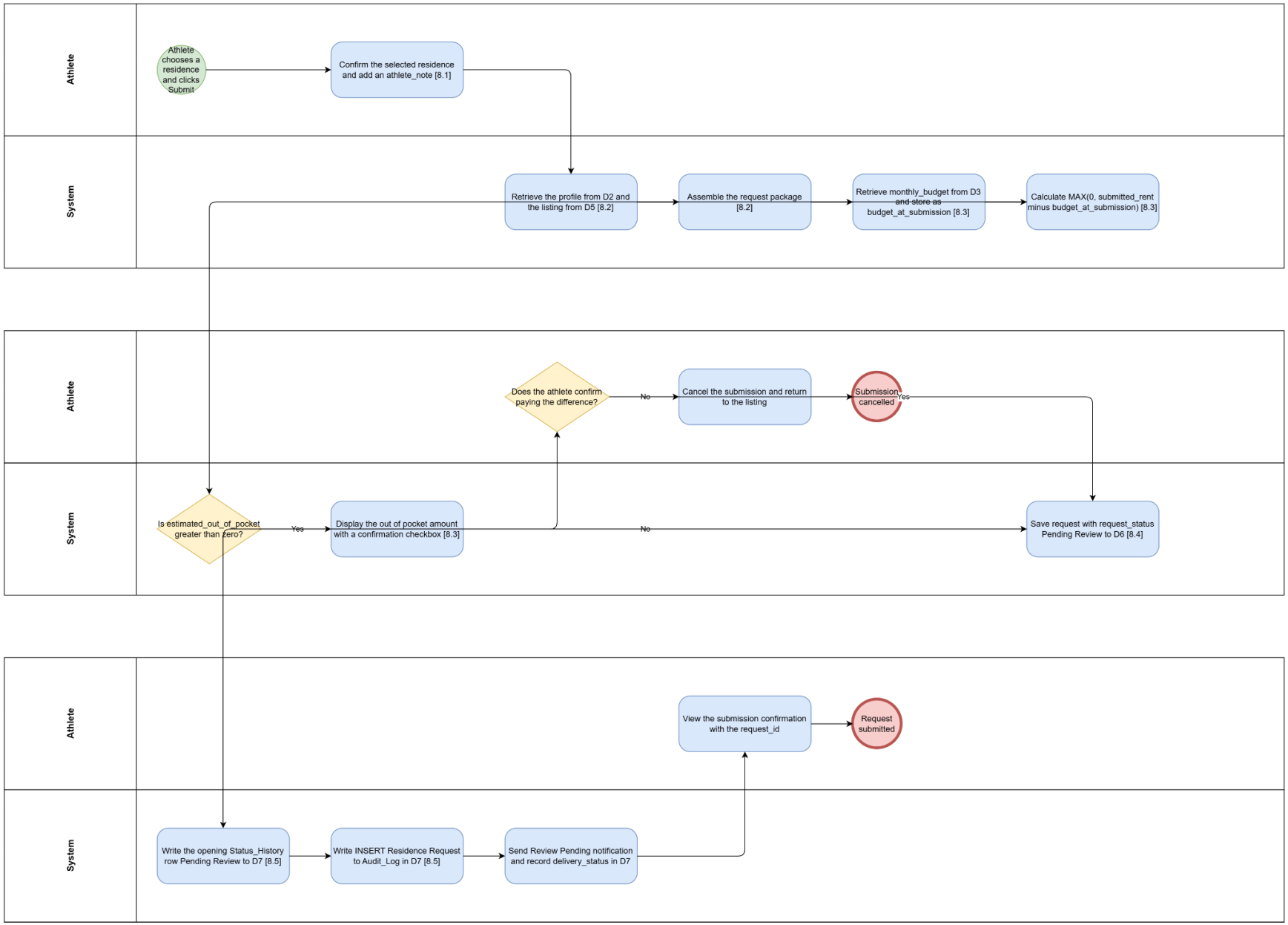
Activity Diagram 6.0 Collect and Clean Residence Listings



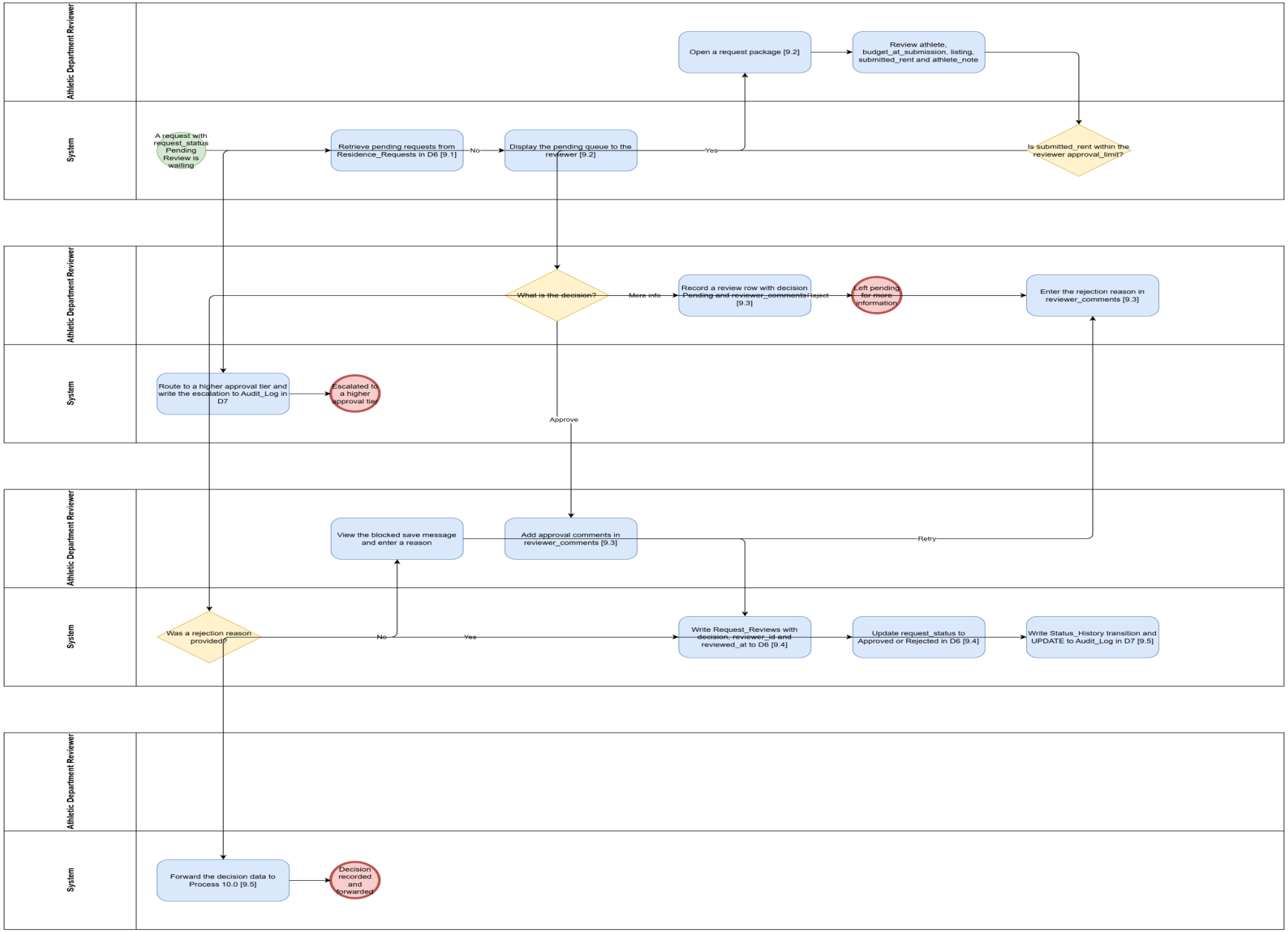
Activity Diagram 7.0 Search Listings and View Details



Activity Diagram 8.0 Submit Residence Request



Activity Diagram 9.0 Review Request and Record Decision



Activity Diagram 10.0 Notify Athlete and Track Status

